FPT ACADEMY INTERNATIONAL

FPT – APTECH COMPUTER EDUCATION

MediFlow

A Microservices – Based Distributed Hospital Management System

Supervisor: *HOÀNG ĐỨC QUANG*

Semester: *04*

Batch No: *T2.2410.E0*

Group No: *04*

Order:	**Full name**	**Roll No.**
1.	*Phạm Đăng Vinh*	*Student1614910*
2.	*Lê Quang Huy*	*Student1394600*
3.	*Trần Hoàng Anh*	*Student1614884*
4.	*Thân Thế Lộc*	*Student1571994*

***Month**: 10 **Year**: 2026*

This is to certify that

Mr. PHAM DANG VINH

Mr. LE QUANG HUY

Mr. TRAN HOANG ANH

Mr. THAN THE LOC

Have successfully Designed & Developed

MEDIFLOW

A MICROSERVICES – BASED DISTRIBUTED HOSPITAL MANAGEMENT SYSTEM

Submitted by:

Mr. HOANG DUC QUANG

Date Of Issue:

Authorized Signature:

Content

Content **3**

Acknowledge **4**

Synopsis **6**

Review 01 ..... **7**

Problem Definition 8

Customer Requirements Specifications 11

Review 02 ..... **24**

Architecture Design 25

Data Flow Diagram 51

Entity Relationship Diagram 53

Table Design 59

SiteMap 93

Algorithm 95

Review 03 ..... **100**

Screen shots 101

Testing document 183

Final checklist 195

Tasksheet 201

Acknowledgments and Declaration **207**

Acknowledge

As we approach the completion of our **MediFlow – Microservices-Based Distributed Hospital Management System** project, we are filled with gratitude, pride, and a sense of accomplishment. This project has been a valuable opportunity to apply our academic knowledge, strengthen our technical skills, and gain practical experience in developing complex software systems.

Heartfelt Gratitude to Our Mentors

First and foremost, we would like to express our sincere gratitude to the **Project Team at FPT Aptech** for providing us with the knowledge, resources, and opportunity to undertake this meaningful project.

We are especially thankful to our lecturer, **Mr. HOANG DUC QUANG**, for his dedicated guidance, patience, and valuable feedback throughout the development process. His mentorship has helped us overcome technical challenges, improve our problem-solving abilities, and develop a more professional approach to software engineering.

Appreciation for Our Team and Families

We extend our heartfelt appreciation to every member of **Team 04** for their dedication, teamwork, and perseverance. Through close collaboration, we have explored and developed a distributed healthcare platform encompassing ten business domains: **Organization, Patient, Clinical, Laboratory, Pharmacy, Billing, Notification, Reporting, Inpatient, and Surgery**.

We are equally grateful to our families, friends, and classmates for their continuous encouragement and understanding throughout this journey.

A Meaningful Learning Experience

The **MediFlow Project** has provided us with valuable experience in microservices architecture, distributed systems, database design, event-driven communication, and cross-platform application development. By working with technologies such as **Spring Boot, Spring Cloud, RabbitMQ, PostgreSQL, Next.js, and Flutter**, we have gained a deeper understanding of modern software engineering practices and the challenges of integrating independent healthcare services.

Beyond technical knowledge, this project has strengthened our communication, adaptability, responsibility, and collaborative problem-solving skills.

Looking Forward

As we conclude this important milestone, we believe the knowledge and experience gained from MediFlow will serve as a strong foundation for our future careers in information technology.

Once again, we sincerely thank everyone who has supported and contributed to our journey.

With heartfelt gratitude,

TEAM 04

MediFlow – Microservices-Based Distributed Hospital Management System

Synopsis

MediFlow – Microservices-Based Distributed Hospital Management System is a healthcare management project designed to address the fragmentation of patient information, hospital operations, and medical workflows across different departments. The system aims to improve coordination, streamline administrative processes, and support efficient healthcare service delivery through a distributed software architecture.

Built on a **microservices-based and event-driven architecture**, MediFlow encompasses ten business domains: **Organization, Patient, Clinical, Laboratory, Pharmacy, Billing, Notification, Reporting, Inpatient, and Surgery**. These services are designed to manage distinct healthcare responsibilities while enabling coordinated workflows, including patient registration, appointment scheduling, clinical examinations, laboratory testing, prescription management, payment processing, medication dispensing, inpatient care, and surgical procedures.

The project utilizes **Java Spring Boot, Spring Cloud Gateway, Netflix Eureka, RabbitMQ, and PostgreSQL** for backend development, service discovery, asynchronous communication, and data management. **Next.js and Flutter** are used to develop web and mobile applications, providing role-based interfaces for healthcare professionals and administrative personnel.

MediFlow follows essential software engineering principles, including **Domain-Driven Design, Clean Architecture, database-per-service ownership, RESTful API communication, and event-driven integration**. These approaches promote modularity, maintainability, scalability, and separation of responsibilities across independent services.

As an academic project under active development, MediFlow demonstrates the practical application of distributed systems and modern software engineering technologies in the healthcare domain. Beyond its functional scope, the project provides valuable experience in system architecture, collaborative development, integration testing, and solving complex real-world business problems.

Keywords: Hospital Management System, Healthcare Information System, Microservices, Distributed Systems, Event-Driven Architecture, Spring Boot, RabbitMQ, Next.js, Flutter.

Review 01

Problem Definition

1. Introduction and Context

MediFlow coordinates departments throughout an outpatient journey: reception, examination, laboratory tests, prescribing, payment, and dispensing. Records share a patient identity while each department remains responsible for its own data and business decisions. Users access the web application through the API Gateway.

The project focuses on an outpatient workflow that can be verified end to end, including message redelivery and temporary service outages

2. Problem Identification

- Disconnected patient information, appointments, and clinical records cause repeated entry and make progress difficult to track.
- Doctors, the laboratory, and the pharmacy lack consistent handoff status, increasing the risk of missed orders or results.
- Payment status is not clearly tied to each visit, test, or prescription, making service authorization difficult to control.
- Poor coordination of stock, reservations, and dispensing can cause duplicate dispensing or insufficient stock.
- Successful payment followed by dispensing failure requires traceable compensation rather than a lost transaction.
- Reports and notifications lag behind operations; users struggle to distinguish missing data from a measured zero.
- Medical and financial information requires access restrictions based on role, work scope, and record ownership.

	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

3. Project Objectives

1. Link identity, appointments, visits, tests, prescriptions, and payments through explicit references.
2. Enforce the relevant business and financial conditions before examination, test processing, and dispensing.
3. Prevent duplicate financial or stock effects when requests or events are processed again.
4. Update reports and notifications from events while preserving fault tracing and recovery.
5. Define acceptance criteria for successful workflows, access denial, redelivery, and partial failure.

4. Project Scope

Service	Responsibility
Core	Accounts, staff, departments, reference room data, and patient identity
Clinical	Appointments, check-in, records, diagnosis, orders, and outpatient completion
Lab	Accept test requests, track processing, and return results
Pharmacy	Drug catalogue, stock, prescriptions, reservations, and dispensing
Billing	Charges, invoices/payment requests, payments, service clearance, and compensation/refunds
Support	Reporting read models and in-app notifications

Gateway and Eureka are infrastructure; common is a shared library. Core and Support co-host modules while retaining each context's data and business boundaries. The design retains eight active context stores; six services do not imply six databases.

5. Users and Stakeholders

The eight human roles are ADMIN, DOCTOR, NURSE, PHARMACIST, CASHIER, LAB_TECH, MANAGER, and PATIENT. SYSTEM is a service identity for automated processing, not a human login account. Functions for each role are listed in B1.

Stakeholders include patients, reception staff, doctors, the laboratory, the pharmacy, cashiers, management, and the development team. Departments agree on requirements and verify outcomes within their authorized responsibilities.

6. Proposed Solutions

The Next.js/TypeScript/Tailwind web application accesses the backend through Gateway. The backend uses Java 21, Spring Boot, Maven, PostgreSQL, RabbitMQ, and Eureka. Each context owns its data; contexts do not query each other's databases. REST supplies information needed immediately; events support handoffs and asynchronous updates. Outbox delivery, duplicate protection, bounded retries, and dead-letter queues support recovery.

The main workflow is identity → check-in → examination payment → examination → tests if needed → prescription if needed → medication payment → dispensing → outpatient completion → report/notification updates. When tests are ordered, their fees and authorization are checked before protected laboratory actions.

Customer Requirements Specifications

1. Functional Requirements

The following functions are fucntional requirements. Each action must respect endpoint permissions, clinical authority, and record access boundaries. A role does not grant access to every record. This list neither adds APIs nor expands default permissions.

1.1. ADMIN — Administrator

- Manage accounts, roles, active status, and account locking.
- Manage departments, staff, and reference data required for outpatient care.
- Link accounts to staff or patients using an authoritative identity source.
- Inspect system status, history, and processing failures within permitted administration scope.
- Perform authorized recovery operations with an audit trail, without bypassing financial or clinical conditions.

1.2. DOCTOR — Doctor

- View appointments and patient records within the assigned scope.
- Start an examination only with valid EXAM clearance for the exact visit; emergency exceptions use only the supported, audited mechanism.
- Record examination findings, diagnosis, and outpatient disposition.
- Create explicit laboratory orders and view results returned for the corresponding record.
- Author prescription content; Pharmacy validates and owns prescriptions, prices, stock, and reservations.

- Follow test results and dispensing status, and complete the outpatient record when its conditions are met.

1.3. NURSE — Nurse / Reception Support

- Search, create, and update patient information within granted permissions.
- Assist with appointment registration and departmental schedule tracking.
- Perform check-in, update permitted reception status, and hand patients over to doctors.
- View visit information and processing progress needed for care coordination.
- Check-in does not itself authorize examination or replace financial verification.

1.4. LAB_TECH — Laboratory Technician

- View valid existing test requests and permitted related information.
- Start test processing only with unexpired LAB_TEST clearance matching the exact test.
- Update processing status and cancel when the workflow permits.
- Record a non-empty result with a valid performed date and complete the test.

- Publish results to relevant departments through the Lab workflow.

1.5. PHARMACIST — Pharmacist

- Manage and search the drug catalogue, stock, available quantities, and expiry information.
- Receive doctor-authored prescription content and verify patient identity and outpatient clinical context.
- Create/process permitted prescriptions using server catalogue prices and line-level reservations.
- Check PRESCRIPTION clearance for the exact prescription, reservation validity, and stock before dispensing.
- Confirm dispensing; apply the stock effect once only when all lines satisfy the conditions.
- Cancel or expire prescriptions through the supported workflow, release reservations, and report failures to Billing when financial adjustment is required.
- A payment receipt does not automatically replace dispensing authorization in the target workflow.

1.6. CASHIER — Cashier

- Find charges belonging to the exact visit/patient within permitted service scope.

- Create invoices or payment requests from valid charges in the same visit account.
- Record payments and inspect transaction status and corresponding documents.
- Track financial clearance for the exact examination, test, or prescription; do not manually grant access using a patient-wide paid flag.
- Process compensation/refunds through Billing's workflow and authorization when cancellation or dispensing failure requires it.
- Inspect transaction history for reconciliation; do not rewrite or delete history to force a balance match.

1.7. MANAGER — Manager

- View reports on visits, tests, prescriptions, and dispensed quantities.
- View cash received, refunds, and net cash from available authoritative sources.
- Filter reports using criteria supported by the reporting contract.
- Distinguish missing source coverage from zero, and cash received from earned revenue.
- Track report freshness without modifying source business data from reporting screens.

1.8. PATIENT — Patient

- Sign in and access information associated with their own patient identity.
- View and update permitted personal information.
- Request, view, and track their own appointments through supported workflows.
- View their own examination information, test results, and prescriptions approved for patient access.
- View their own charges, invoices, payment status, and dispensing status through available contracts.
- View in-app notifications addressed to them; do not access another patient's records or notifications.

1.9. SYSTEM — Service Identity

- Transfer valid outpatient events between contexts while preserving identifiers and traceability.
- Automatically create charges/service clearance from valid business sources in the selected profile.
- Handle redelivery, retries, and pending-work recovery without duplicate financial or stock effects.
- Update reporting read models and create in-app notifications for the correct recipients.
- Move messages that remain unprocessable after bounded retries to dead-letter handling for inspection.

- Do not substitute service tokens for human accounts on operations requiring an authenticated human actor.

2. Hardware / Software requirements

2.1. Client requirements

2.1.1. Hardware

Client profile	Minimum	Recommended
Staff desktop/laptop	64-bit dual-core CPU, 4 GB RAM, 1366×768 display, stable network	Modern 4-core CPU, 8 GB RAM, Full HD display, wired or reliable Wi-Fi
Patient browser device	Recent smartphone, tablet, or computer with at least 2 GB available memory	Device receiving current OS/browser security updates, 4 GB or more memory
Development workstation	64-bit 4-core CPU, 16 GB RAM, 30 GB free storage	8-core CPU, 32 GB RAM, SSD with 60 GB or more free storage

Client profile	Minimum	Recommended
Shared reception/lab terminal	Keyboard, pointing device, 1366×768 display, reliable network	Full HD display, barcode-capable peripherals where locally required, UPS-backed power

2.1.2. Software

Item	Requirement
Browser	Current Chrome, Edge, Firefox, or Safari with JavaScript, cookies, and HTTPS enabled
Operating system	A vendor-supported Windows, macOS, Linux, Android, or iOS release
Network	Stable access to the deployed HTTPS endpoint through approved firewall/proxy rules
Document tools	Microsoft Word, LibreOffice Writer, or compatible tool for report conversion

Item	Requirement
Mobile status	Responsive browser access is assumed; a complete native mobile client is not currently delivered

2.2. Server requirements

2.2.1. Hardware

Environment	CPU	Memory	Storage	Intended use
Developer demonstration	8 logical cores	16 GB RAM	40 GB free SSD	Local containers, selected services, frontend, and tests
Team integration	16 logical cores	32 GB RAM	150 GB SSD	Concurrent services, shared testing, sample data, and monitoring
Initial production baseline	24+ logical cores distributed across hosts	64+ GB RAM distributed	500+ GB resilient SSD plus backups	Starting point only; must be validated by performance and recovery tests

2.2.2. Software

Item	Source-aligned baseline
Java	21
Spring Boot	3.3.5
Spring Cloud	2023.0.3
Backend build	Maven multi-module
Frontend	Next.js 16.2.11, React 19.2.4, TypeScript 5, Tailwind CSS 4, pnpm
Database	PostgreSQL 16-compatible, isolated logical database per service
Messaging	RabbitMQ 3-compatible with operational management support
Discovery	Eureka Server
Containers	Docker Engine and Docker Compose compatible with repository definitions

Item	**Source-aligned baseline**
Production platform services	HTTPS certificate management, secret management, centralized logs/metrics/alerts, backups, time synchronization

Task Sheet Review 1

Project Ref. No		Project Title	Activity Plan Prepared By	Date of Preparation of Activity Plan			
No.	**Task**			**Actual Start Date**	**Actuals Days**	**Teammate Names**	**Status**
1	Problem Definition	Mediflow	Hoang Anh	15/10/2026	2	All members	Completed
2	Requirement Specification			15/10/2026	2	All members	Completed
3	Task sheet review			15/10/2026	2	All members	Completed

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

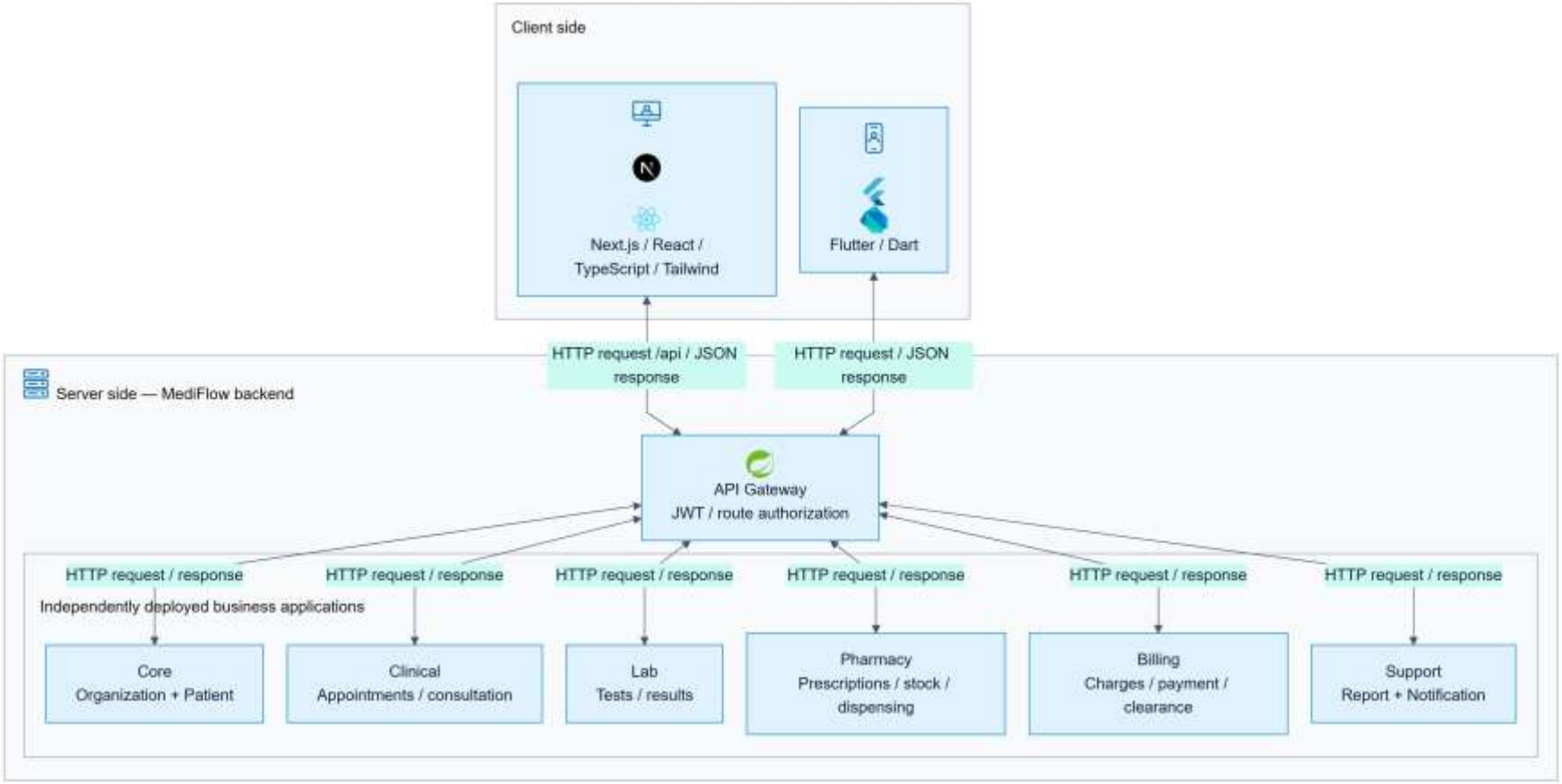
Review 02

Architecture Design

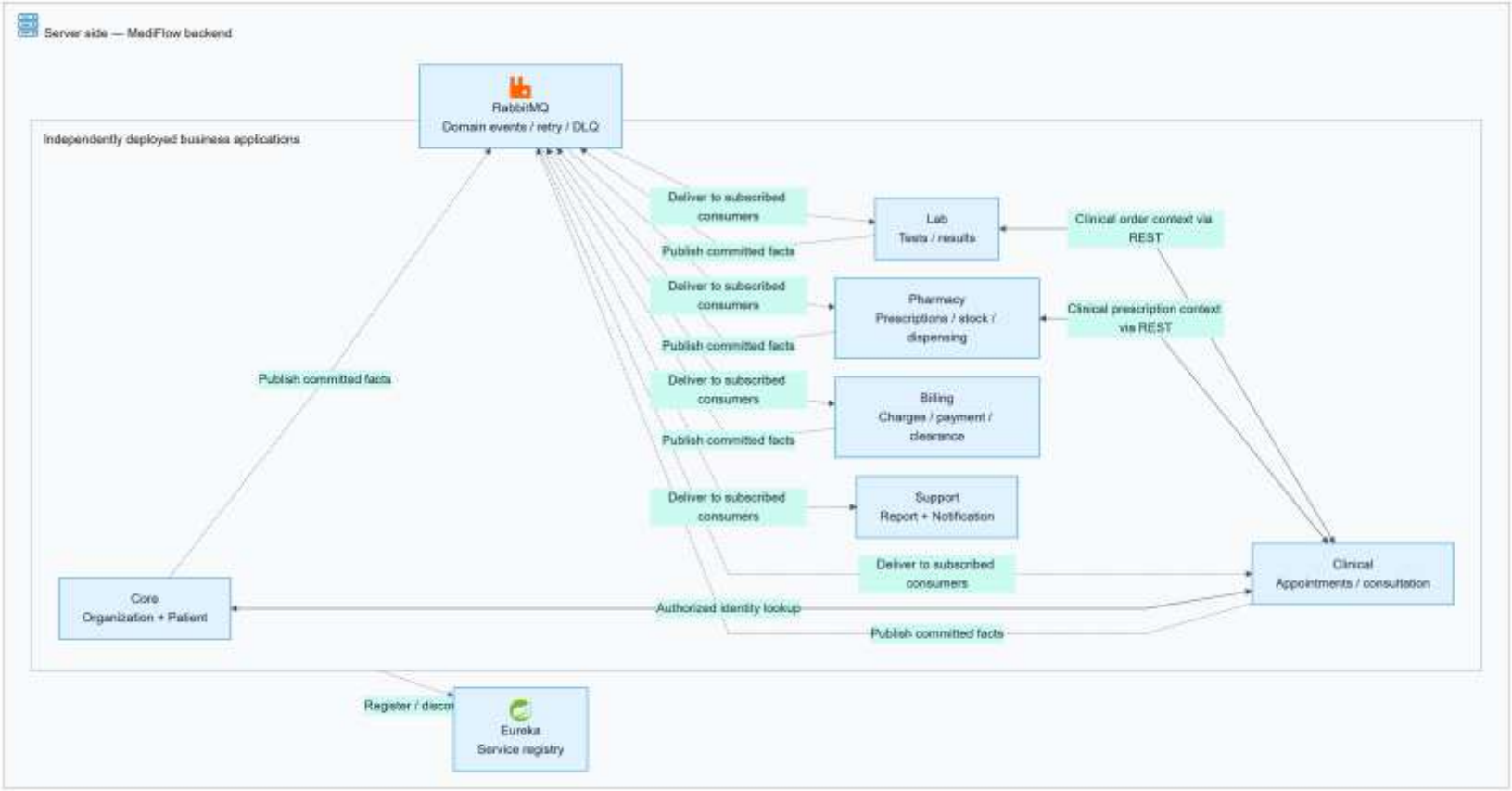
1. Architecture Design

Web calls same-origin /api/*, proxied to Gateway; Flutter calls Gateway through GATEWAY_URL. Gateway verifies JWT/route policy; downstream services enforce roles, ownership and business authority again. PostgreSQL 16 is one development container hosting separate context databases. RabbitMQ supplies per-consumer queues; cross-context SQL is prohibited.

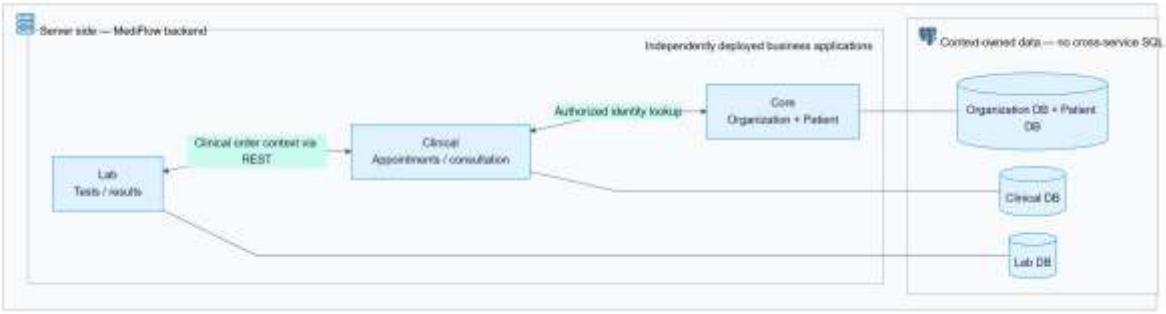
Client side and Server side are explicit boundaries. Labelled request and response arrows show synchronous HTTP traffic; double-headed edges summarize a request and its return path. Dashed event edges are asynchronous messages, not HTTP responses. The Next.js server proxy runs on Server side even though its UI executes on the client. Deployment ports are intentionally omitted.

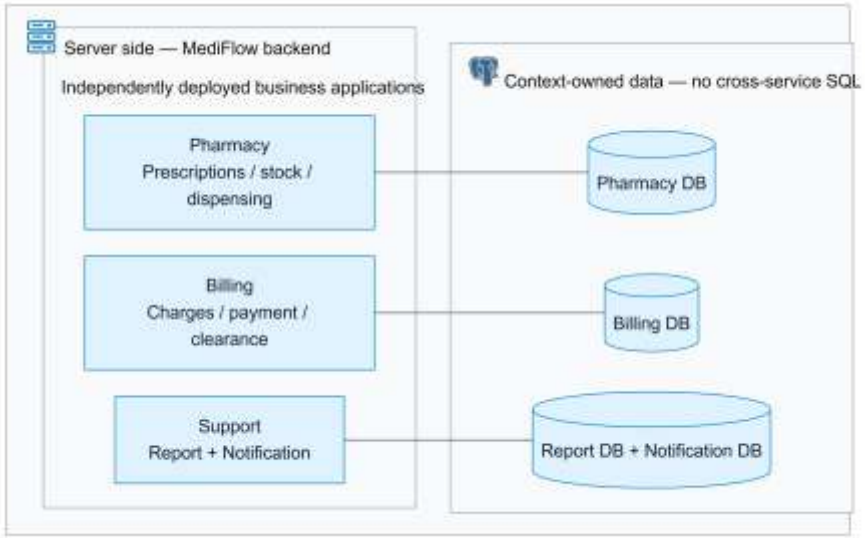


	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

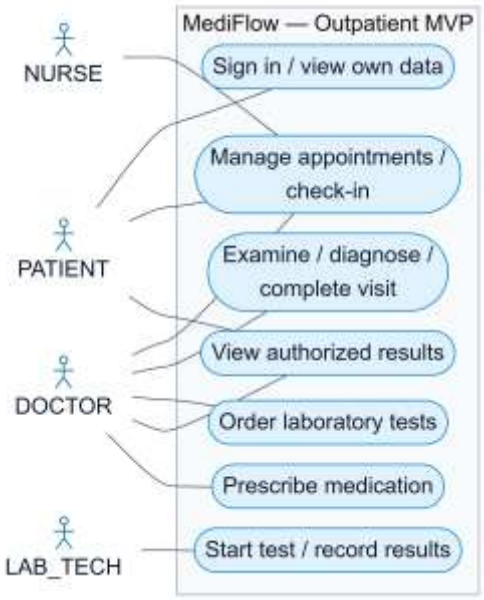


	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

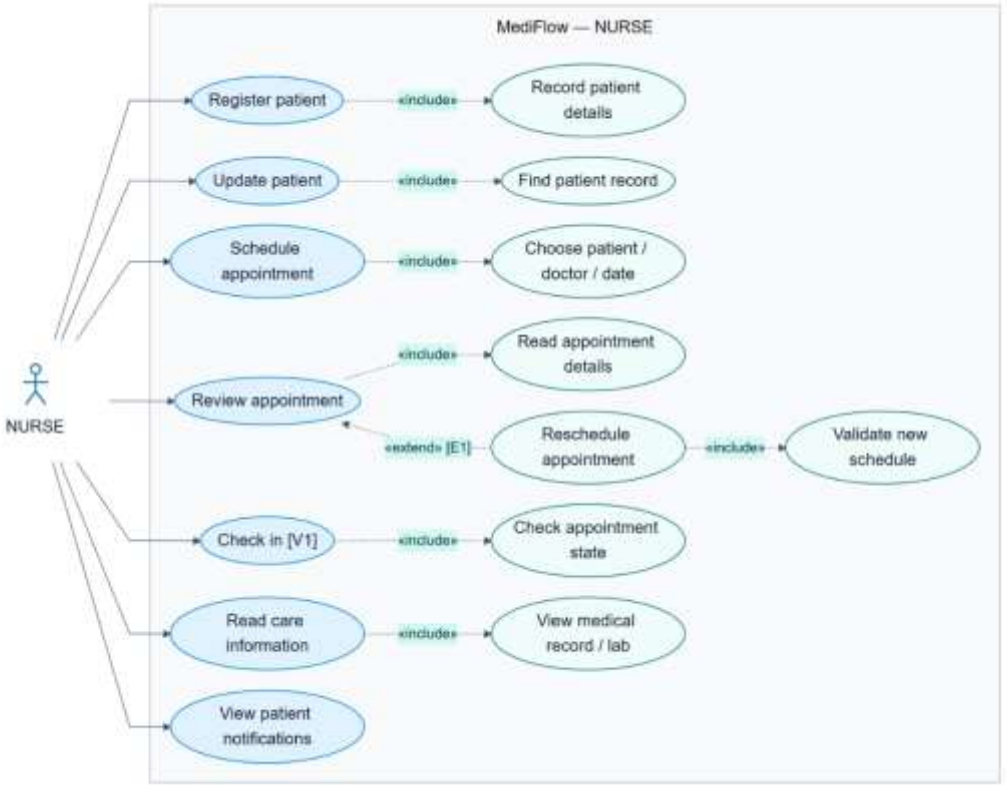




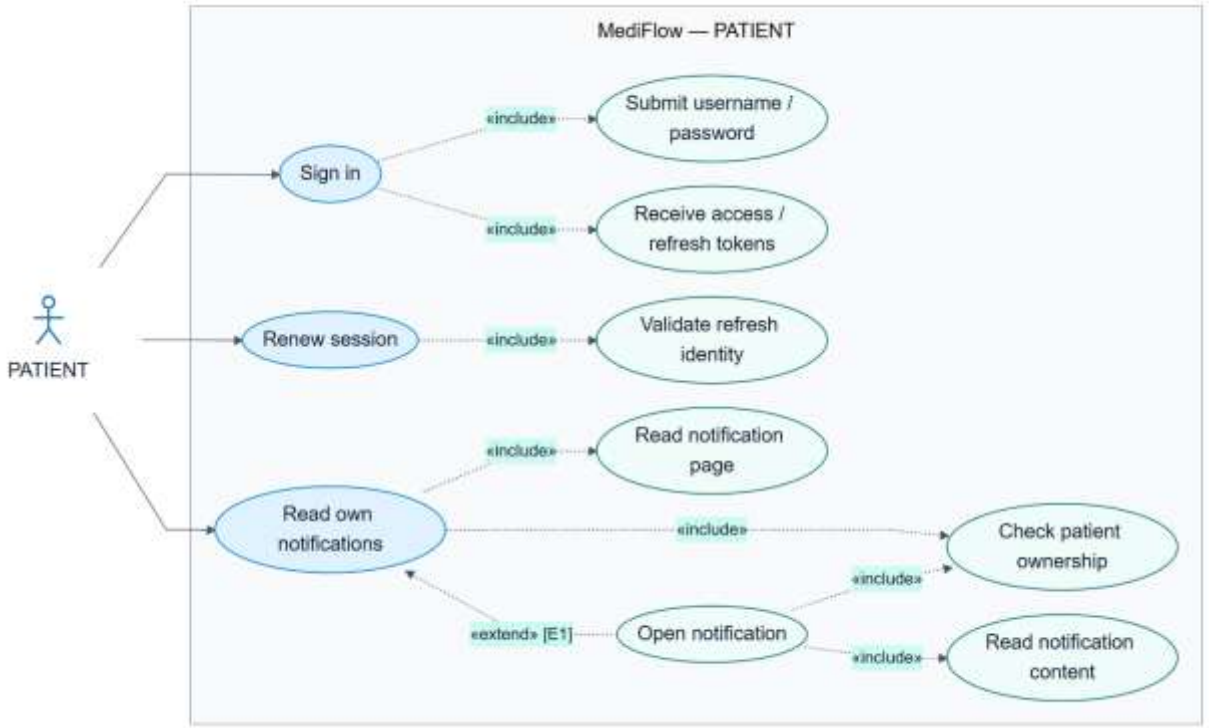
	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		


MediFlow — Outpatient MVP
NURSE
PATIENT
DOCTOR
LAB_TECH
Sign in / view own data
Manage appointments / check-in
Examine / diagnose / complete visit
View authorized results
Order laboratory tests
Prescribe medication
Start test / record results

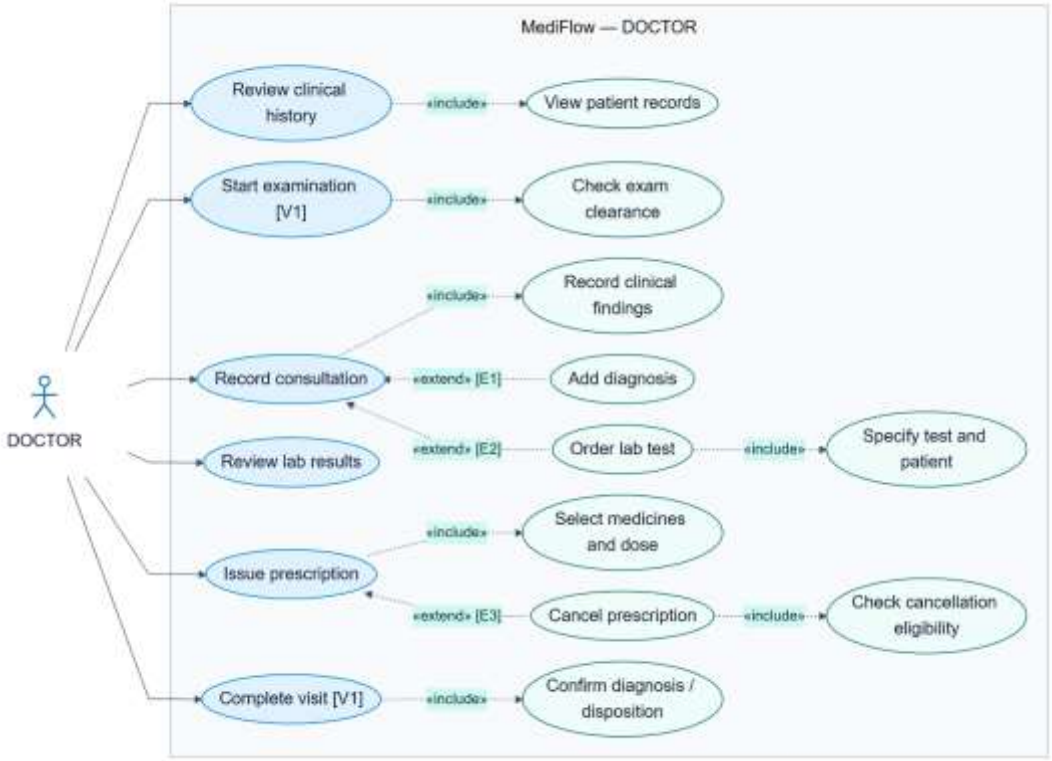
1.1. NURSE – Nurse tasks



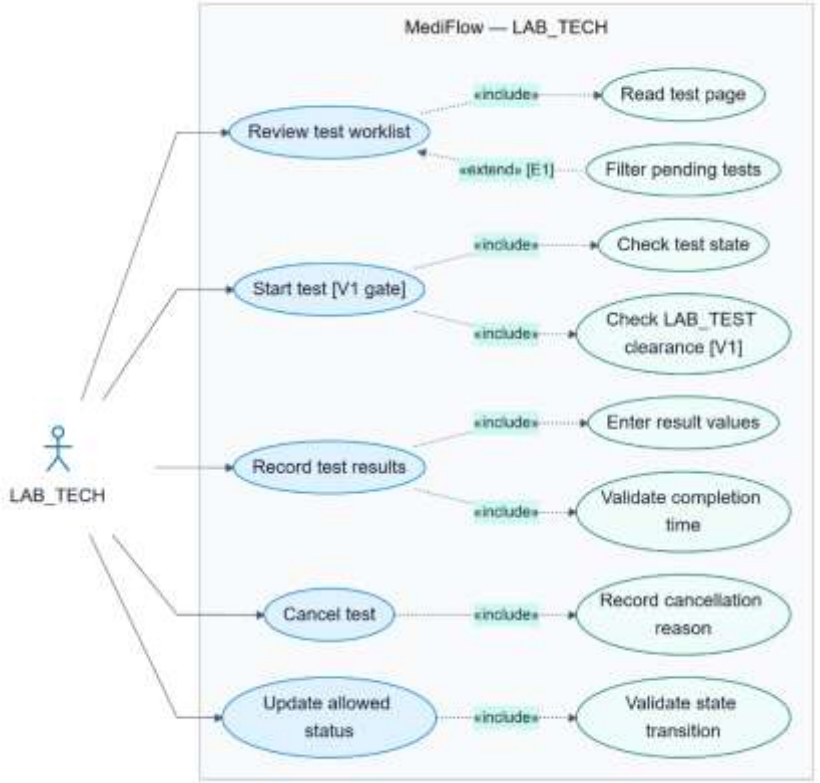
1.2. PATIENT


MediFlow — PATIENT
PATIENT
Sign in
Renew session
Read own notifications
Open notification
Submit username / password
Receive access / refresh tokens
Validate refresh identity
Read notification page
Check patient ownership
Read notification content
«include»
«extend» [E1]

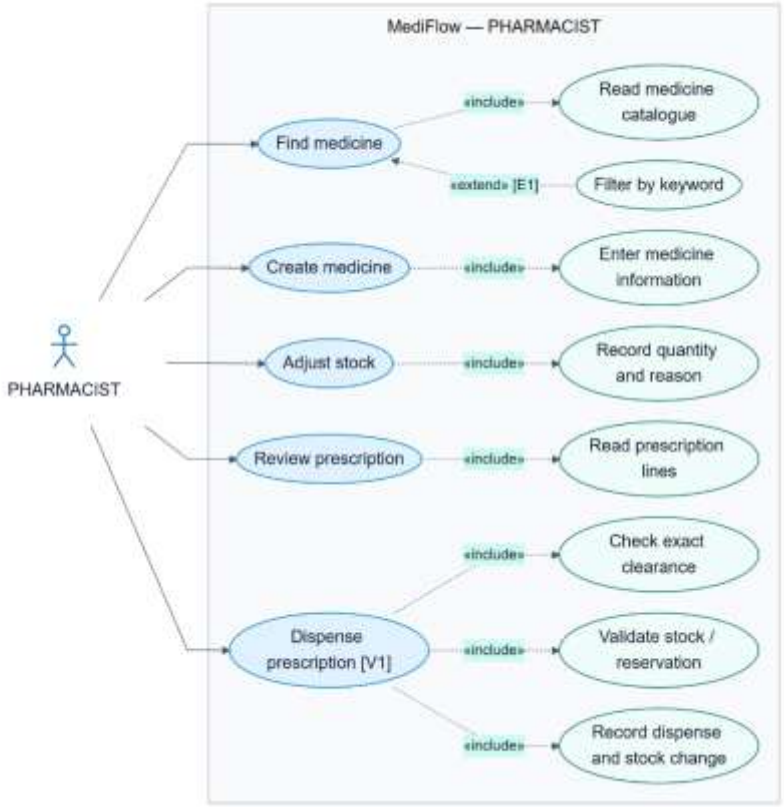
1.3. DOCTOR – Doctor tasks



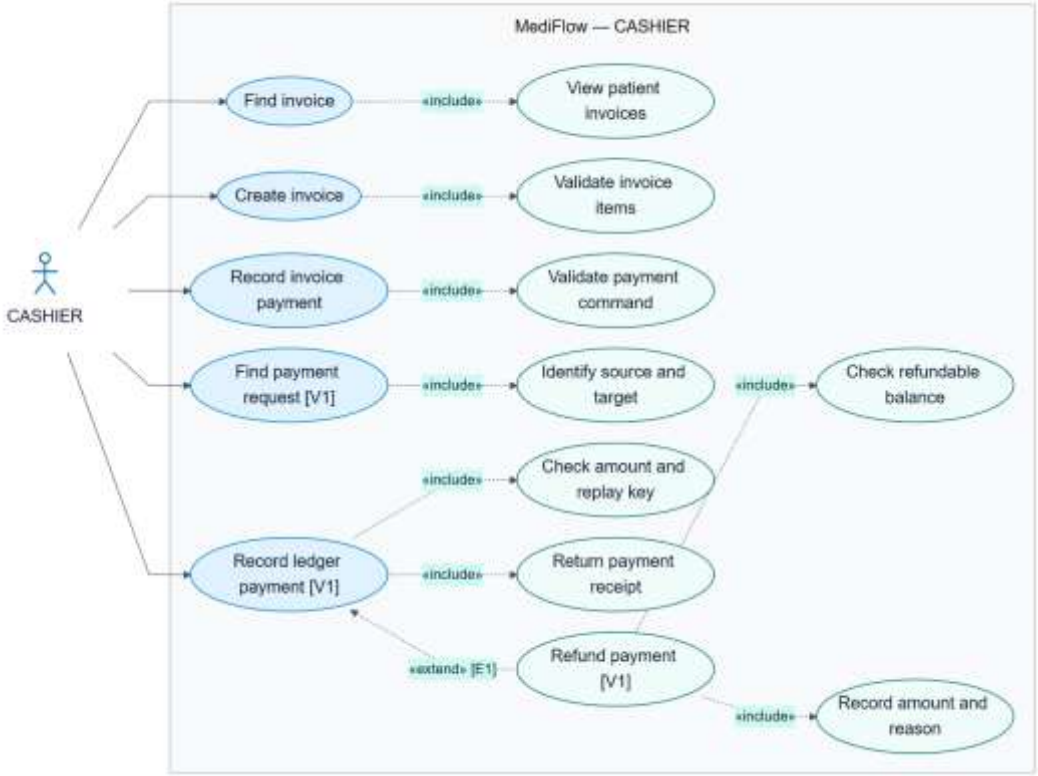
1.4. LAB_TECH – Laboratory technician tasks



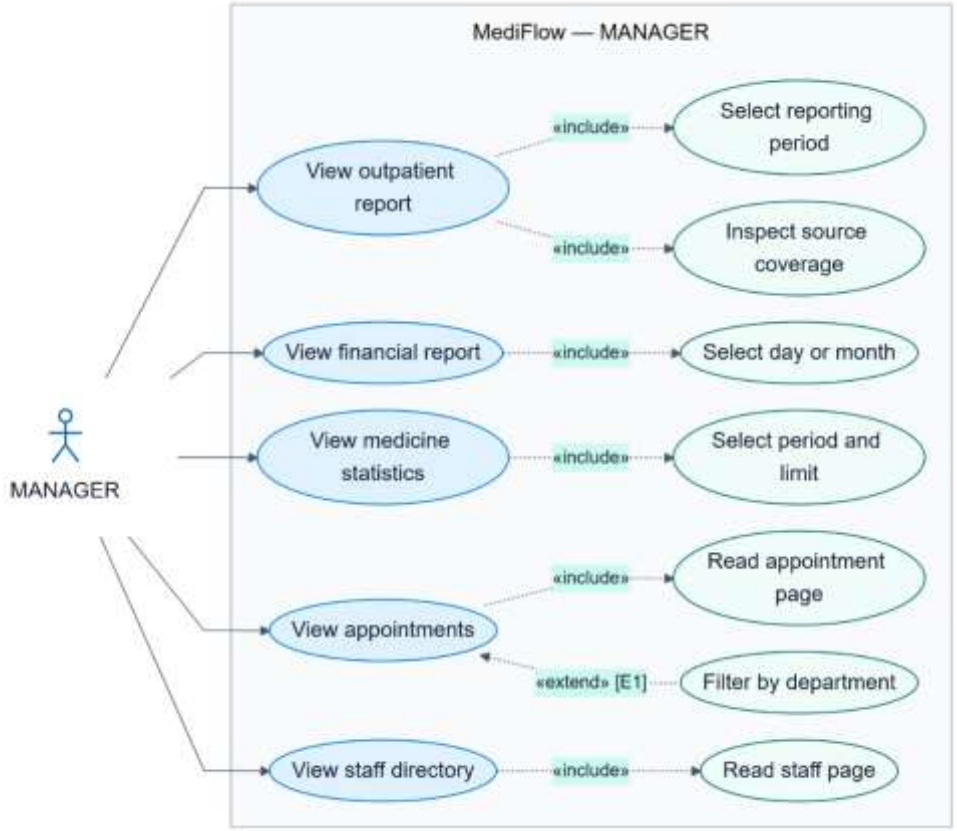
1.5. PHARMACIST



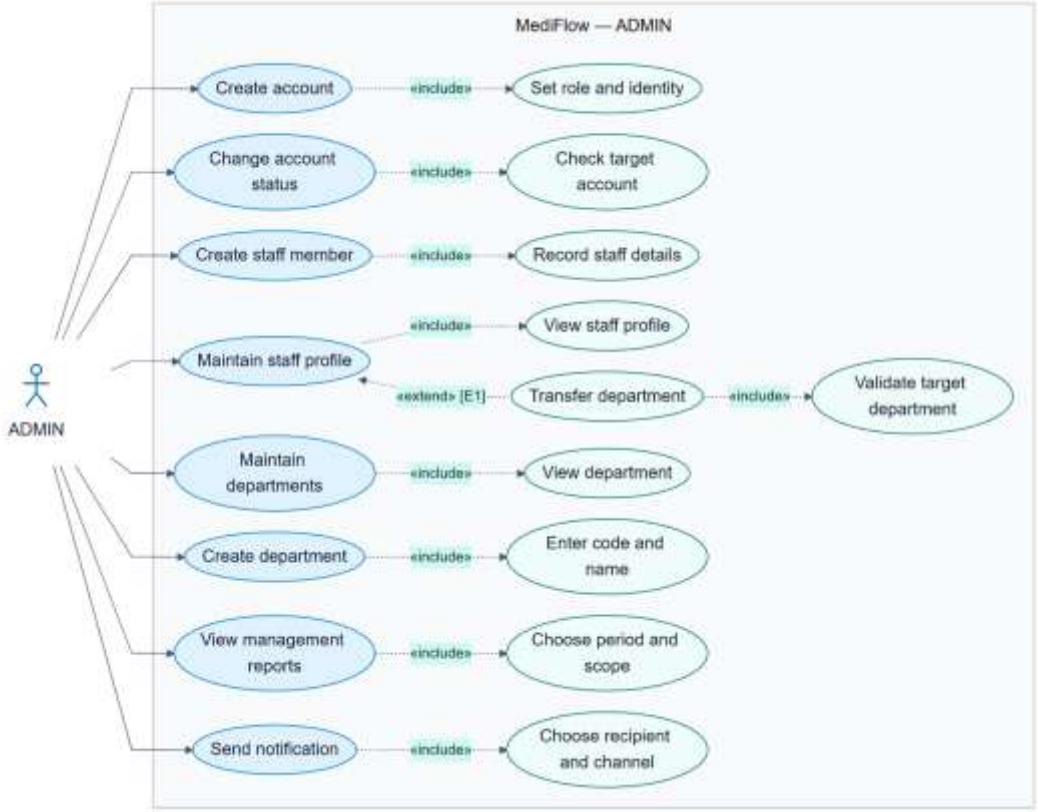
1.6. CASHIER


MediFlow — CASHIER
CASHIER
Find invoice
«include»
View patient invoices
Create invoice
«include»
Validate invoice items
Record invoice payment
«include»
Validate payment command
Find payment request [V1]
«include»
Identify source and target
«include»
Check refundable balance
«include»
Check amount and replay key
Record ledger payment [V1]
«include»
Return payment receipt
«extend» [E1]
Refund payment [V1]
«include»
Record amount and reason

1.7. MANAGER – Manager tasks

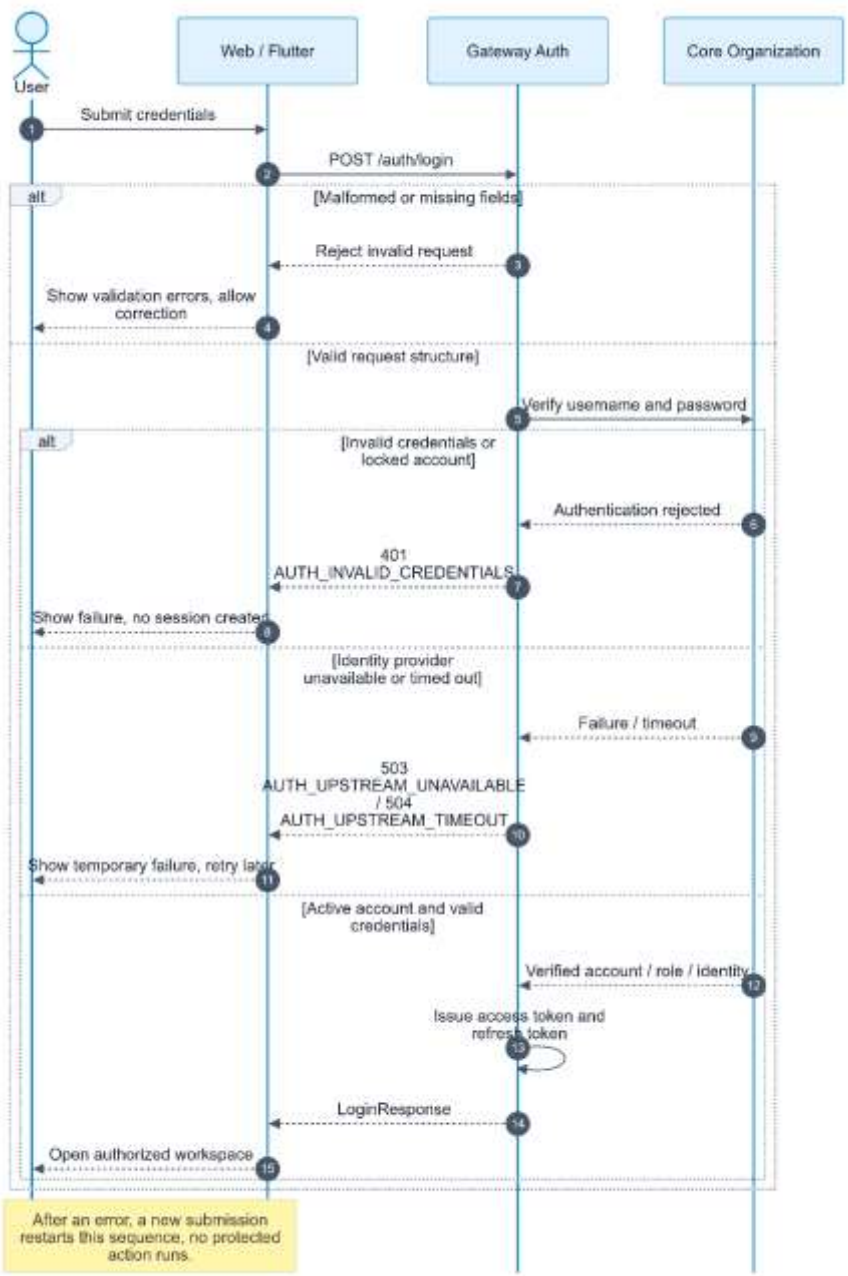


1.8. ADMIN – Administrator tasks

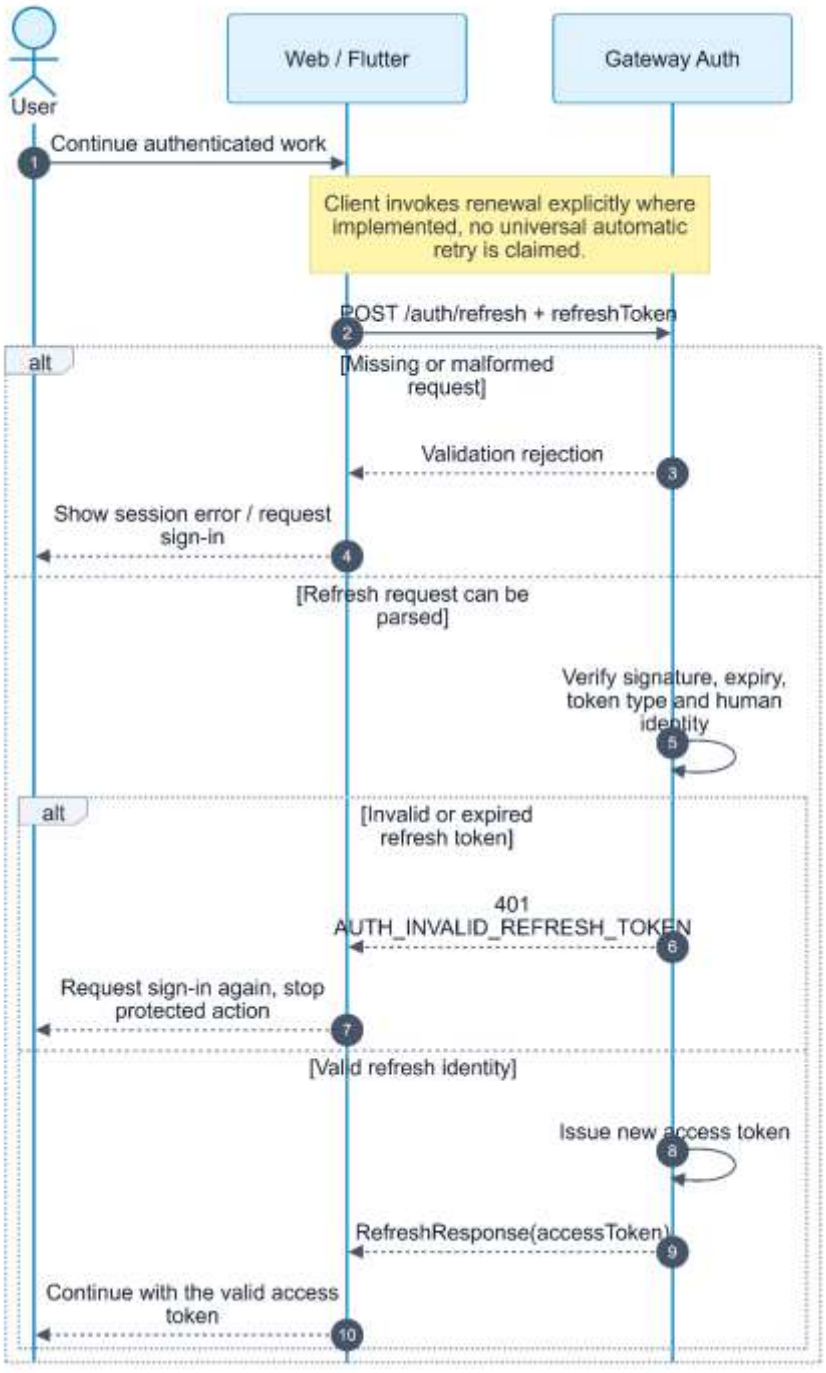


2. Sequences Diagrams

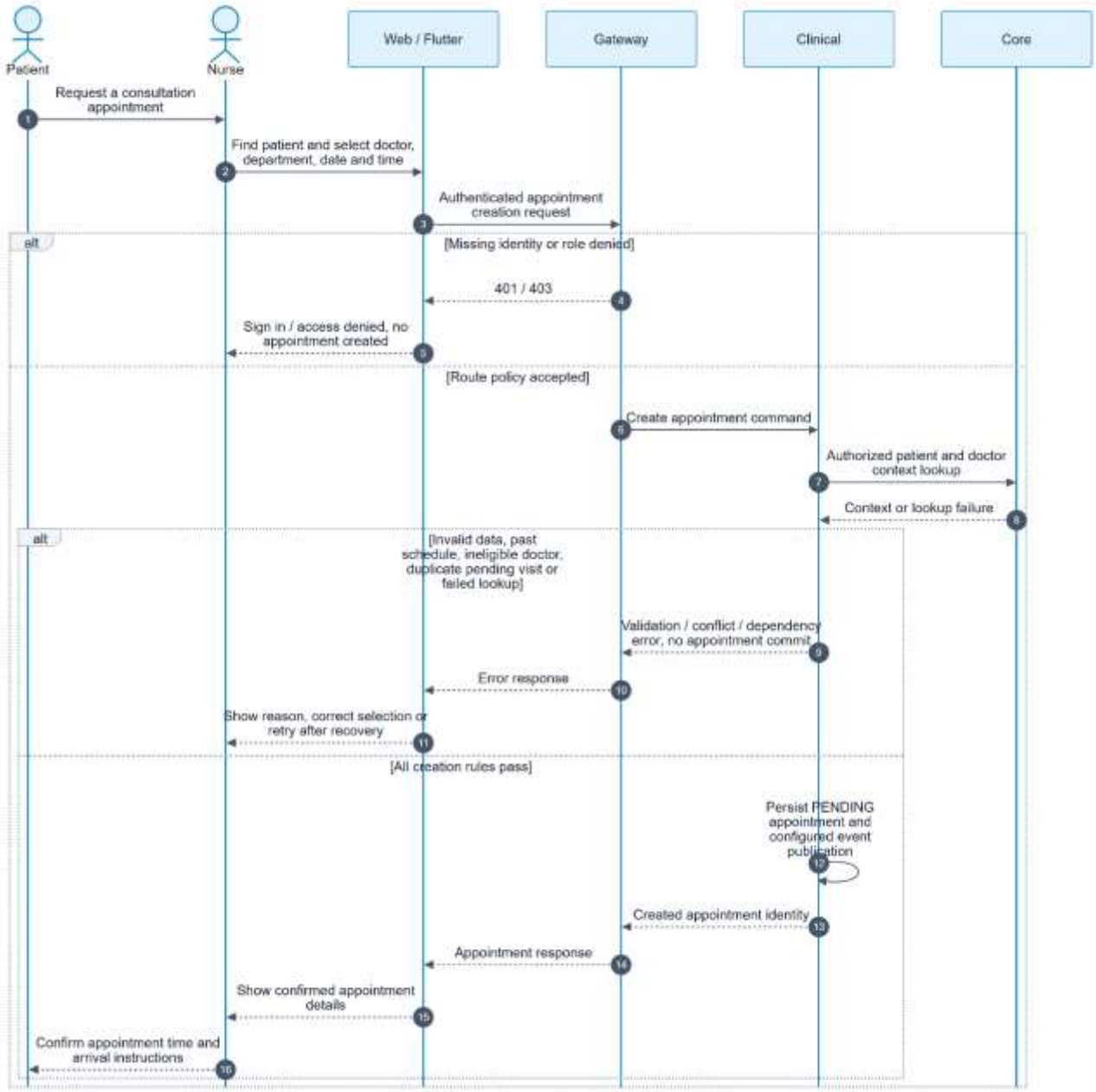
2.1. Sign in — validation, rejected credentials and recovery



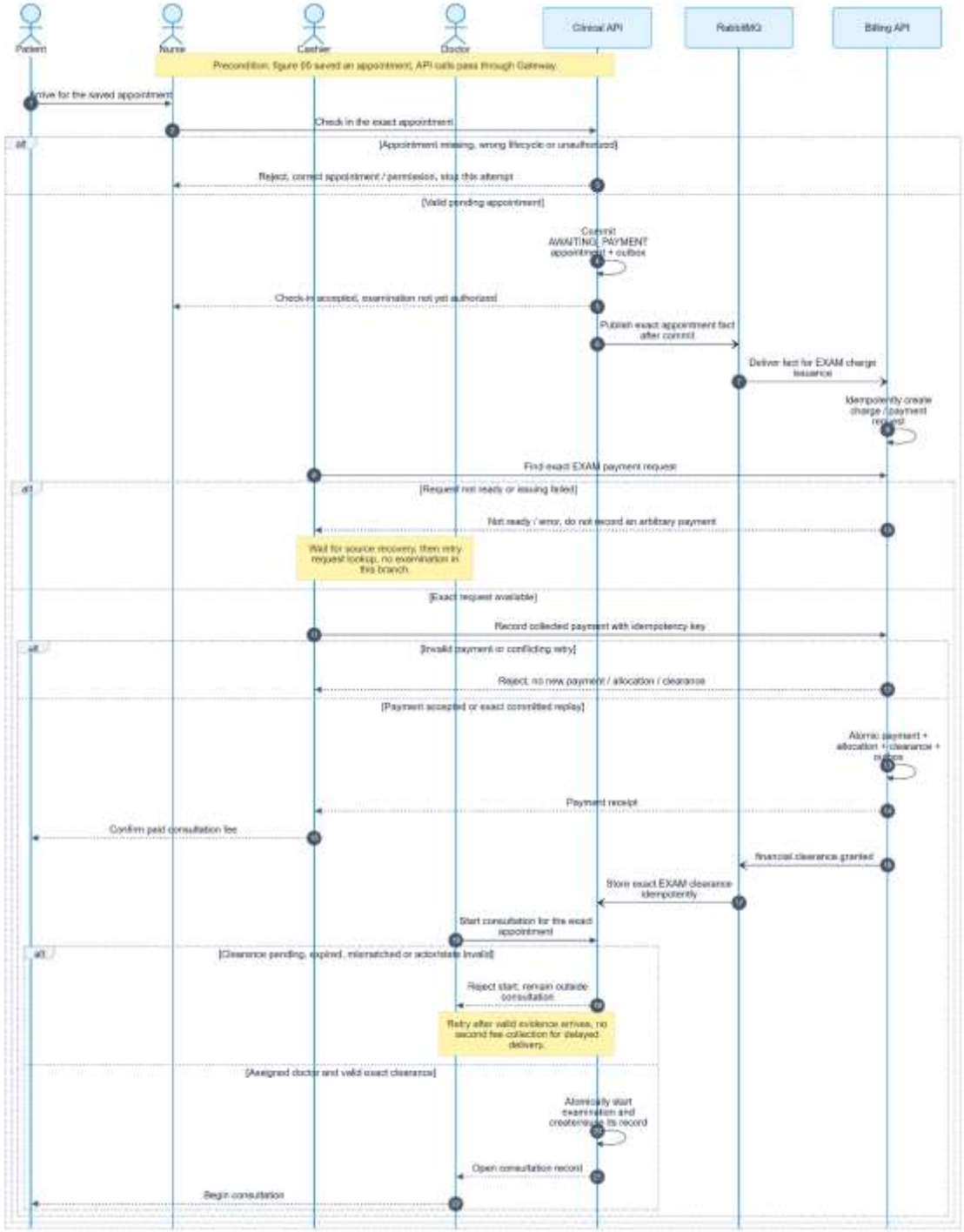
2.2. Renew session — a separate authentication use case



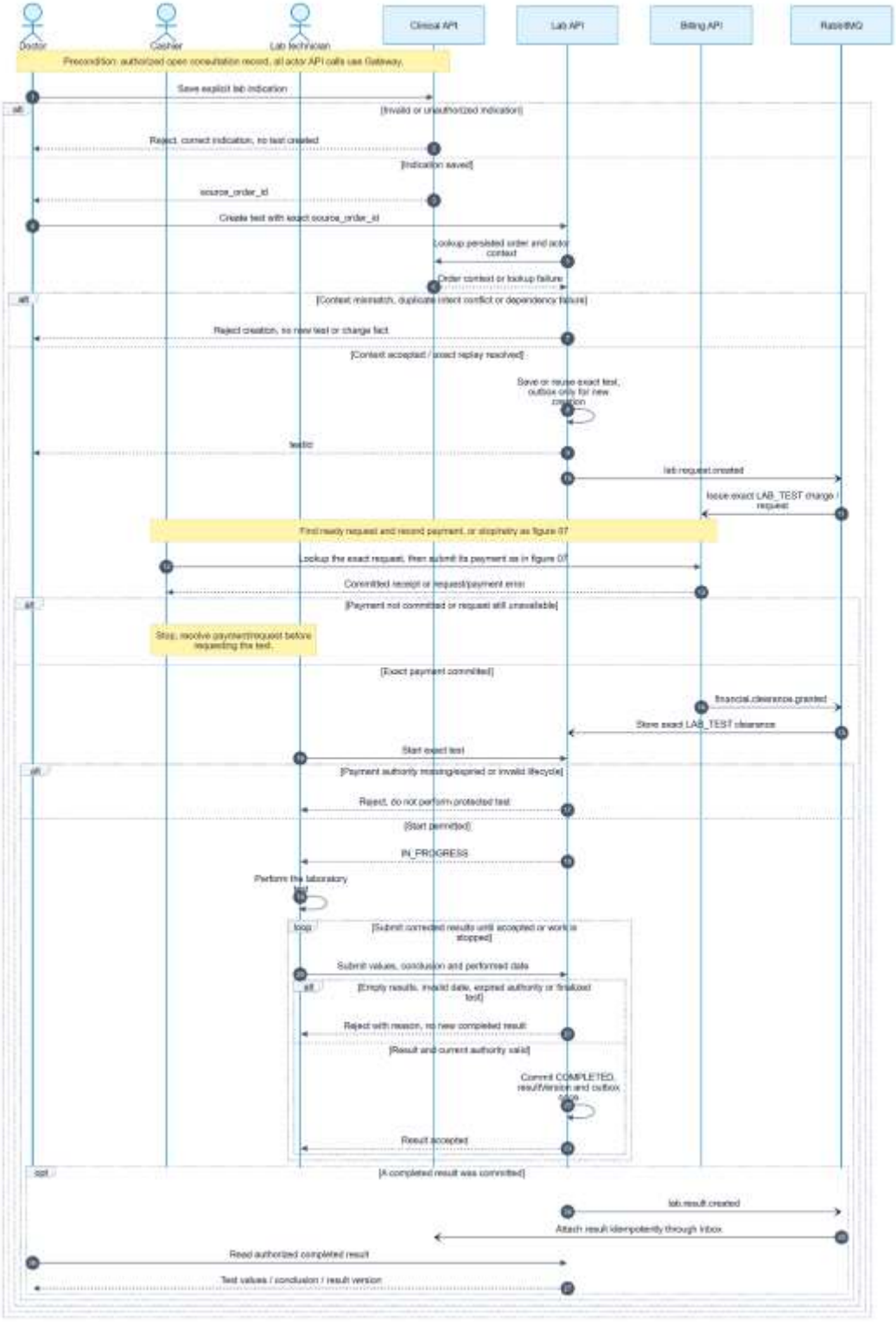
2.3. Schedule appointment before arrival



2.4. Arrival → exam payment → start consultation

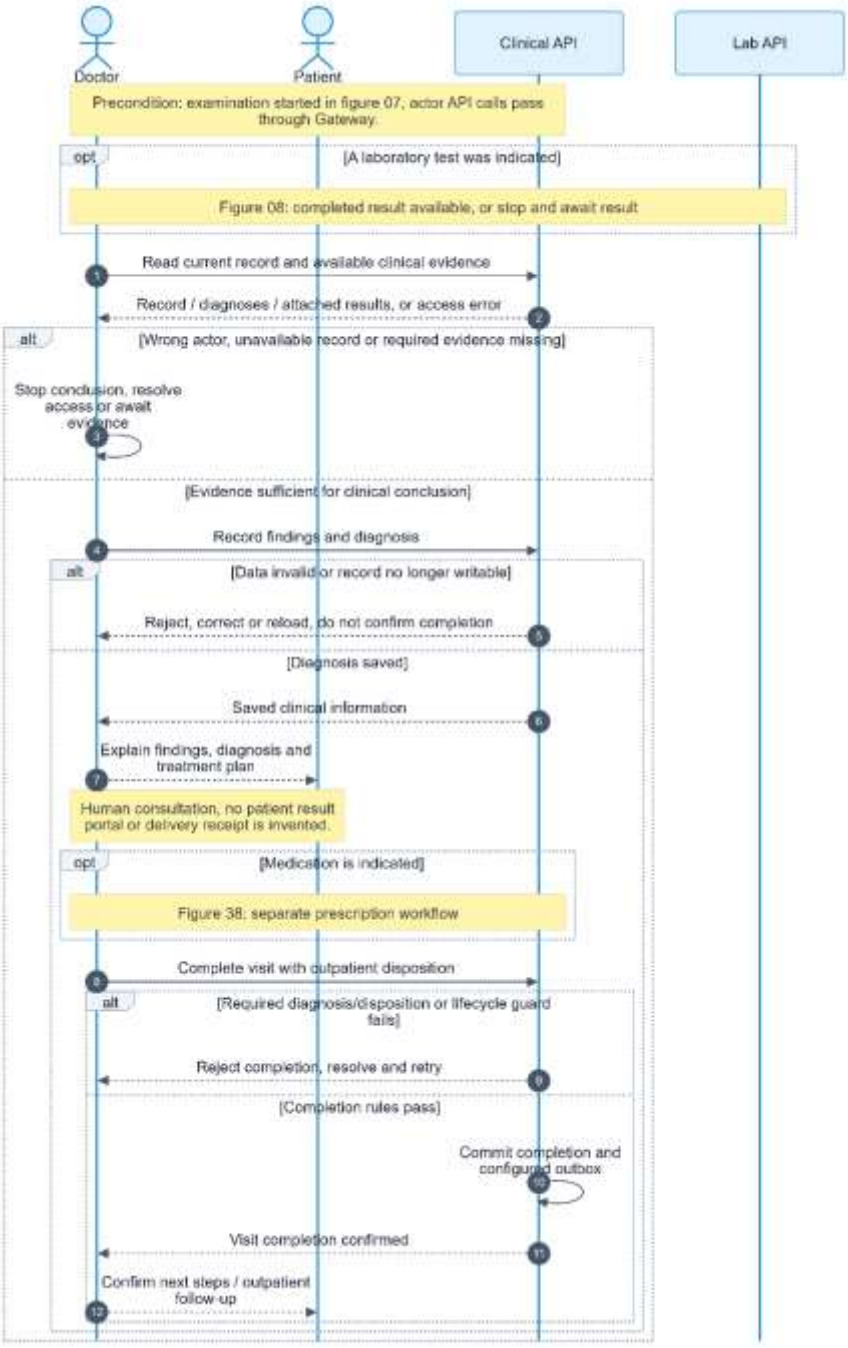


2.5. Lab order → payment → test → doctor reads results

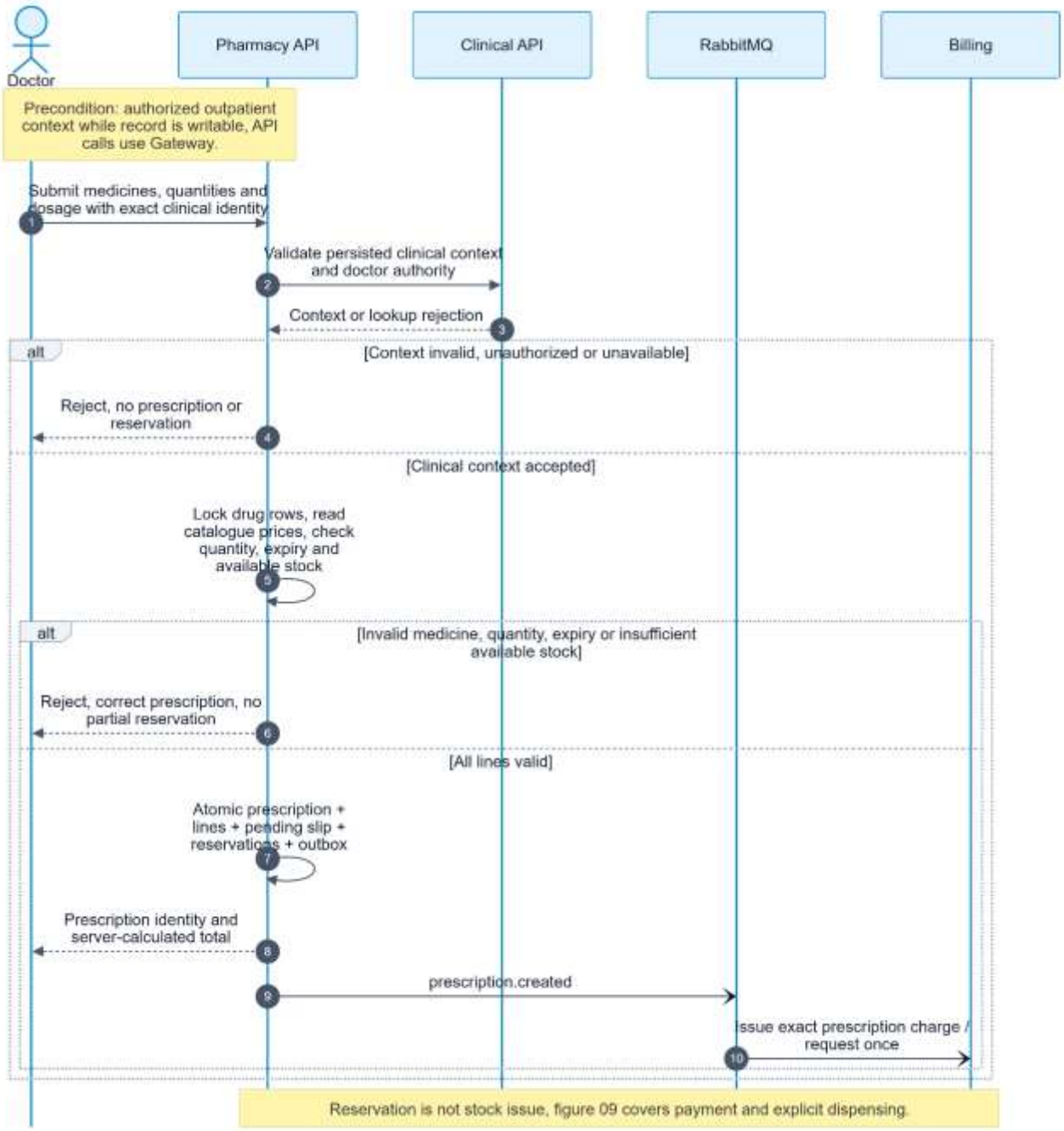


	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

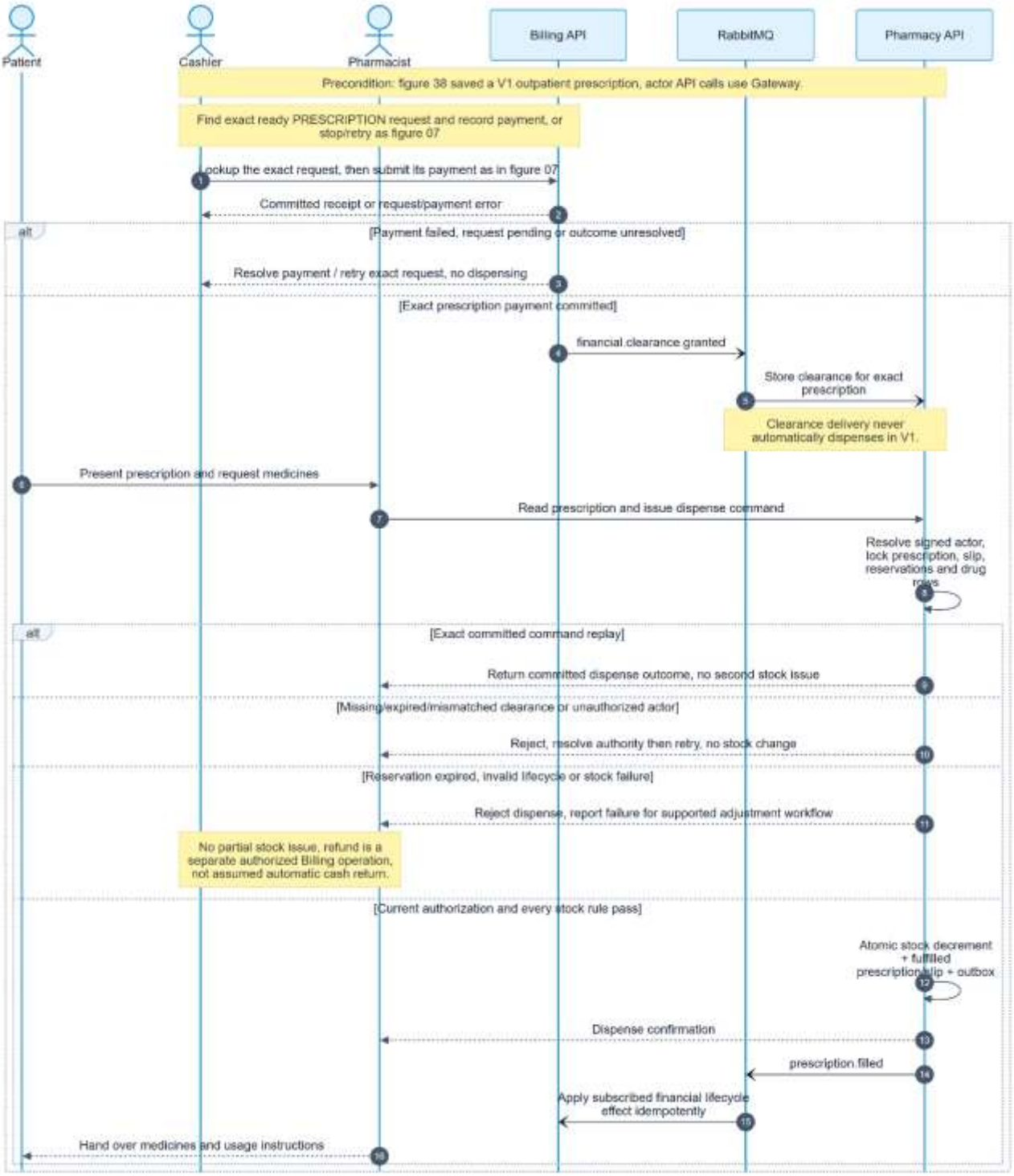
2.6. Doctor explains consultation outcome to patient



2.7. Create prescription — an optional separate workflow

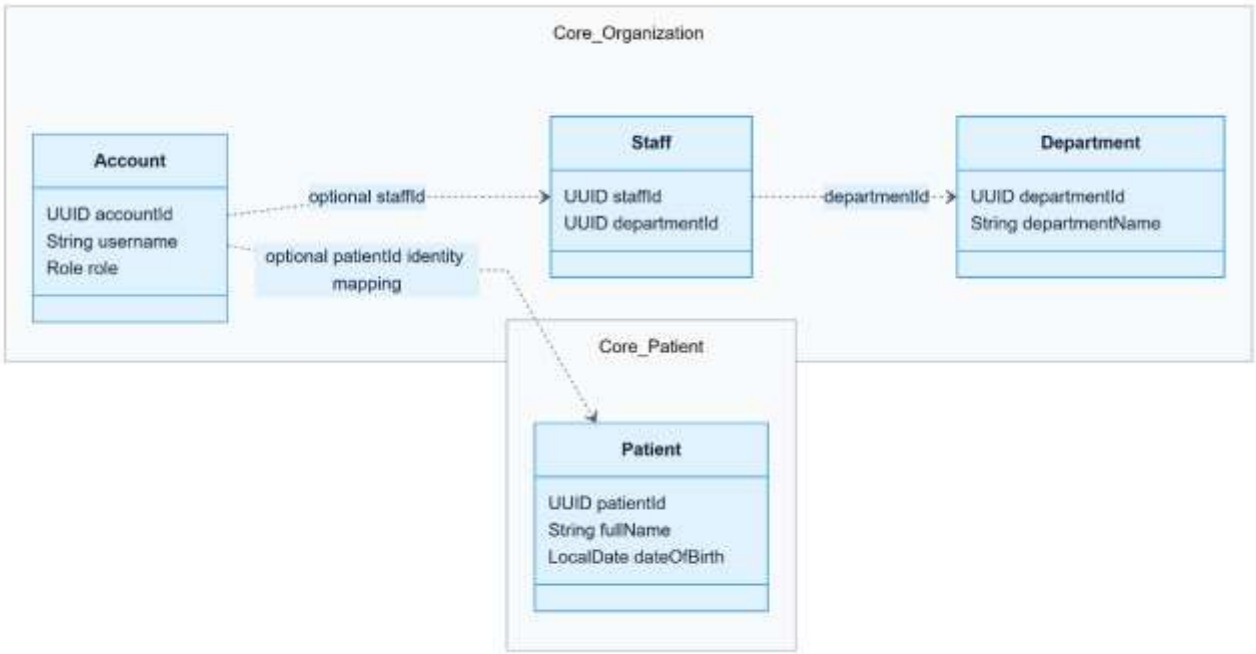


2.8. Prescription payment and explicit pharmacist dispensing

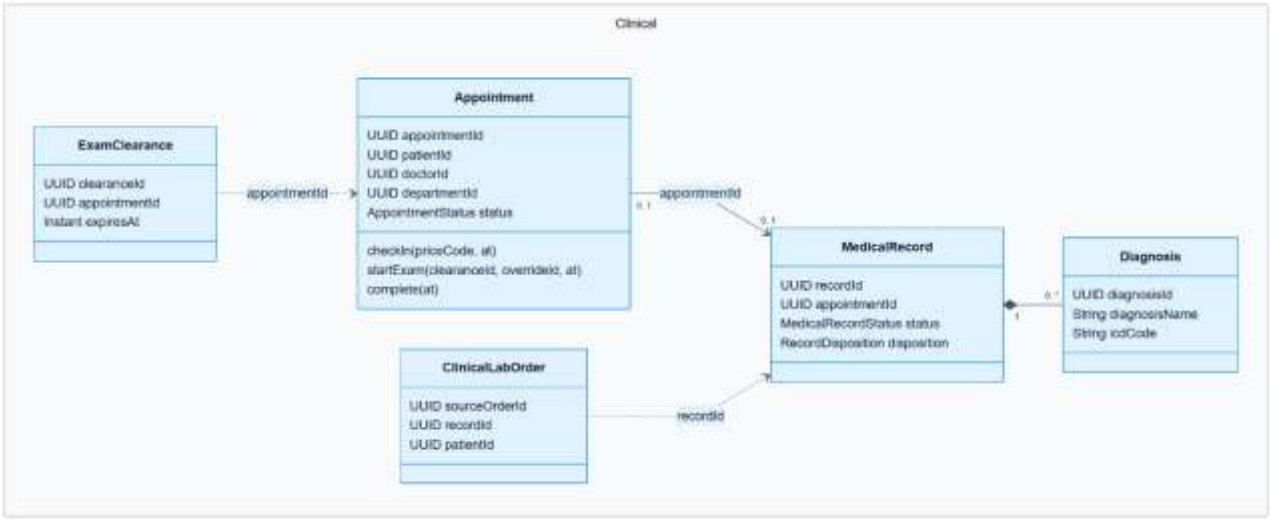


3. Class Diagrams

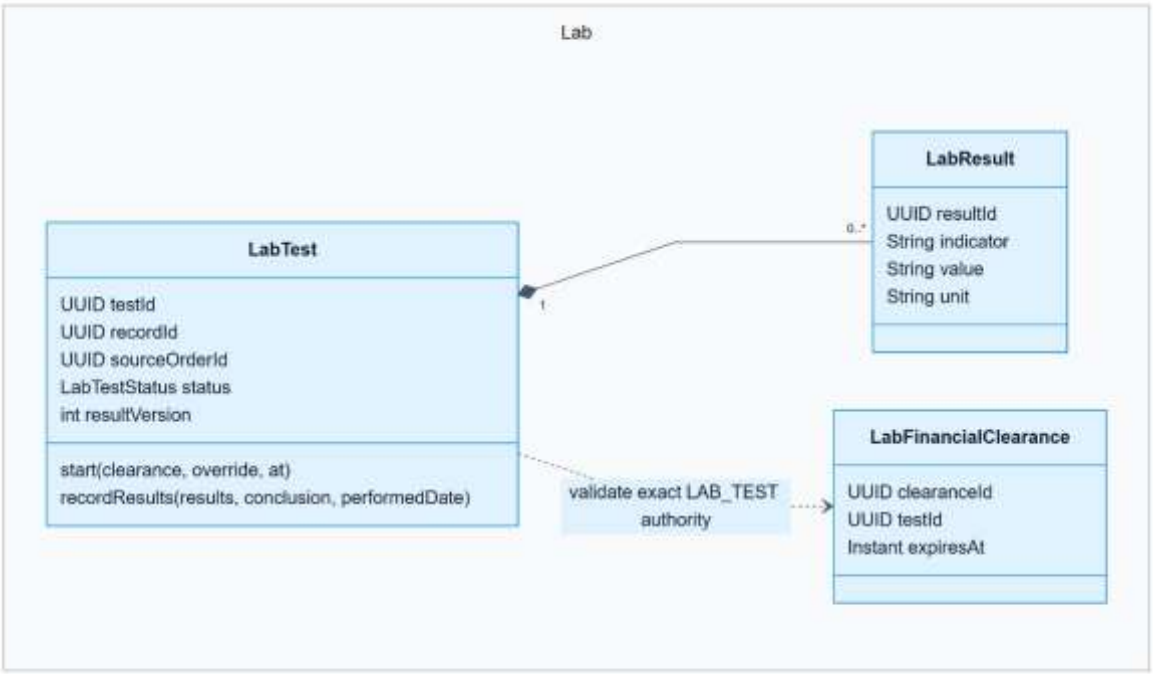
3.1. Core service — Organization and Patient modules



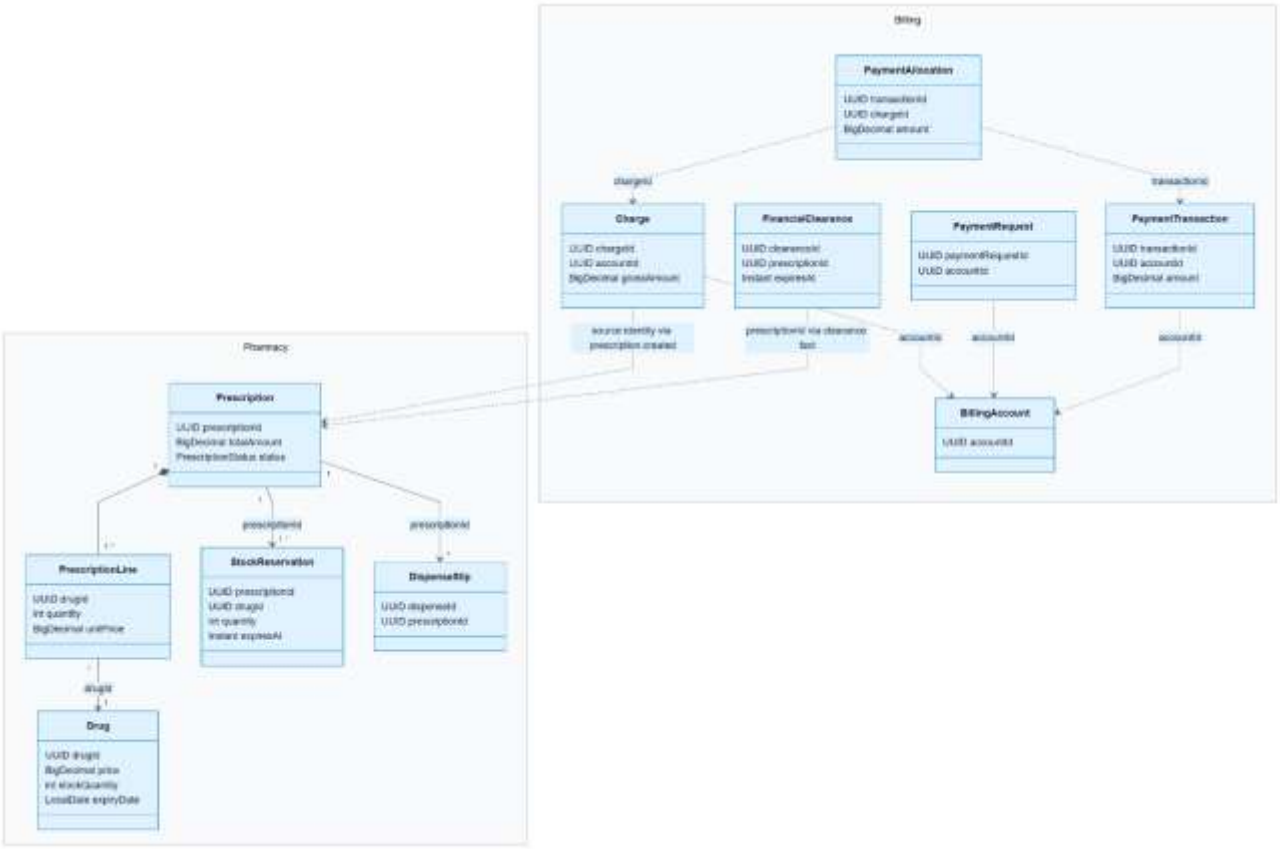
3.2. Clinical service — domain classes



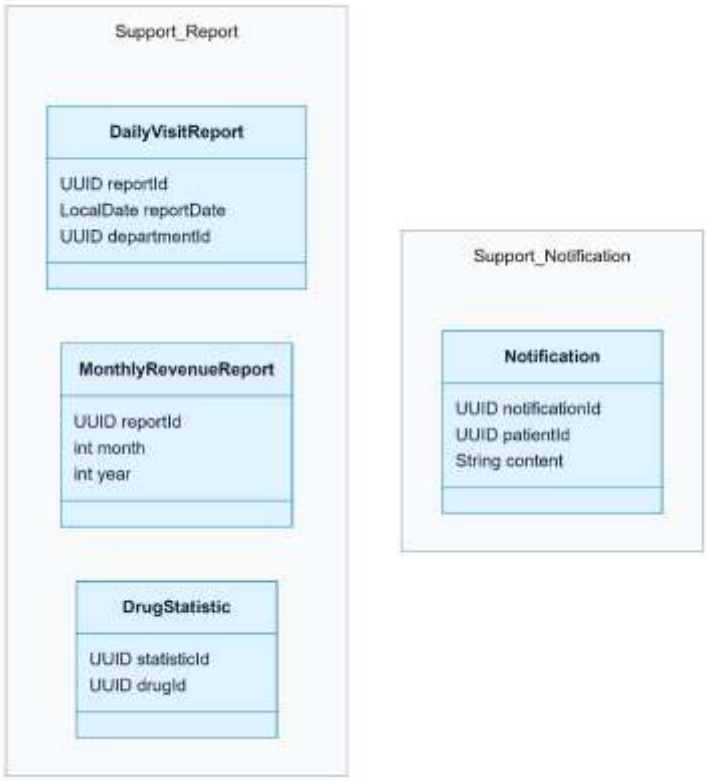
3.3. Lab service — tests, results and financial authority



3.4. Pharmacy and Billing — separate services, explicit contracts

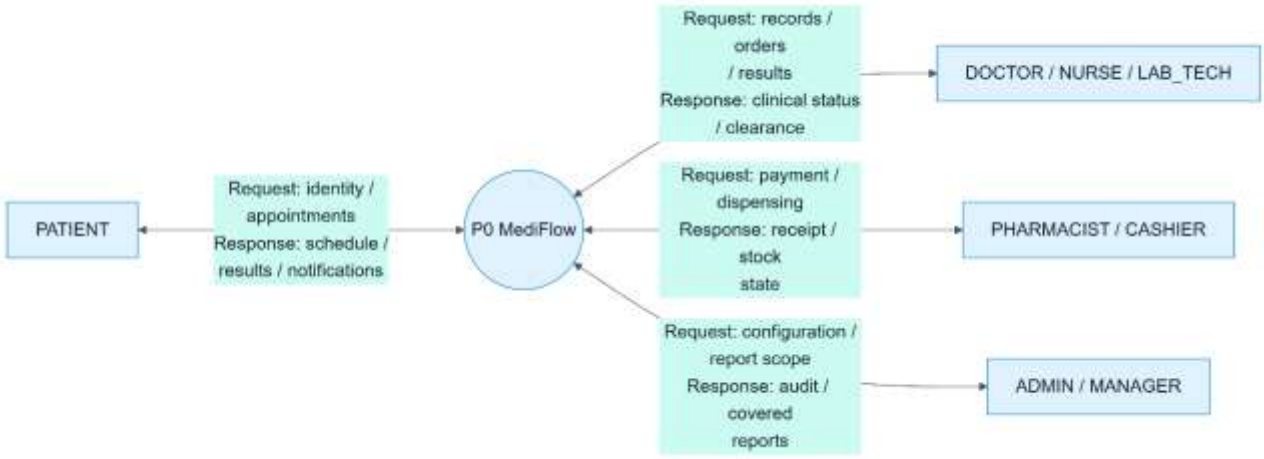


3.5. Support service — separate reporting and notification models

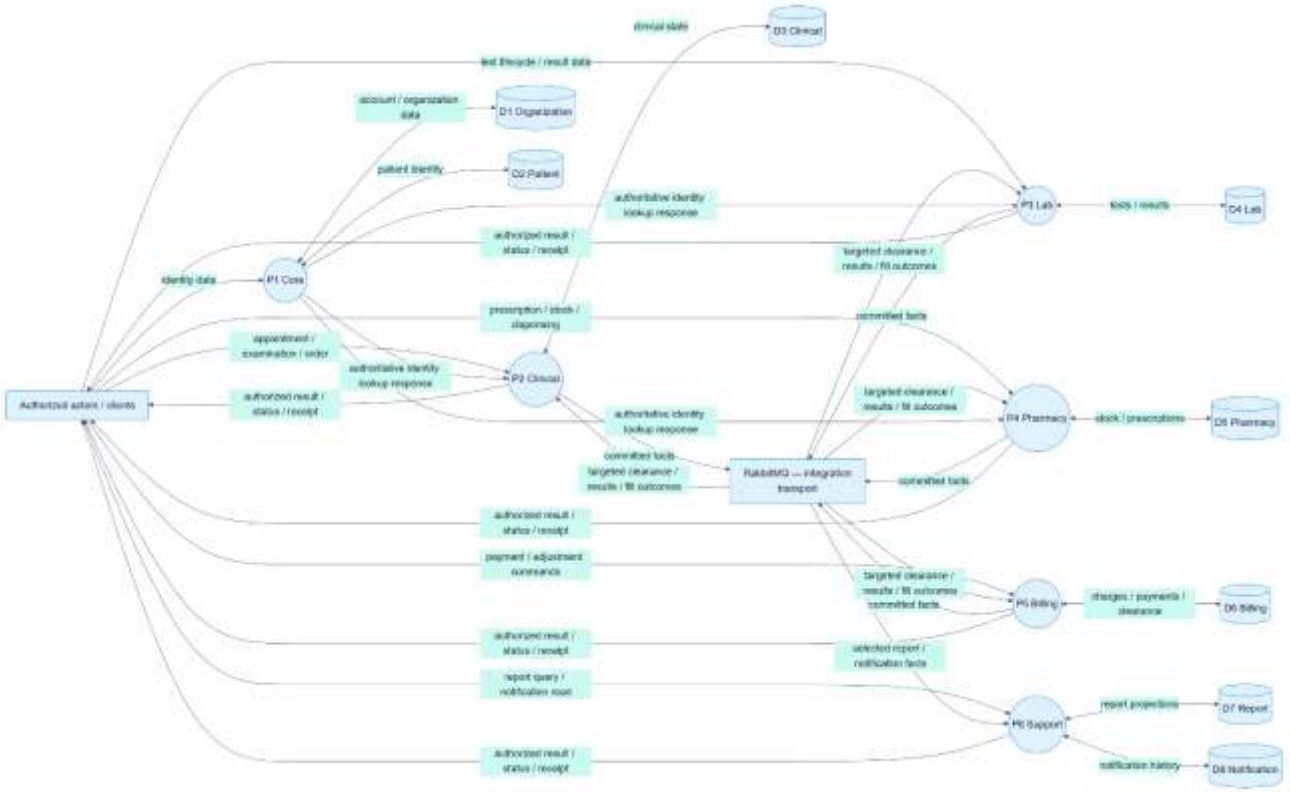


Data Flow Diagram

1. DFD Level 0 — system boundary



2. DFD Level 1 — six processes and eight stores



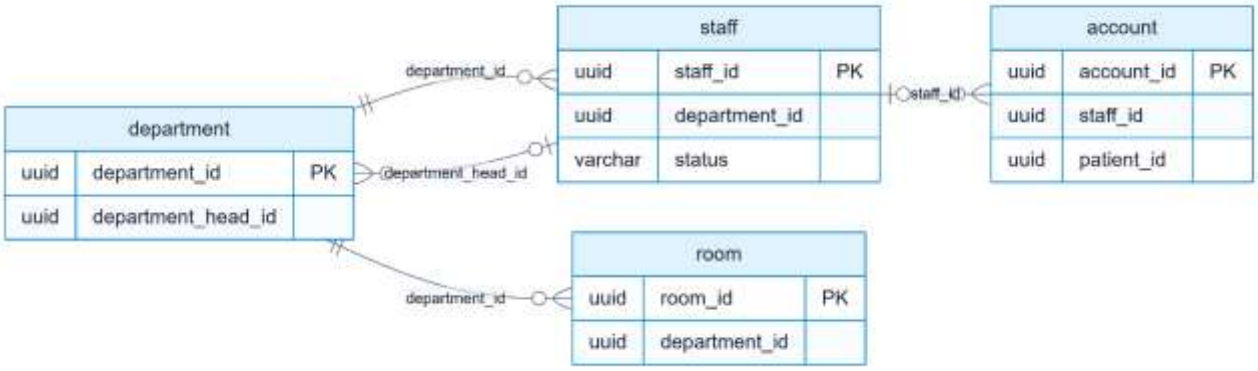
	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

3. DFD Level 2 — V1 payment and service authorization

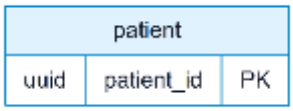


Entity Relationship Diagram

1. Organization

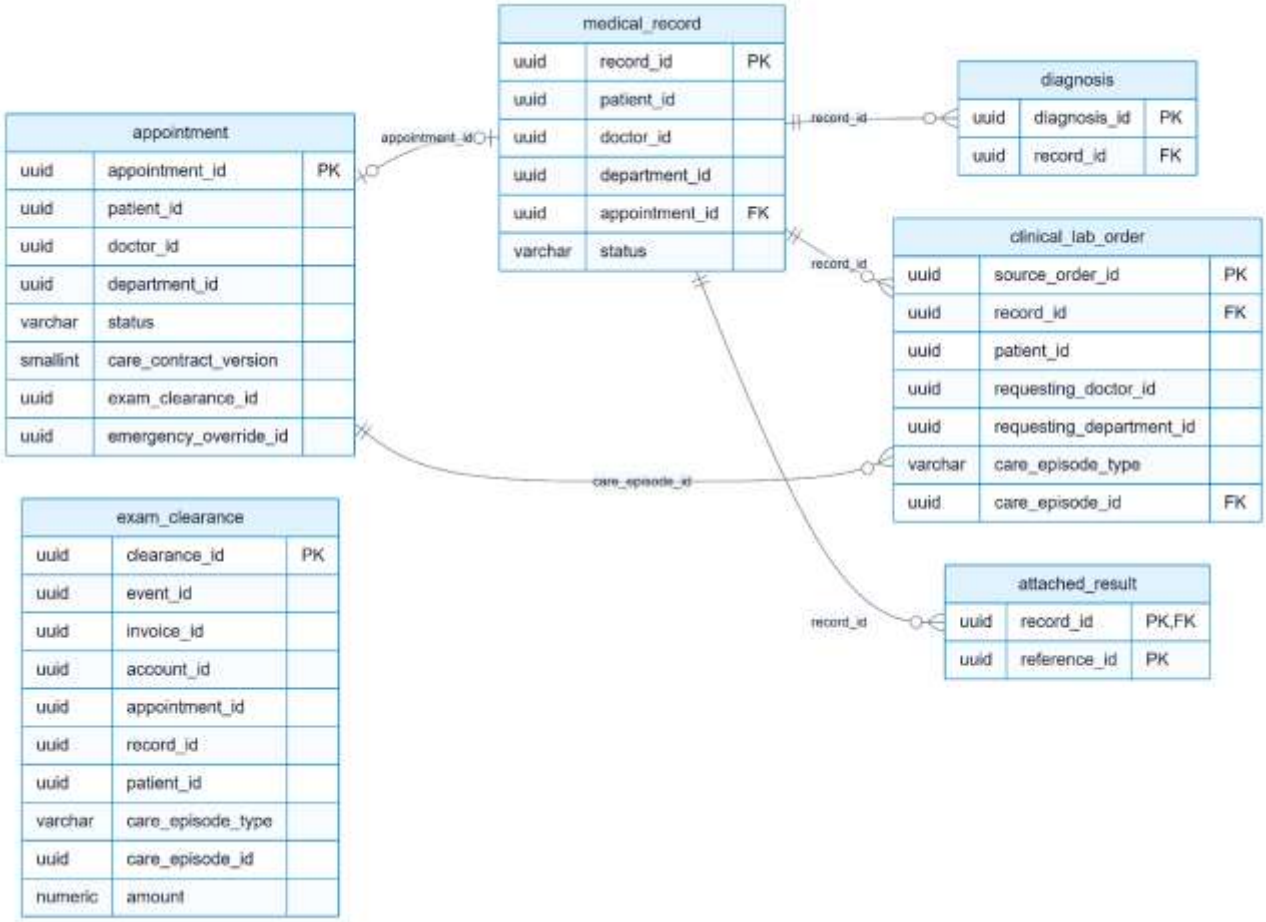


2. Patient

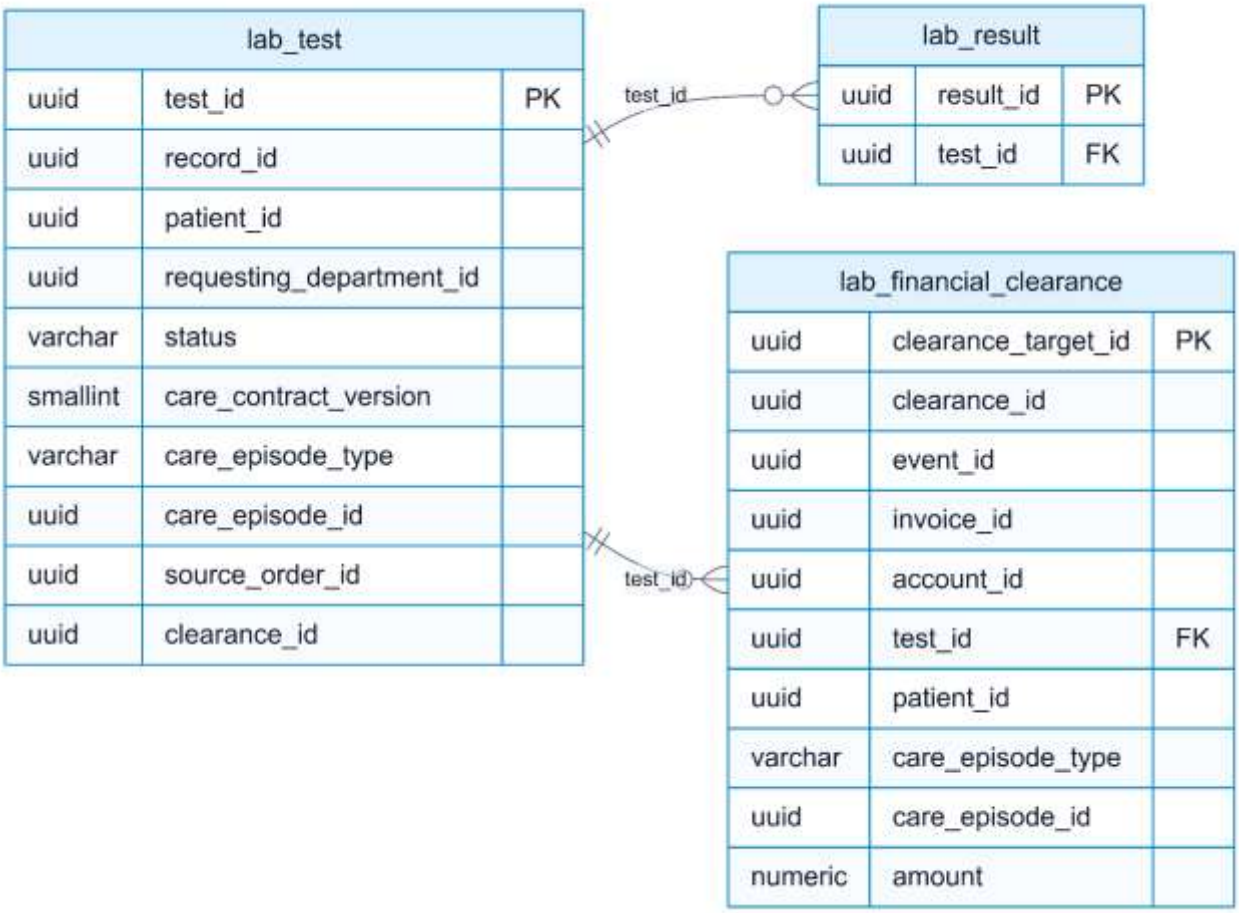


	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

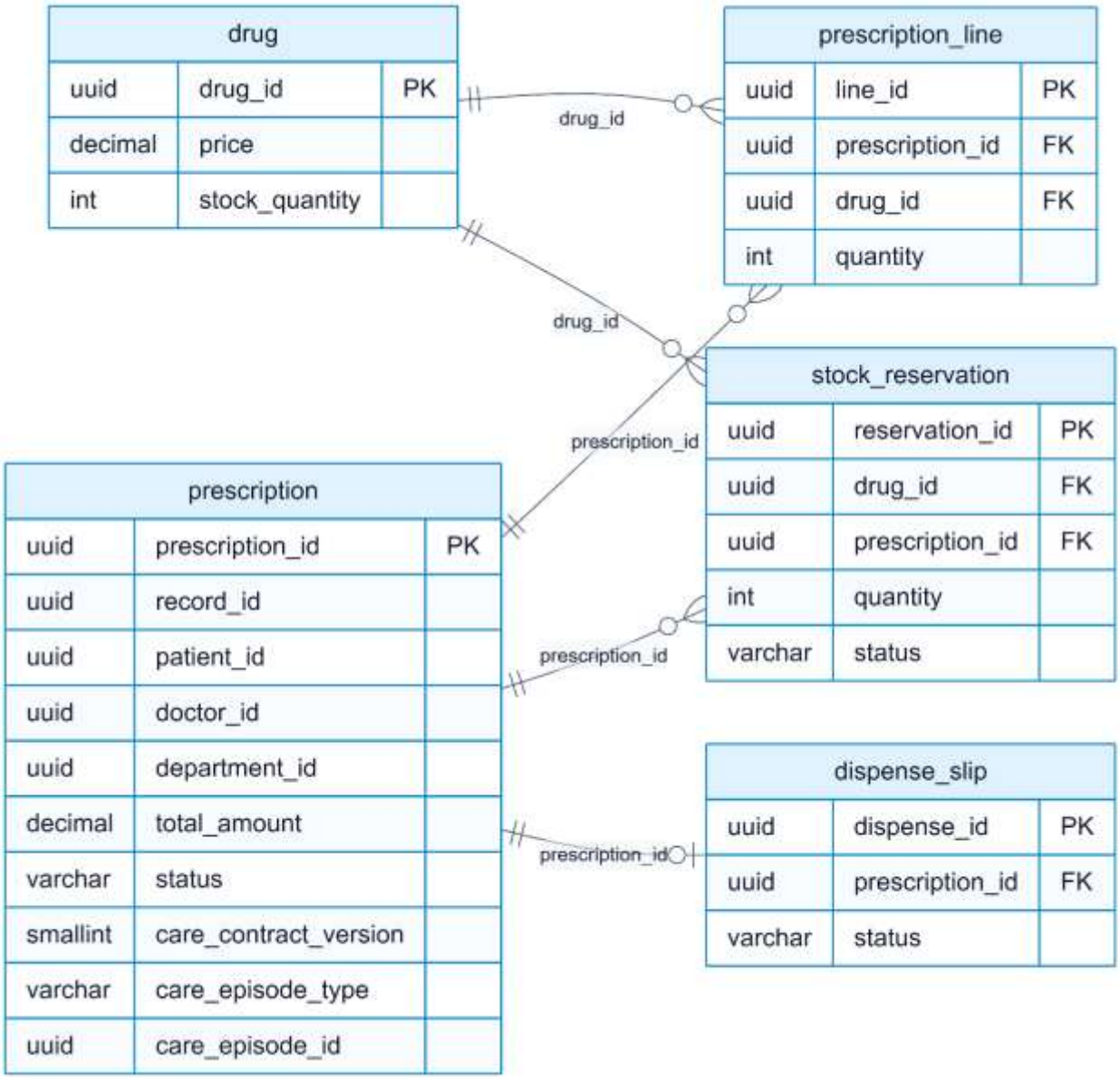
3. Clinical


medical_record
uuid record_id PK
uuid patient_id
uuid doctor_id
uuid department_id
uuid appointment_id FK
varchar status
appointment
uuid appointment_id PK
uuid patient_id
uuid doctor_id
uuid department_id
varchar status
smallint care_contract_version
uuid exam_clearance_id
uuid emergency_override_id
diagnosis
uuid diagnosis_id PK
uuid record_id FK
clinical_lab_order
uuid source_order_id PK
uuid record_id FK
uuid patient_id
uuid requesting_doctor_id
uuid requesting_department_id
varchar care_episode_type
uuid care_episode_id FK
attached_result
uuid record_id PK,FK
uuid reference_id PK
exam_clearance
uuid clearance_id PK
uuid event_id
uuid invoice_id
uuid account_id
uuid appointment_id
uuid record_id
uuid patient_id
varchar care_episode_type
uuid care_episode_id
numeric amount
appointment_id
record_id
record_id
record_id
care_episode_id

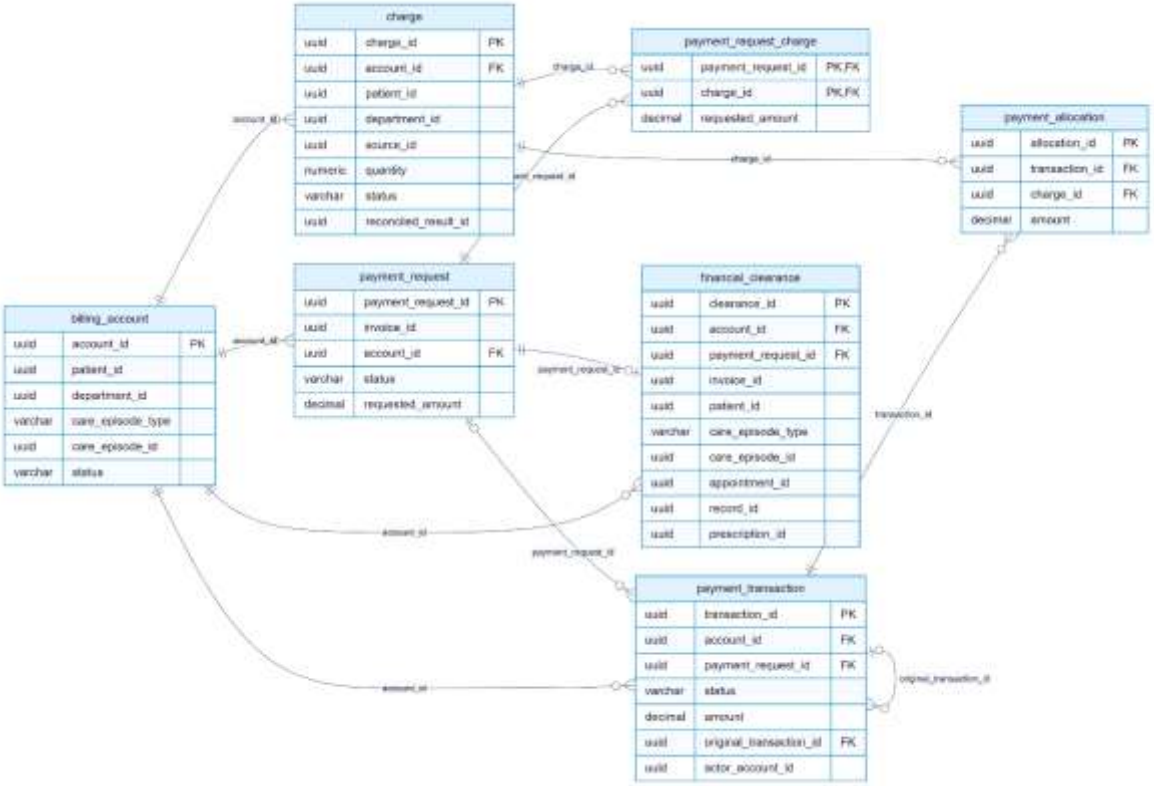
4. Lab


lab_test
uuid test_id PK
uuid record_id
uuid patient_id
uuid requesting_department_id
varchar status
smallint care_contract_version
varchar care_episode_type
uuid care_episode_id
uuid source_order_id
uuid clearance_id
test_id
lab_result
uuid result_id PK
uuid test_id FK
test_id
lab_financial_clearance
uuid clearance_target_id PK
uuid clearance_id
uuid event_id
uuid invoice_id
uuid account_id
uuid test_id FK
uuid patient_id
varchar care_episode_type
uuid care_episode_id
numeric amount

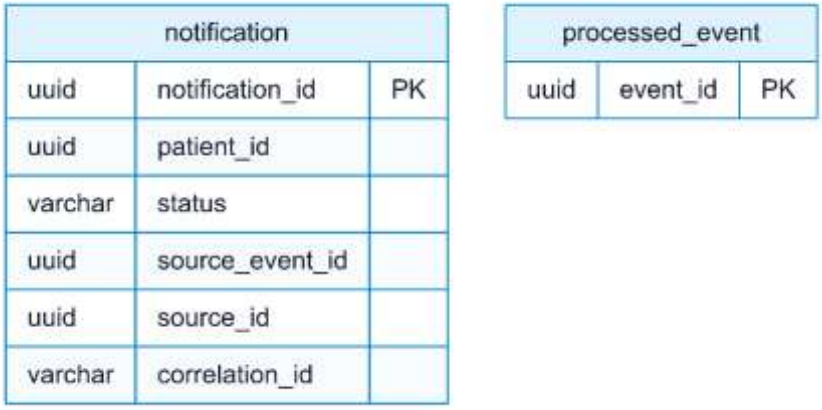
5. Pharmacy


drug
uuid drug_id PK
decimal price
int stock_quantity
prescription_line
uuid line_id PK
uuid prescription_id FK
uuid drug_id FK
int quantity
drug_id
drug_id
stock_reservation
uuid reservation_id PK
uuid drug_id FK
uuid prescription_id FK
int quantity
varchar status
prescription_id
prescription
uuid prescription_id PK
uuid record_id
uuid patient_id
uuid doctor_id
uuid department_id
decimal total_amount
varchar status
smallint care_contract_version
varchar care_episode_type
uuid care_episode_id
prescription_id
dispense_slip
uuid dispense_id PK
uuid prescription_id FK
varchar status
prescription_id

6. Billing



7. Notification



8. Report

daily_visit_report		
uuid	report_id	PK
date	report_date	
uuid	department_id	

monthly_revenue_report		
uuid	report_id	PK
uuid	department_id	

drug_statistic		
uuid	statistic_id	PK
uuid	drug_id	
date	report_date	
uuid	department_id	

payment_contribution		
uuid	invoice_id	PK
uuid	completed_event_id	
uuid	failed_event_id	
uuid	department_id	
decimal	amount	
varchar	status	

processed_event		
uuid	event_id	PK

Table Design

1. Database Tables

Unquoted PostgreSQL identifiers fold to lowercase even when migrations use uppercase. This inventory reads migrations by V number and records each table's creation and ALTER TABLE migrations. It inventories source, not a live DB: unapplied tables and inactive features may appear. Flyway's automatically created flyway_schema_history is excluded from business table counts.

Service Context	Database	Declared tables	Migrations
Organization	mediflow_organization	7	4
Patient	mediflow_patient	1	1
Clinical	mediflow_clinical	12	7
Lab	mediflow_lab	8	3
Pharmacy	mediflow_pharmacy	17	20
Billing	mediflow_billing	42	23
Notification	mediflow_notification	7	4
Report	mediflow_report	58	24

1.1. Organization

Table	PK / identity
department	department_id
staff	staff_id
account	account_id
room	room_id
operating_room	room_id
surgical_capability	staff_id, team_role
surgery_authority_outbox	event_id

1.2. Patient

Table	PK / identity
patient	patient_id

1.3. Clinical

Table	PK / identity
appointment	appointment_id
medical_record	record_id
diagnosis	diagnosis_id
attached_result	record_id, type, reference_id
processed_event	event_id

Table	PK / identity
exam_clearance	clearance_id
clinical_emergency_override	override_id
admission_referral	admission_request_id
clinical_outbox_event	event_id
clinical_lab_order	source_order_id
consumer_inbox_delivery	consumer_name, event_id
consumer_inbox_operation	consumer_name, operation_key

1.4. Lab

Table	PK / identity
lab_test	test_id
lab_result	result_id
processed_event	event_id
lab_financial_clearance	clearance_target_id
lab_emergency_override	override_id
lab_outbox_event	event_id
consumer_inbox_delivery	consumer_name, event_id
consumer_inbox_operation	consumer_name, operation_key

1.5. Pharmacy

Table	PK / identity
drug	drug_id
prescription	prescription_id
prescription_line	line_id
dispense_slip	dispense_id
processed_event	event_id
stock_reservation	reservation_id
pharmacy_event_outbox	event_id
payment_receipt	receipt_id
stock_adjustment	adjustment_id
pharmacy_scheduler_lease	job_name
admission_medication_context	admission_id
admission_medication_event	event_id
prescription_clearance_target	prescription_id
prescription_clearance	clearance_id
prescription_clearance_event	event_id
care_prescription_creation	command_id
pharmacy_outpatient_delivery_cohort	prescription_id

1.6. Billing

Table	PK / identity
fee	fee_id
invoice	invoice_id
processed_event	event_id
lab_test_type	lab_id
billing_event_outbox	event_id
billing_account	account_id
charge	charge_id
payment_request	payment_request_id
payment_request_charge	payment_request_id, charge_id
payment_transaction	transaction_id
payment_allocation	allocation_id
financial_clearance	clearance_id
insurance_adjustment	adjustment_id
settlement	settlement_id
payment_request_target	payment_request_id
ledger_refund_evidence	refund_transaction_id
surgery_charge_delivery	event_id

Table	PK / identity
surgery_charge_source	surgery_case_id
surgery_charge_item	surgery_case_id,item_code
surgery_cancellation_source	surgery_case_id
surgery_cancellation_delivery	event_id
surgery_cancellation_refund_due	cancellation_id,original_transaction_id,charge_id
lab_test_charge_delivery	event_id
lab_test_charge_source	lab_id
admission_deposit_delivery	event_id
admission_deposit_source	admission_id
admission_reconciliation	settlement_id
admission_charge_close	admission_id
admission_charge_close_delivery	event_id
deposit_allocation_operation	allocation_operation_id
surgery_completion_source	surgery_case_id
surgery_completion_delivery	event_id
final_admission_settlement_operation	recognition_operation_id
prescription_charge_source	prescription_id
prescription_charge_item	prescription_id,drug_id

Table	PK / identity
prescription_charge_delivery	event_id
prescription_terminal_source	prescription_id
prescription_terminal_delivery	event_id
prescription_terminal_refund_due	prescription_id, original_transaction_id, charge_id
exam_charge_delivery	event_id
exam_charge_source	appointment_id
billing_outpatient_publication_cohort	payment_request_id

1.7. Notification

Table	PK / identity
notification	notification_id
processed_event	event_id
notification_event_outbox	event_id
surgery_notice_case	surgery_case_id
surgery_notice_source	event_type,source_id
surgery_notice_snapshot	snapshot_id
care_notification_source	event_type,source_id

1.8. Report

Table	PK / identity
daily_visit_report	report_id
monthly_revenue_report	report_id
drug_statistic	statistic_id
processed_event	event_id
payment_contribution	invoice_id
financial_contribution	contribution_id
daily_financial_report	report_id
operational_contribution	contribution_id
daily_operational_report	report_id
operational_event_journal	event_id
operational_delivery	event_id, metric_type
operational_replay_generation	generation_id
operational_replay_input	generation_id, event_id
operational_replay_contribution	generation_id, source_type, source_id, source_revision, metric_type
operational_replay_scope	see DDL
report_admission_target	admission_id
report_admission_delivery	event_id

Table	PK / identity
report_admission_fact	admission_id, fact_type
operational_report_publication	report_kind
operational_source_snapshot	source_type,source_id,source_revision,event_type
report_cash_receipt	transaction_id
report_cash_delivery	event_id
report_gross_cash_daily	scope_id
cash_replay_generation	generation_id
cash_replay_input	generation_id,transaction_id
cash_replay_receipt	generation_id,transaction_id
cash_replay_scope	see DDL
report_cash_refund	refund_transaction_id
report_cash_refund_delivery	event_id
report_refund_cash_daily	scope_id
prescription_fill_receipt	prescription_id
refund_replay_generation	generation_id
refund_replay_input	generation_id,refund_transaction_id
refund_replay_fact	generation_id,refund_transaction_id
refund_replay_scope	see DDL

Table	PK / identity
report_settlement_snapshot	settlement_id
report_settlement_delivery	event_id
report_receivable_current	account_id
report_receivable_stock	see DDL
financial_report_generation	generation_id
financial_report_day	see DDL
financial_report_publication	currency,report_zone
report_medical_discharge	admission_id
report_medical_discharge_delivery	event_id
report_final_recognition	recognition_operation_id
report_final_recognition_delivery	event_id
report_final_recognition_contribution	see DDL
report_final_deposit_release	recognition_operation_id,original_transaction_id
report_final_recognition_daily	see DDL
financial_source_rebuild	generation_id
financial_source_rebuild_input	generation_id,source_kind,source_id
financial_source_rebuild_day	see DDL
financial_archive_restore	archive_fingerprint

Table	PK / identity
financial_generation_acceptance	generation_id
financial_publication_audit	command_id
operational_generation_acceptance	generation_id
operational_publication_audit	command_id
report_outpatient_coverage	coverage_id

2. Core table column dictionary

The following tables represent the outpatient journey. Column definitions are extracted from CREATE and ADD COLUMN; dropped columns are removed. This is not a full SQL execution engine: ALTER TYPE/DEFAULT, constraints/indexes and conditional SQL must also be read in linked migrations; do not reconstruct production DDL from this summary. Removed fields such as invoice.dispense_id are excluded.

2.1. Organization

2.1.1. Organization.account

Column	Declared type and column constraints
account_id	UUID PRIMARY KEY
username	VARCHAR(50) NOT NULL
password_hash	VARCHAR(255) NOT NULL
staff_id	UUID
role	VARCHAR(20) NOT NULL

Column	Declared type and column constraints
is_active	BOOLEAN NOT NULL DEFAULT TRUE
last_login_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ
patient_id	UUID

2.1.2. Organization.staff

Column	Declared type and column constraints
staff_id	UUID PRIMARY KEY
full_name	VARCHAR(100) NOT NULL
department_id	UUID NOT NULL
job_title	VARCHAR(20) NOT NULL
specialization	VARCHAR(100)
license_number	VARCHAR(50)
phone_number	VARCHAR(15)
email	VARCHAR(100)
status	VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.1.3. Organization.department

Column	Declared type and column constraints
department_id	UUID PRIMARY KEY
department_name	VARCHAR(100) NOT NULL
abbreviation	VARCHAR(20) NOT NULL
department_type	VARCHAR(20) NOT NULL
department_head_id	UUID
location	VARCHAR(255)
is_active	BOOLEAN NOT NULL DEFAULT TRUE
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.2. Patient

2.2.1. Patient.patient

Column	Declared type and column constraints
patient_id	UUID PRIMARY KEY
full_name	VARCHAR(100) NOT NULL
date_of_birth	DATE NOT NULL
gender	VARCHAR(1) NOT NULL
identity_number	VARCHAR(20) NOT NULL

Column	Declared type and column constraints
address	VARCHAR(255)
phone_number	VARCHAR(15)
email	VARCHAR(100)
health_insurance_number	VARCHAR(20)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.3. Clinical

2.3.1. Clinical.appointment

Column	Declared type and column constraints
appointment_id	UUID PRIMARY KEY
patient_id	UUID NOT NULL
doctor_id	UUID NOT NULL
department_id	UUID NOT NULL
appointment_date	DATE NOT NULL
appointment_time	TIME NOT NULL
status	VARCHAR(20) NOT NULL DEFAULT 'PENDING'
reason	TEXT
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()

Column	Declared type and column constraints
updated_at	TIMESTAMPTZ
care_contract_version	SMALLINT NOT NULL DEFAULT 0
exam_clearance_id	UUID
exam_clearance_at	TIMESTAMPTZ
emergency_override_id	UUID
exam_price_code	VARCHAR(64)
checked_in_at	TIMESTAMPTZ
examination_started_at	TIMESTAMPTZ
completed_at	TIMESTAMPTZ

2.3.2. Clinical.medical_record

Column	Declared type and column constraints
record_id	UUID PRIMARY KEY
patient_id	UUID NOT NULL
doctor_id	UUID NOT NULL
department_id	UUID NOT NULL
examination_date	DATE NOT NULL
symptoms	TEXT
appointment_id	UUID REFERENCES appointment(appointment_id)

Column	Declared type and column constraints
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ
status	VARCHAR(20) NOT NULL DEFAULT 'OPEN'
disposition	VARCHAR(32)
disposition_note	TEXT
completed_at	TIMESTAMPTZ

2.3.3. Clinical.diagnosis

Column	Declared type and column constraints
diagnosis_id	UUID PRIMARY KEY
record_id	UUID NOT NULL REFERENCES medical_record(record_id) ON DELETE CASCADE
diagnosis_name	VARCHAR(255) NOT NULL
description	TEXT
icd_code	VARCHAR(10)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()

2.3.4. Clinical.clinical_lab_order

Column	Declared type and column constraints
source_order_id	UUID PRIMARY KEY

Column	Declared type and column constraints
record_id	UUID NOT NULL REFERENCES medical_record(record_id)
patient_id	UUID NOT NULL
requesting_doctor_id	UUID NOT NULL
requesting_department_id	UUID NOT NULL
care_episode_type	VARCHAR(32) NOT NULL CHECK (care_episode_type = 'OUTPATIENT_VISIT')
care_episode_id	UUID NOT NULL REFERENCES appointment(appointment_id)
lab_type	VARCHAR(50) NOT NULL CHECK (length(btrim(lab_type)) > 0)
requested_date	DATE NOT NULL
price_code	VARCHAR(64) NOT NULL CHECK (length(btrim(price_code)) > 0)
ordered_at	TIMESTAMPTZ NOT NULL

2.4. Lab

2.4.1. Lab.lab_test

Column	Declared type and column constraints
test_id	UUID PRIMARY KEY
record_id	UUID NOT NULL

Column	Declared type and column constraints
patient_id	UUID NOT NULL
requesting_department_id	UUID NOT NULL
test_type	VARCHAR(50) NOT NULL
requested_date	DATE NOT NULL
performed_date	DATE
status	VARCHAR(20) NOT NULL DEFAULT 'PENDING'
conclusion	TEXT
is_paid	BOOLEAN NOT NULL DEFAULT false
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ
care_contract_version	SMALLINT NOT NULL DEFAULT 0
care_episode_type	VARCHAR(32)
care_episode_id	UUID
source_order_id	UUID
price_code	VARCHAR(64)
clearance_id	UUID
clearance_granted_at	TIMESTAMPTZ
emergency_override_id	UUID

Column	Declared type and column constraints
result_version	INTEGER NOT NULL DEFAULT 0

2.4.2. Lab.lab_result

Column	Declared type and column constraints
result_id	UUID PRIMARY KEY
test_id	UUID NOT NULL REFERENCES lab_test(test_id) ON DELETE CASCADE
indicator	VARCHAR(100) NOT NULL
value	VARCHAR(50) NOT NULL
unit	VARCHAR(20)
reference_range	VARCHAR(50)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()

2.5. Pharmacy

2.5.1. Pharmacy.drug

Column	Declared type and column constraints
drug_id	UUID PRIMARY KEY
drug_name	VARCHAR(150) NOT NULL
active_ingredient	VARCHAR(150)
unit	VARCHAR(20) NOT NULL

Column	Declared type and column constraints
price	DECIMAL(15,2) NOT NULL
stock_quantity	INT NOT NULL DEFAULT 0
expiry_date	DATE NOT NULL
manufacturer	VARCHAR(150)
low_stock_threshold	INT NOT NULL DEFAULT 10
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.5.2. Pharmacy.prescription

Column	Declared type and column constraints
prescription_id	UUID PRIMARY KEY
record_id	UUID
patient_id	UUID NOT NULL
doctor_id	UUID NOT NULL
department_id	UUID NOT NULL
prescribed_date	DATE NOT NULL
total_amount	DECIMAL(15,2) NOT NULL DEFAULT 0
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

Column	Declared type and column constraints
status	VARCHAR(30) NOT NULL DEFAULT 'ACTIVE'
cancelled_at	TIMESTAMPTZ
cancelled_by	UUID
cancellation_reason	VARCHAR(500)
care_contract_version	SMALLINT NOT NULL DEFAULT 0
care_context	VARCHAR(20) NOT NULL DEFAULT 'OUTPATIENT'
care_episode_type	VARCHAR(32)
care_episode_id	UUID
admission_id	UUID
price_code	VARCHAR(64)
lifecycle_at_iso	VARCHAR(35)

2.5.3. Pharmacy.prescription_line

Column	Declared type and column constraints
line_id	UUID PRIMARY KEY
prescription_id	UUID NOT NULL REFERENCES PRESCRIPTION(prescription_id) ON DELETE CASCADE
drug_id	UUID NOT NULL REFERENCES DRUG(drug_id)
quantity	INT NOT NULL
unit_price	DECIMAL(15,2) NOT NULL

Column	Declared type and column constraints
dosage	VARCHAR(255)
line_total	DECIMAL(15,2) NOT NULL
drug_name_snapshot	VARCHAR(150)

2.5.4. Pharmacy.stock_reservation

Column	Declared type and column constraints
reservation_id	UUID PRIMARY KEY
drug_id	UUID NOT NULL REFERENCES DRUG(drug_id)
prescription_id	UUID NOT NULL REFERENCES PRESCRIPTION(prescription_id)
quantity	INT NOT NULL
status	VARCHAR(20) NOT NULL DEFAULT 'RESERVED'
expires_at	TIMESTAMPTZ NOT NULL
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ
release_reason	VARCHAR(50)
released_at	TIMESTAMPTZ
released_by	UUID

2.5.5. Pharmacy.dispense_slip

Column	Declared type and column constraints
dispense_id	UUID PRIMARY KEY
prescription_id	UUID NOT NULL UNIQUE REFERENCES PRESCRIPTION(prescription_id)
status	VARCHAR(20) NOT NULL DEFAULT 'PENDING'
dispensed_at	TIMESTAMPTZ
dispensed_by	UUID
failure_reason	VARCHAR(500)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ
dispensed_actor_type	VARCHAR(30)
lifecycle_at_iso	VARCHAR(35)

2.6. Billing

2.6.1. Billing.invoice

Column	Declared type and column constraints
invoice_id	UUID PRIMARY KEY
patient_id	UUID NOT NULL
created_date	DATE NOT NULL
total_amount	DECIMAL(15,2) NOT NULL

Column	Declared type and column constraints
is_paid	BOOLEAN NOT NULL DEFAULT false
payment_method	VARCHAR(20)
prescription_id	UUID
saga_status	VARCHAR(20) NOT NULL DEFAULT 'NONE'
paid_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.6.2. Billing.billing_account

Column	Declared type and column constraints
account_id	UUID PRIMARY KEY
patient_id	UUID NOT NULL
department_id	UUID NOT NULL
care_episode_type	VARCHAR(32) NOT NULL
care_episode_id	UUID NOT NULL
status	VARCHAR(32) NOT NULL DEFAULT 'OPEN'
currency	CHAR(3) NOT NULL DEFAULT 'VND'
version	BIGINT NOT NULL DEFAULT 0
opened_at	TIMESTAMPTZ NOT NULL

Column	Declared type and column constraints
charge_closed_at	TIMESTAMPTZ
closed_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ NOT NULL DEFAULT now()

2.6.3. Billing.charge

Column	Declared type and column constraints
charge_id	UUID PRIMARY KEY
account_id	UUID NOT NULL REFERENCES BILLING_ACCOUNT(account_id)
patient_id	UUID NOT NULL
department_id	UUID NOT NULL
source_type	VARCHAR(32) NOT NULL
source_id	UUID NOT NULL
price_code	VARCHAR(64) NOT NULL
description	VARCHAR(255) NOT NULL
quantity	NUMERIC(19,4) NOT NULL DEFAULT 1 CHECK (quantity > 0)
unit_amount	DECIMAL(19,2) NOT NULL CHECK (unit_amount >= 0)
gross_amount	DECIMAL(19,2) NOT NULL CHECK (gross_amount >= 0)
status	VARCHAR(20) NOT NULL DEFAULT 'POSTED'

Column	Declared type and column constraints
void_reason	VARCHAR(500)
incurred_at	TIMESTAMPTZ NOT NULL
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
reconciled_result_id	UUID

2.6.4. Billing.payment_request

Column	Declared type and column constraints
payment_request_id	UUID PRIMARY KEY
invoice_id	UUID NOT NULL UNIQUE
account_id	UUID NOT NULL REFERENCES BILLING_ACCOUNT(account_id)
purpose	VARCHAR(32) NOT NULL
status	VARCHAR(24) NOT NULL DEFAULT 'PENDING'
requested_amount	DECIMAL(19,2) NOT NULL CHECK (requested_amount >= 0)
currency	CHAR(3) NOT NULL DEFAULT 'VND'
expires_at	TIMESTAMPTZ
created_by	UUID
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
completed_at	TIMESTAMPTZ

2.6.5. Billing.payment_transaction

Column	Declared type and column constraints
transaction_id	UUID PRIMARY KEY
account_id	UUID NOT NULL REFERENCES BILLING_ACCOUNT(account_id)
payment_request_id	UUID REFERENCES PAYMENT_REQUEST(payment_request_id)
transaction_type	VARCHAR(20) NOT NULL
classification	VARCHAR(24) NOT NULL
status	VARCHAR(20) NOT NULL
amount	DECIMAL(19,2) NOT NULL CHECK (amount > 0)
currency	CHAR(3) NOT NULL DEFAULT 'VND'
payment_method	VARCHAR(20) NOT NULL
provider_reference	VARCHAR(120)
idempotency_key	VARCHAR(120) NOT NULL UNIQUE
original_transaction_id	UUID REFERENCES PAYMENT_TRANSACTION(transaction_id)
completed_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
actor_account_id	UUID

2.6.6. Billing.payment_allocation

Column	Declared type and column constraints
allocation_id	UUID PRIMARY KEY
transaction_id	UUID NOT NULL REFERENCES PAYMENT_TRANSACTION(transaction_id)
charge_id	UUID NOT NULL REFERENCES CHARGE(charge_id)
amount	DECIMAL(19,2) NOT NULL CHECK (amount > 0)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()

2.6.7. Billing.financial_clearance

Column	Declared type and column constraints
clearance_id	UUID PRIMARY KEY
account_id	UUID NOT NULL REFERENCES BILLING_ACCOUNT(account_id)
payment_request_id	UUID NOT NULL REFERENCES PAYMENT_REQUEST(payment_request_id)
invoice_id	UUID NOT NULL
patient_id	UUID NOT NULL
purpose	VARCHAR(32) NOT NULL
care_episode_type	VARCHAR(32) NOT NULL
care_episode_id	UUID NOT NULL

Column	Declared type and column constraints
appointment_id	UUID
record_id	UUID
lab_test_ids	UUID[] NOT NULL DEFAULT '{}'
prescription_id	UUID
admission_id	UUID
surgery_case_id	UUID
amount	DECIMAL(19,2) NOT NULL CHECK (amount >= 0)
currency	CHAR(3) NOT NULL DEFAULT 'VND'
payment_method	VARCHAR(20) NOT NULL
emergency_override	BOOLEAN NOT NULL DEFAULT FALSE
expires_at	TIMESTAMPTZ
granted_at	TIMESTAMPTZ NOT NULL
revoked_at	TIMESTAMPTZ

2.6.8. Notification.notification

Column	Declared type and column constraints
notification_id	UUID PRIMARY KEY
patient_id	UUID NOT NULL
title	VARCHAR(255) NOT NULL

Column	Declared type and column constraints
content	TEXT NOT NULL
channel	VARCHAR(10) NOT NULL
recipient_address	VARCHAR(150)
status	VARCHAR(10) NOT NULL DEFAULT 'PENDING'
failure_reason	VARCHAR(255)
retry_count	INT NOT NULL DEFAULT 0
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
sent_at	TIMESTAMPTZ
template_key	VARCHAR(64)
source_event_id	UUID
source_event_type	VARCHAR(100)
source_id	UUID
correlation_id	VARCHAR(120)
sensitivity	VARCHAR(32) NOT NULL DEFAULT 'STANDARD'

2.7. Report

2.7.1. Report.daily_visit_report

Column	Declared type and column constraints
report_id	UUID PRIMARY KEY

Column	Declared type and column constraints
report_date	DATE NOT NULL
department_id	UUID
visit_count	INT NOT NULL DEFAULT 0
lab_count	INT NOT NULL DEFAULT 0
prescription_count	INT NOT NULL DEFAULT 0
revenue	DECIMAL(15,2) NOT NULL DEFAULT 0
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

2.7.2. Report.monthly_revenue_report

Column	Declared type and column constraints
report_id	UUID PRIMARY KEY
month	INT NOT NULL
year	INT NOT NULL
department_id	UUID
total_revenue	DECIMAL(15,2) NOT NULL DEFAULT 0
invoice_count	INT NOT NULL DEFAULT 0
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at	TIMESTAMPTZ

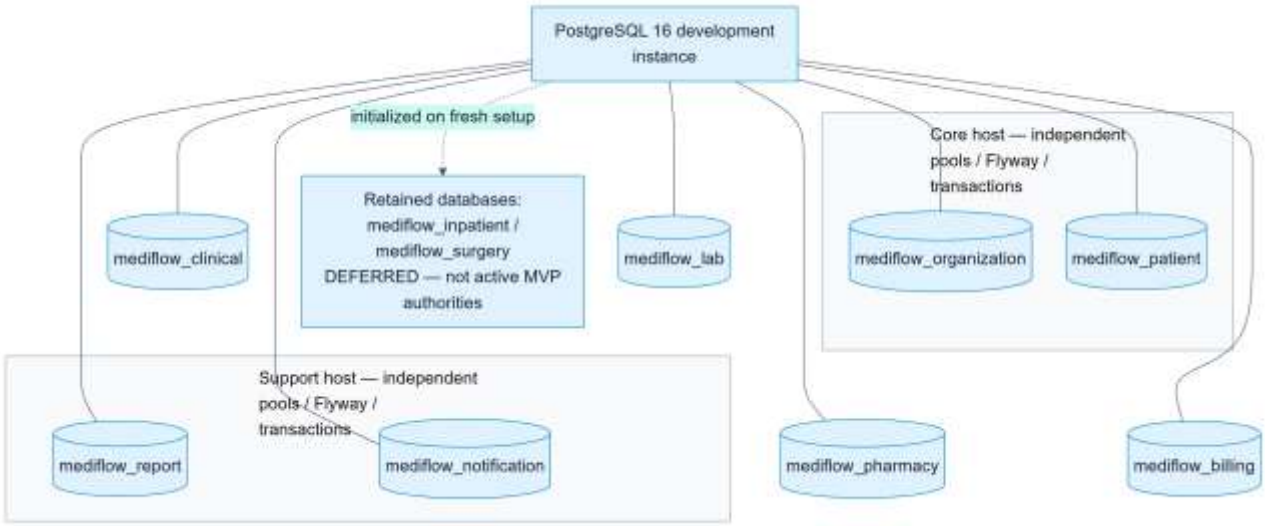
2.7.3. Report.drug_statistic

Column	Declared type and column constraints
statistic_id	UUID PRIMARY KEY
drug_id	UUID NOT NULL
drug_name	VARCHAR(150) NOT NULL
report_date	DATE NOT NULL
department_id	UUID
dispensed_quantity	INT NOT NULL DEFAULT 0
updated_at	TIMESTAMPTZ

2.8. Constraints and persistence strategy

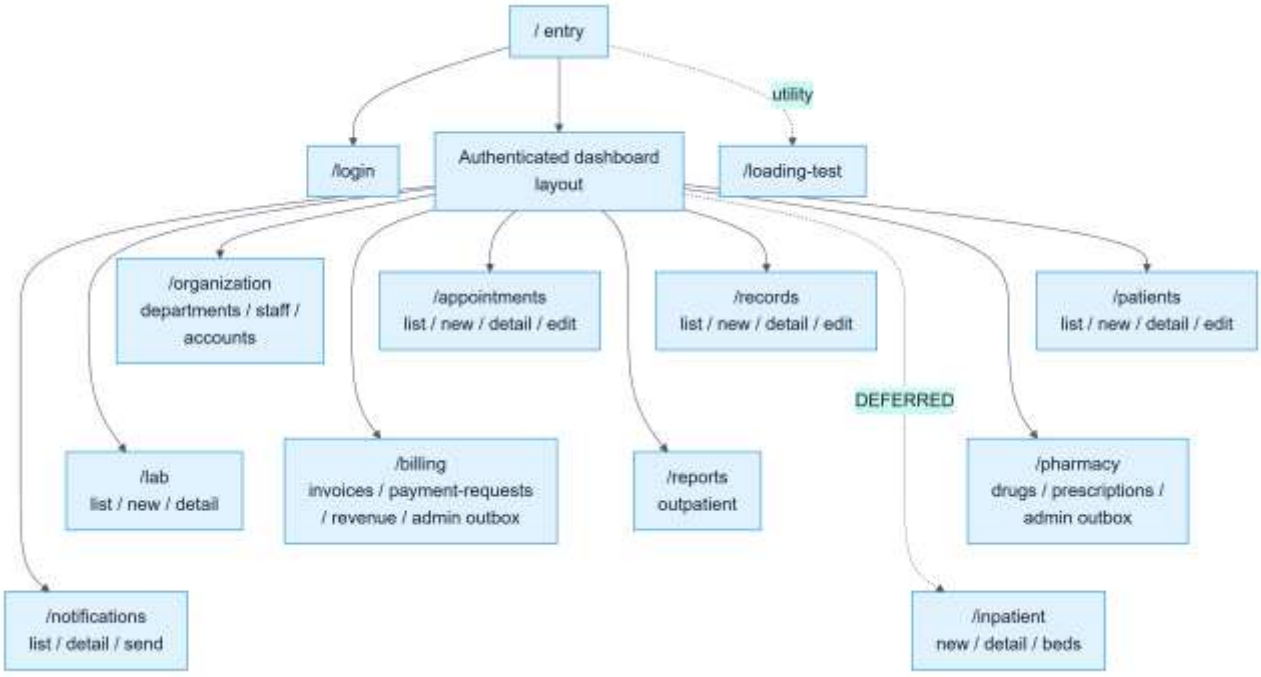
- UUIDs are business identifiers; cross-context references have no JPA relationship/FK. Organization.account.patient_id remains a logical Patient reference even inside the Core JVM.
- Money uses BigDecimal and DDL-specific NUMERIC/DECIMAL: V0 commonly (15,2), ledger (19,2). This report does not normalize all precision to one type.
- Appointment has a patient/date PENDING uniqueness guard; medical_record has a unique appointment; prescription_line quantity > 0; stock/price are nonnegative; reservation has prescription/drug uniqueness.
- Billing has unique charge (source_type, source_id, price_code) and payment_transaction.idempotency_key; FKs/allocations stay within Billing.
- Business timestamps use TIMESTAMPTZ/Instant, dates use DATE/LocalDate. Lab result.value is VARCHAR, preserving textual results and reference_range.
- Outbox commits with state; inbox/receipt commits with effects. Retries preserve eventId/bytes; poison messages reach DLQ after bounded retry; HELD rows are not automatically released.
- Released Flyway migrations are append-only. Indexes and multi-column CHECKs live in the linked DDL, beyond the column summaries above.

2.9. Database Diagram

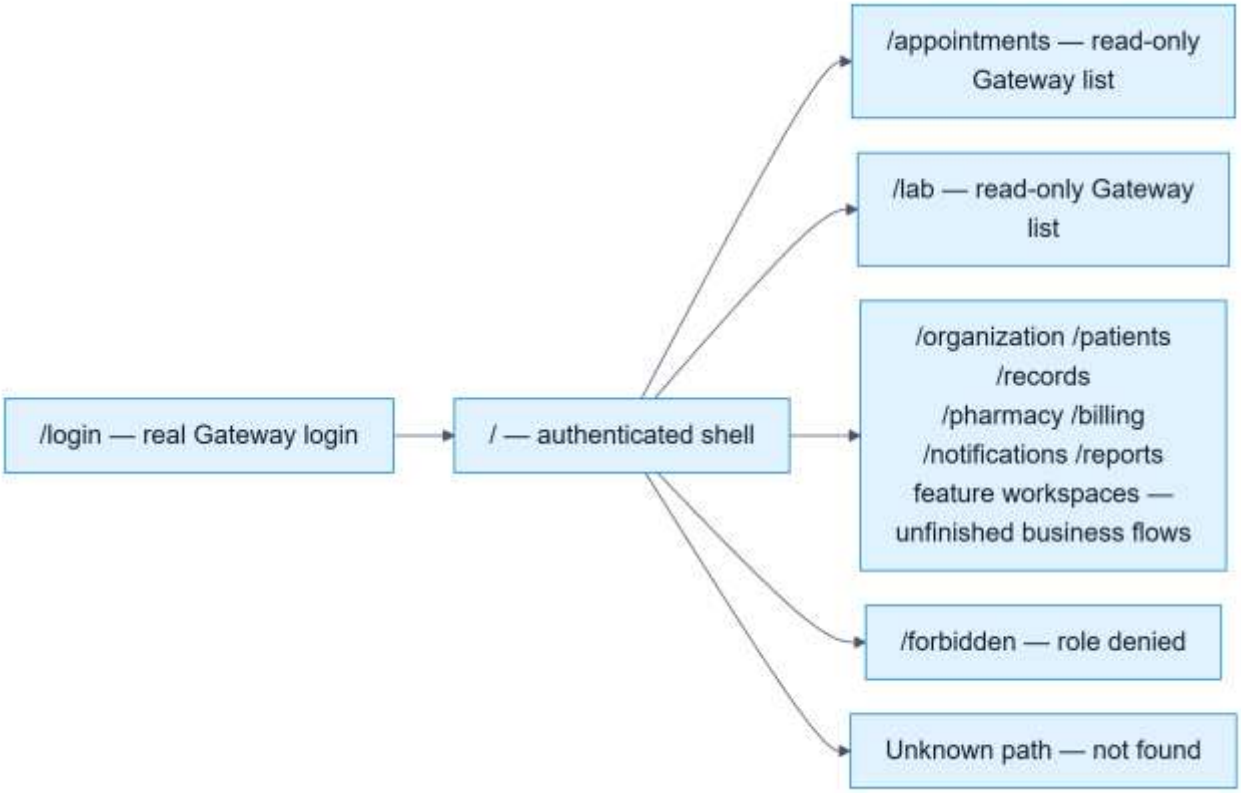

PostgreSQL 16 development instance
initialized on fresh setup
Retained databases: mediflow_inpatient / mediflow_surgery DEFERRED — not active MVP authorities
Core host — independent pools / Flyway / transactions
Support host — independent pools / Flyway / transactions
mediflow_clinical
mediflow_lab
mediflow_organization
mediflow_patient
mediflow_report
mediflow_notification
mediflow_pharmacy
mediflow_billing

SiteMap

1. Web sitemap by route group

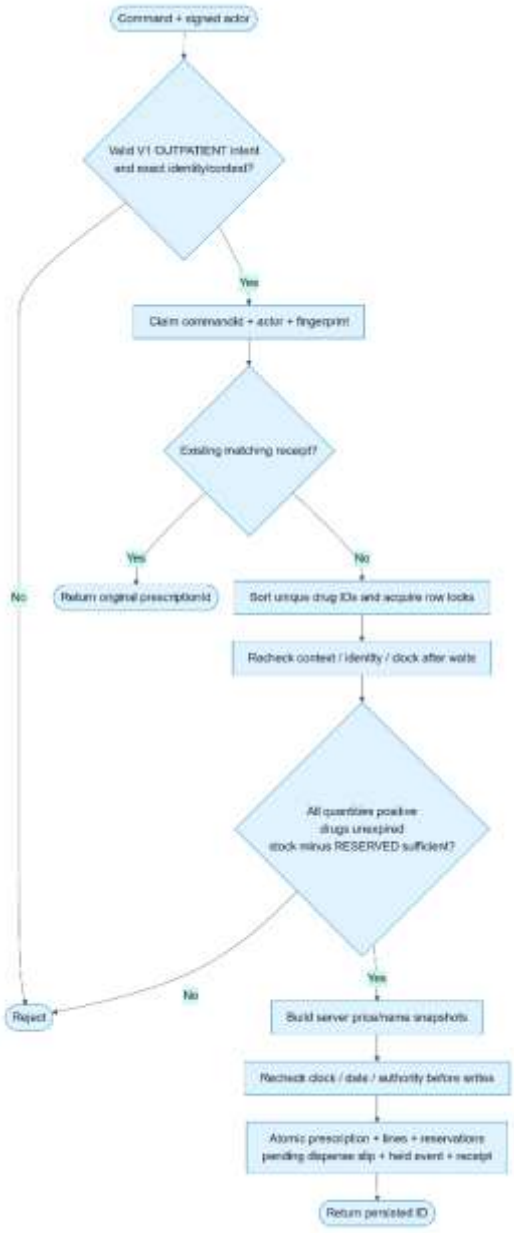


2. Mobile sitemap and implementation level



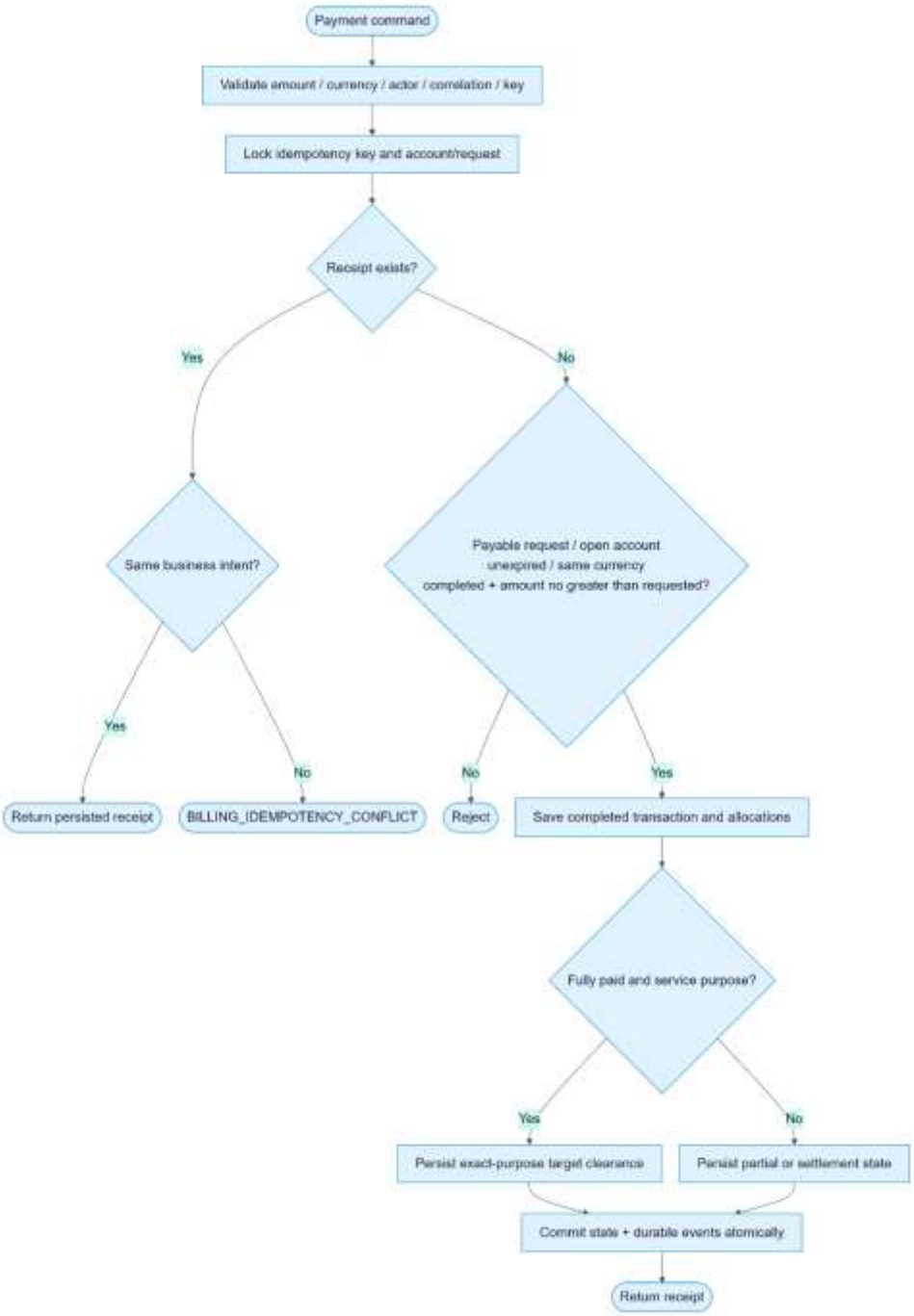
Algorithm

1. Prescription creation and stock reservation

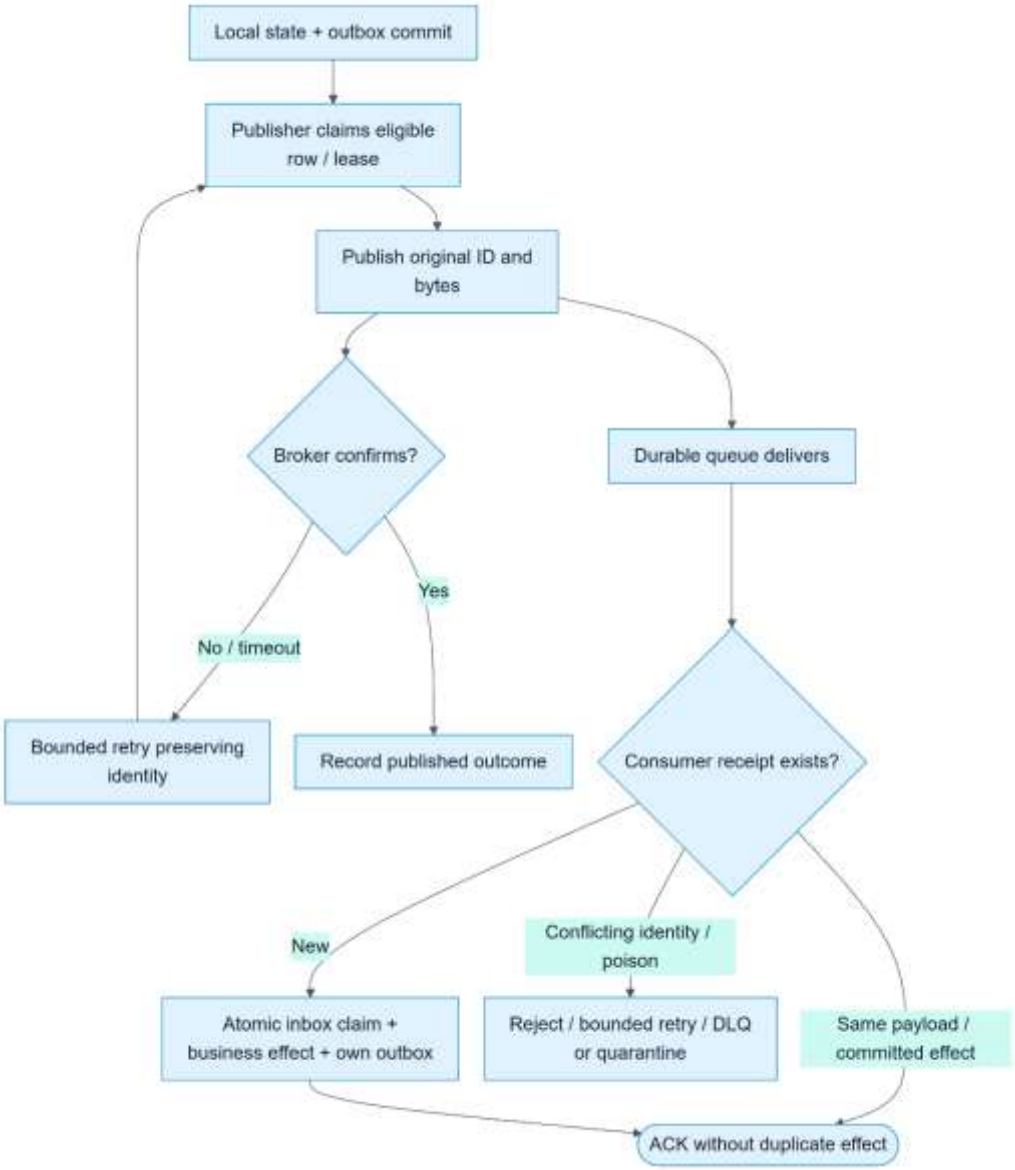


2. Idempotent payment recording

	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		


Payment command
Validate amount / currency / actor / correlation / key
Lock idempotency key and account/request
Receipt exists?
Yes
No
Same business intent?
Payable request / open account
unexpired / same currency
completed + amount no greater than requested?
Yes
No
No
Yes
Return persisted receipt
BILLING_IDEMPOTENCY_CONFLICT
Reject
Save completed transaction and allocations
Fully paid and service purpose?
Yes
No
Persist exact-purpose target clearance
Persist partial or settlement state
Commit state + durable events atomically
Return receipt

3. Event delivery, deduplication and recovery



Tasksheet Review 2

Project Ref. No		Project Title	Activity Plan Prepared By	Date of Preparation of Activity Plan			
No.	**Task**			**Actual Start Date**	**Actuals Days**	**Teammate Names**	**Status**
1	Architecture Design	Mediflow	Dang Vinh	18/10/2026	2	All members	Completed
2	Data Flow Diagram			18/10/2026	2	All members	Completed
3	Entity Relationship Diagram			18/10/2026	2	All members	Completed

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

4	Table Design	Mediflow	Dang Vinh	18/10/2026	2	All members	Completed
5	SiteMap			18/10/2026	2	All members	Completed
6	Algorithm			18/10/2026	2	All members	Completed
7	Tasksheet review			18/10/2026	2	All members	Completed

Review 03

Screen shots

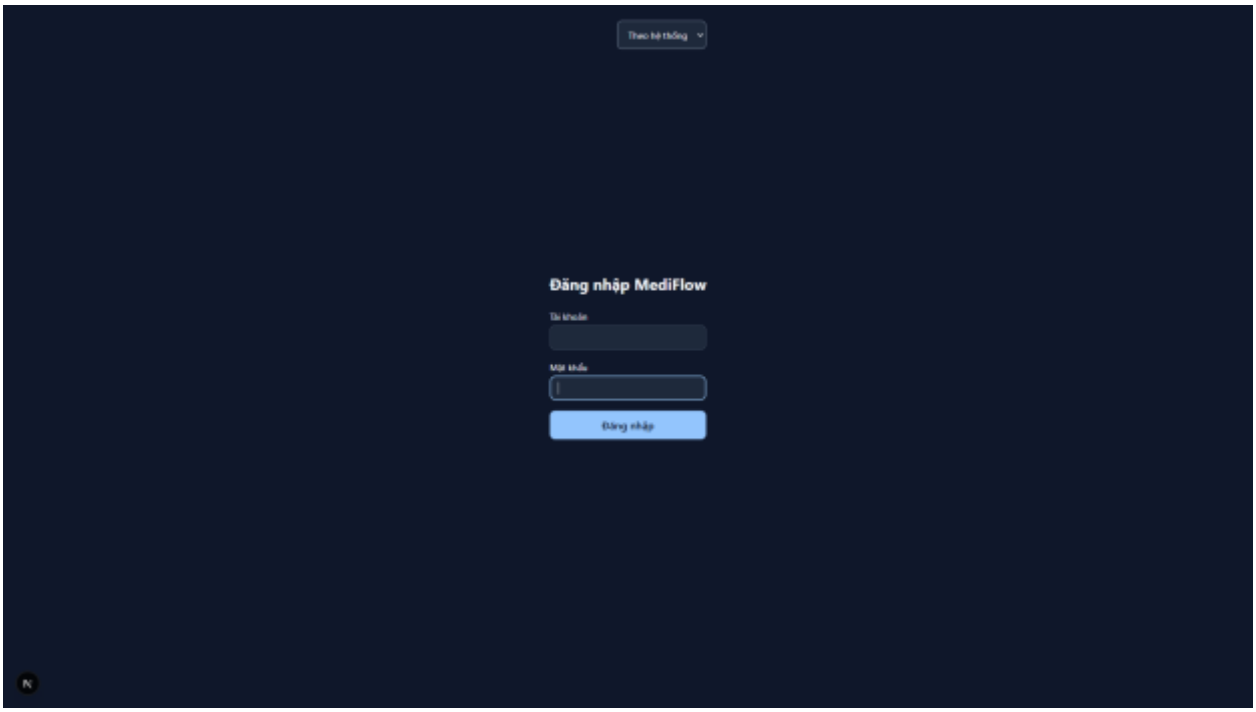
Refreshed on 22 October 2026 from the source candidate based on commit 4eac7ba2. Screenshots use the running web and Flutter clients through the real Gateway, PostgreSQL, RabbitMQ and six packaged business applications. Patient and staff records are synthetic demonstration data.

Mobile evidence uses Flutter web at a 430 by 932 phone viewport, captured at 1290 by 2796 pixels. Web captures are 1904 or 1920 by 1080 pixels. This does not establish native Android or iOS execution. Lab clearance rejections, restricted name lookups and unavailable outpatient report coverage are described as observed. Inpatient care and surgery remain deferred.

1. Web GUI

1.1. Shared screens

1.1.1. MediFlow sign-in screen.

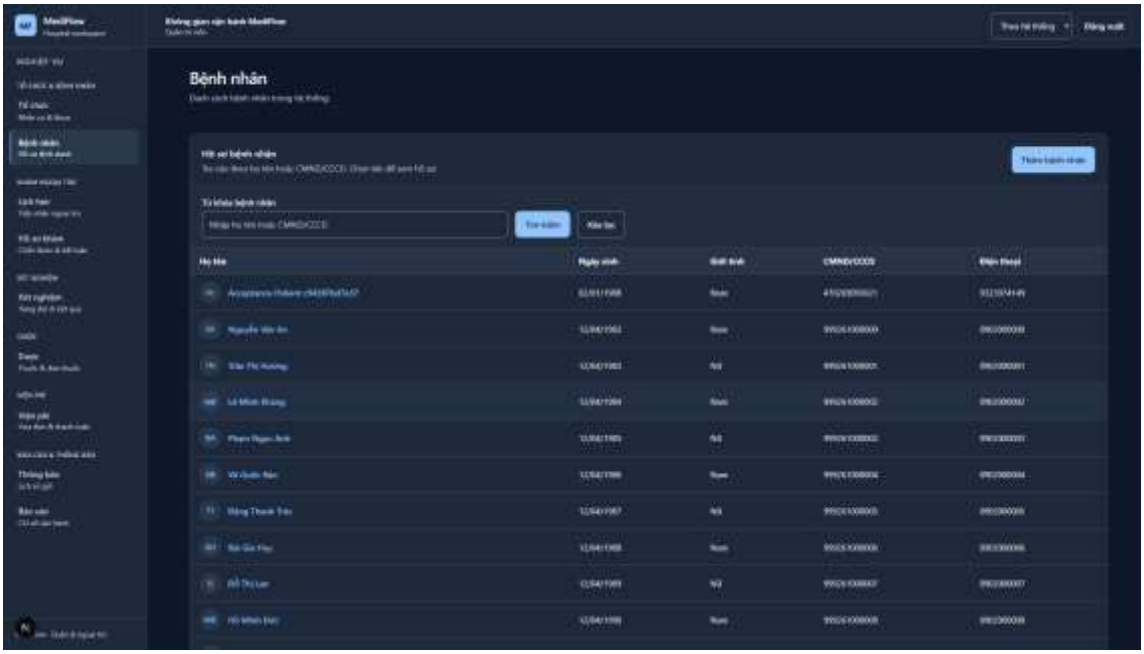


1.1.2. Public MediFlow landing page links the outpatient workspaces.

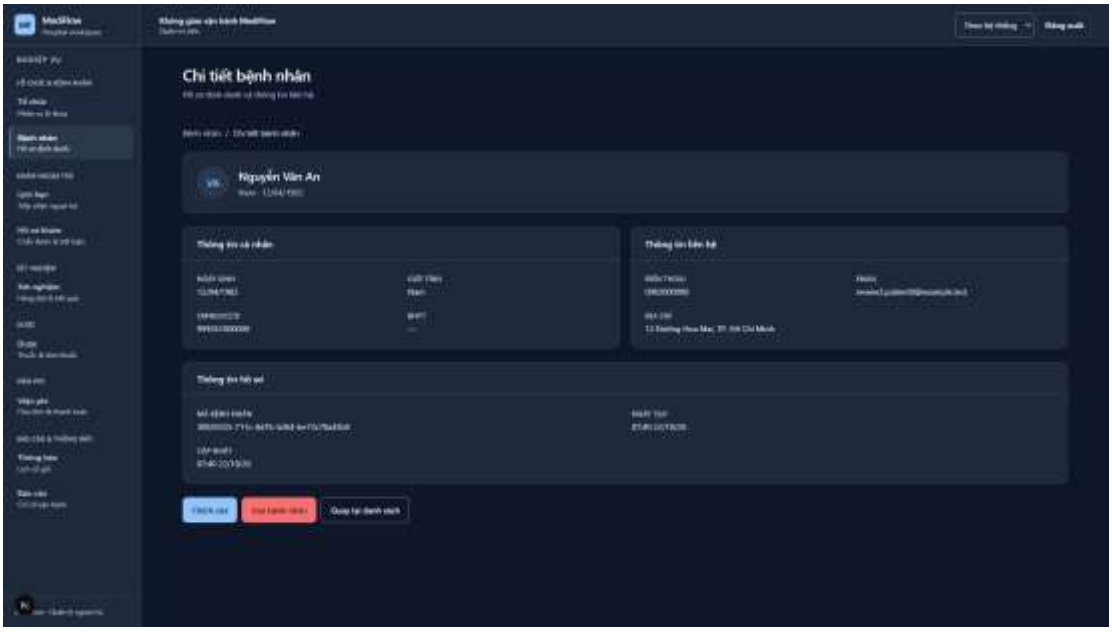


1.2. Administrator screens

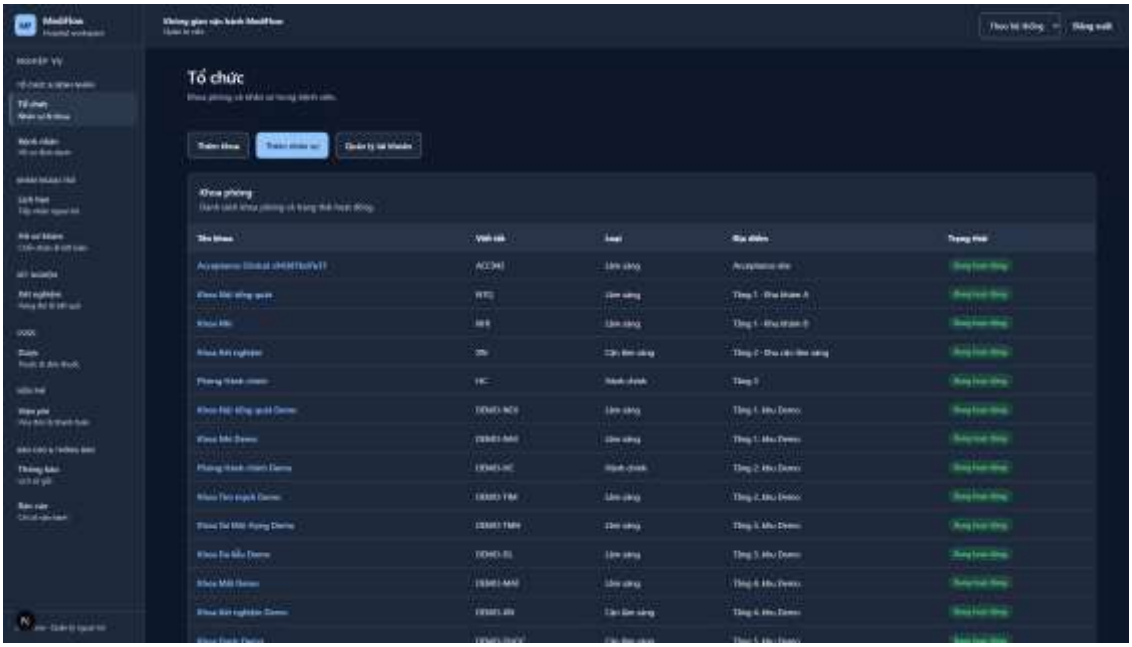
1.2.1. Administrator searches and opens patient identity records.



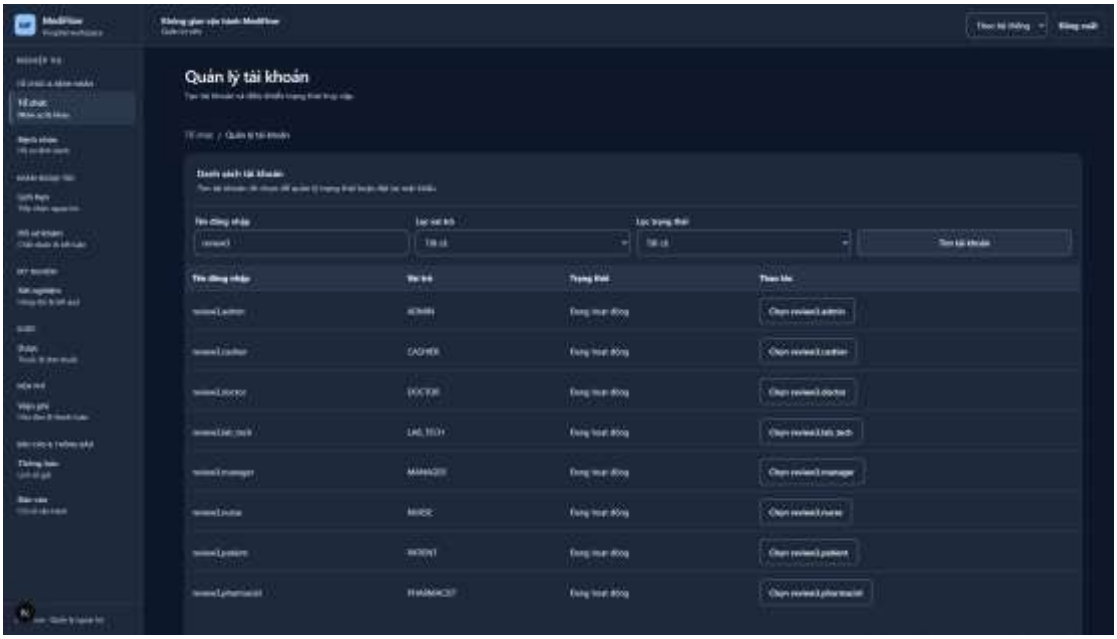
1.2.2. Patient identity detail shows the patient name, demographics and contact data.



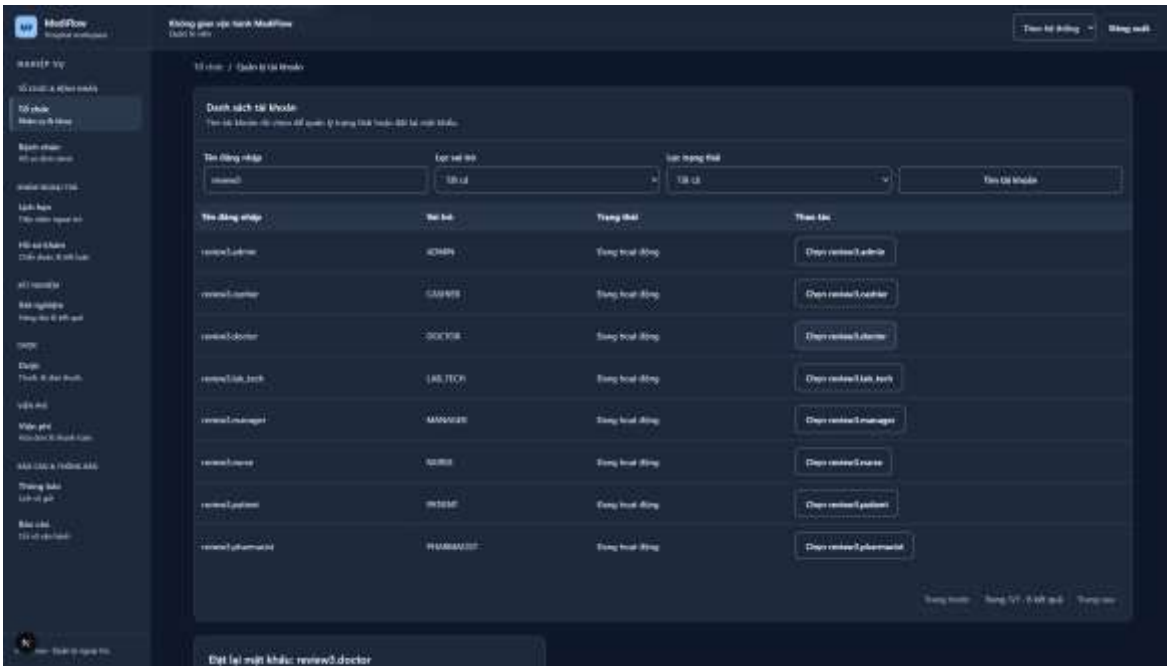
1.2.3. Administrator organization workspace lists departments and staff with management controls.



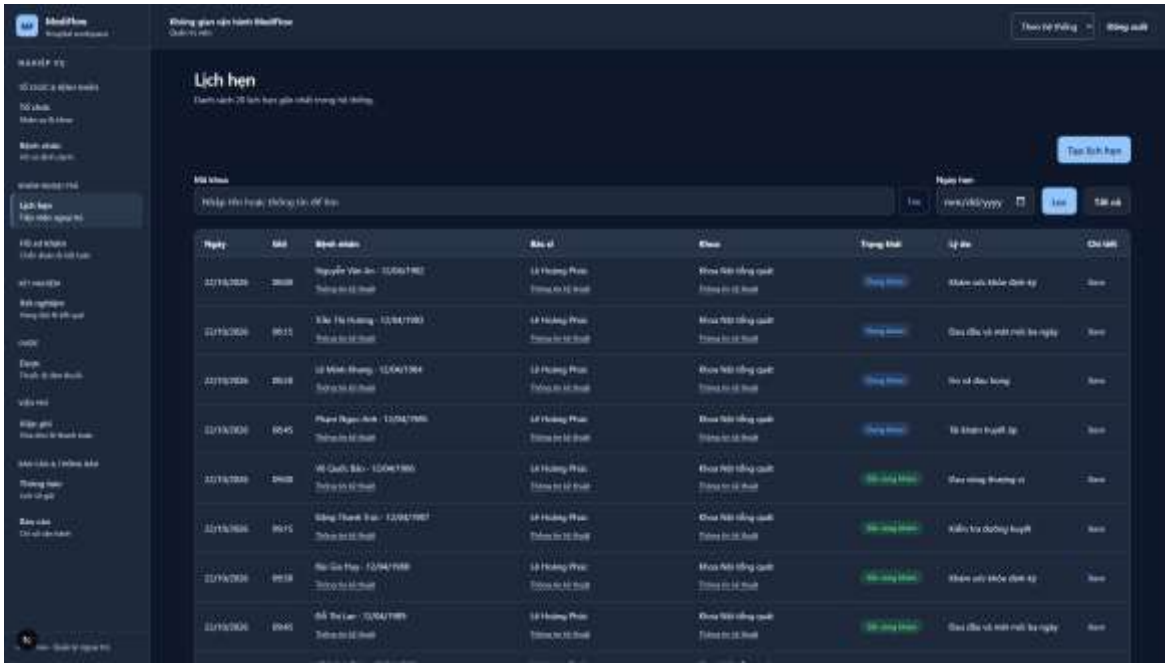
1.2.4. Administrator filters the eight role accounts



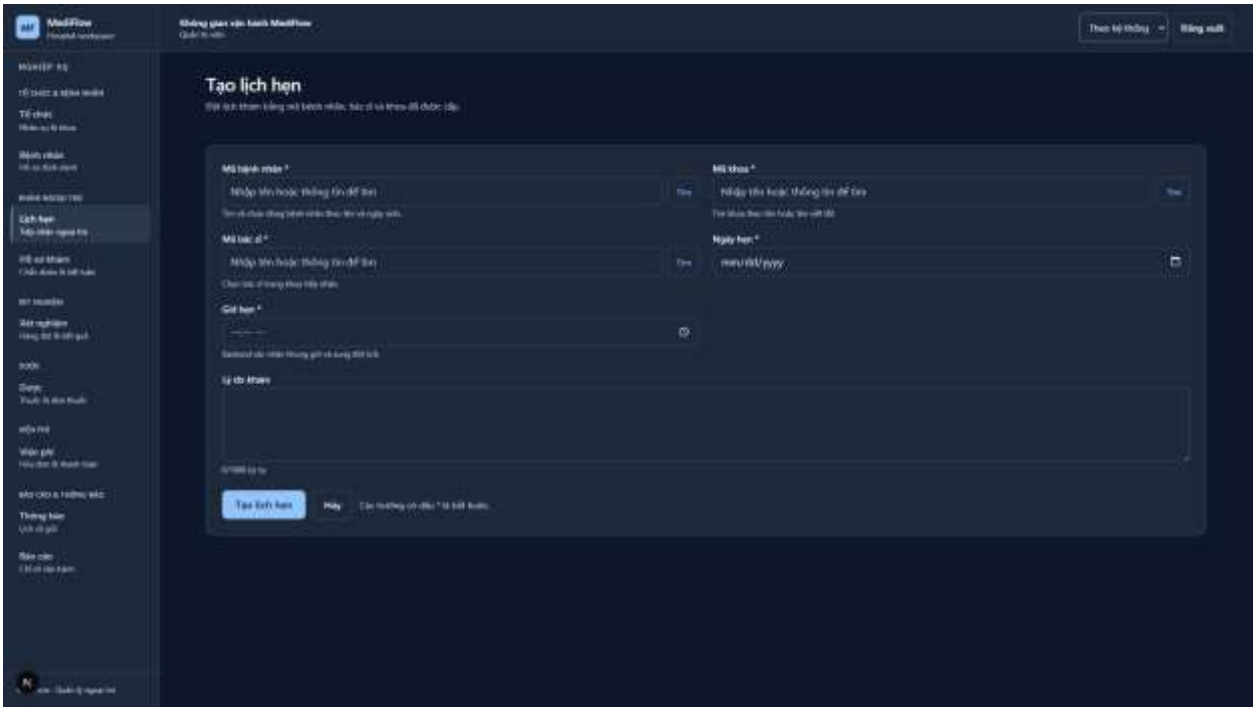
1.2.5. Selecting the doctor account populates account-management controls; credentials are not changed.



1.2.6. Administrator appointment list resolves patient, doctor and department names.

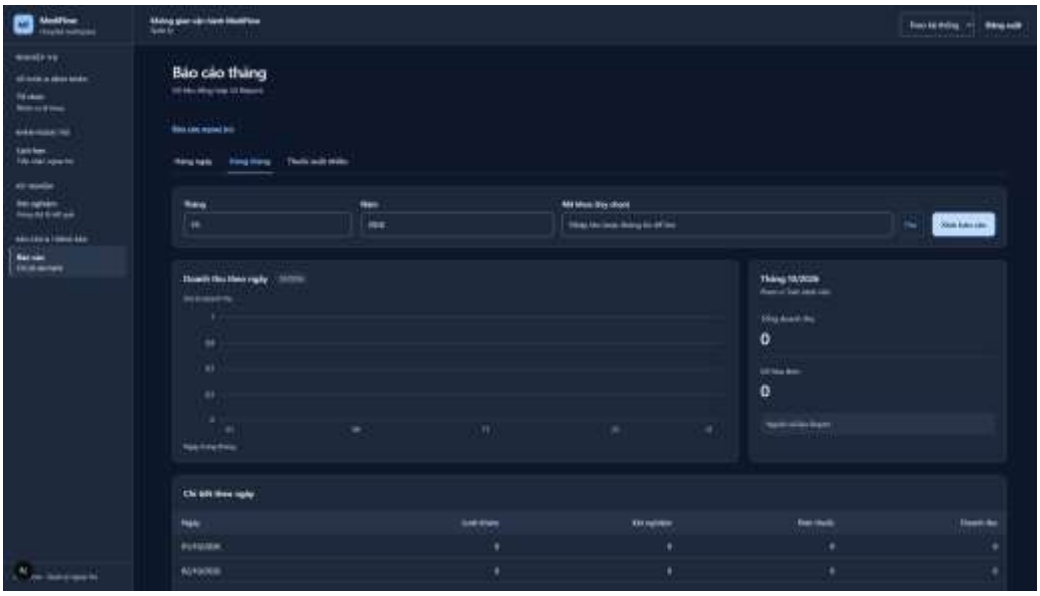


1.2.7. Appointment creation uses name lookup for patient, department and doctor.

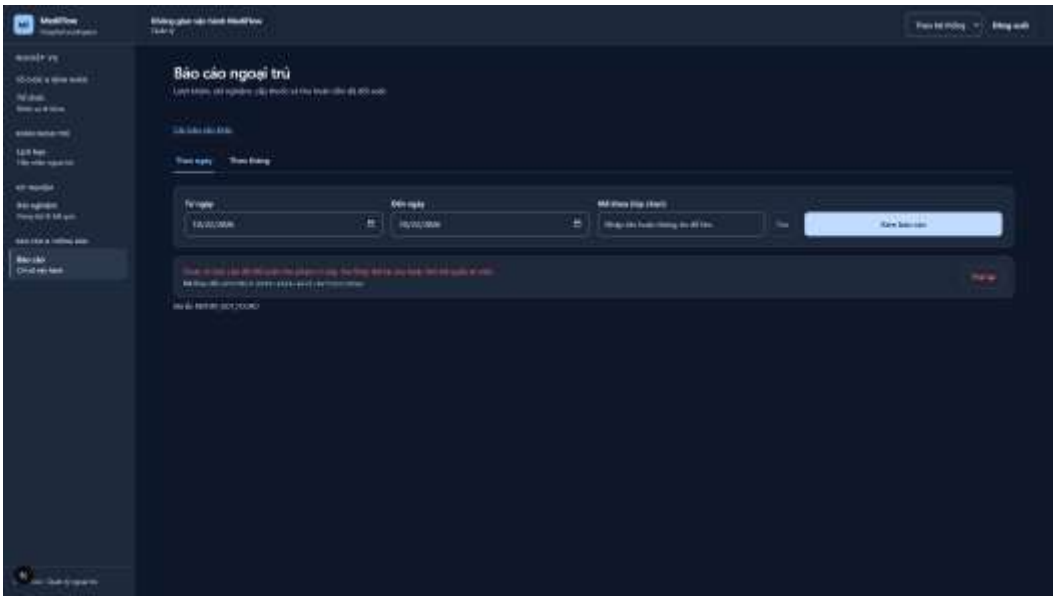


1.3. Manager screens

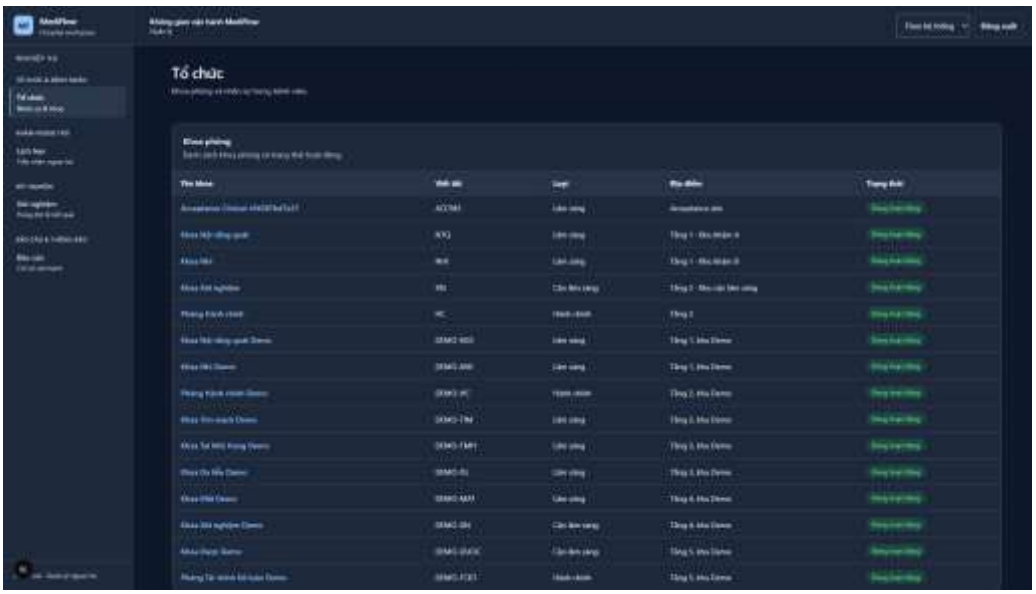
1.3.1. Manager monthly report returns the current Report aggregate; zero values are not presented as reconciled outpatient coverage.



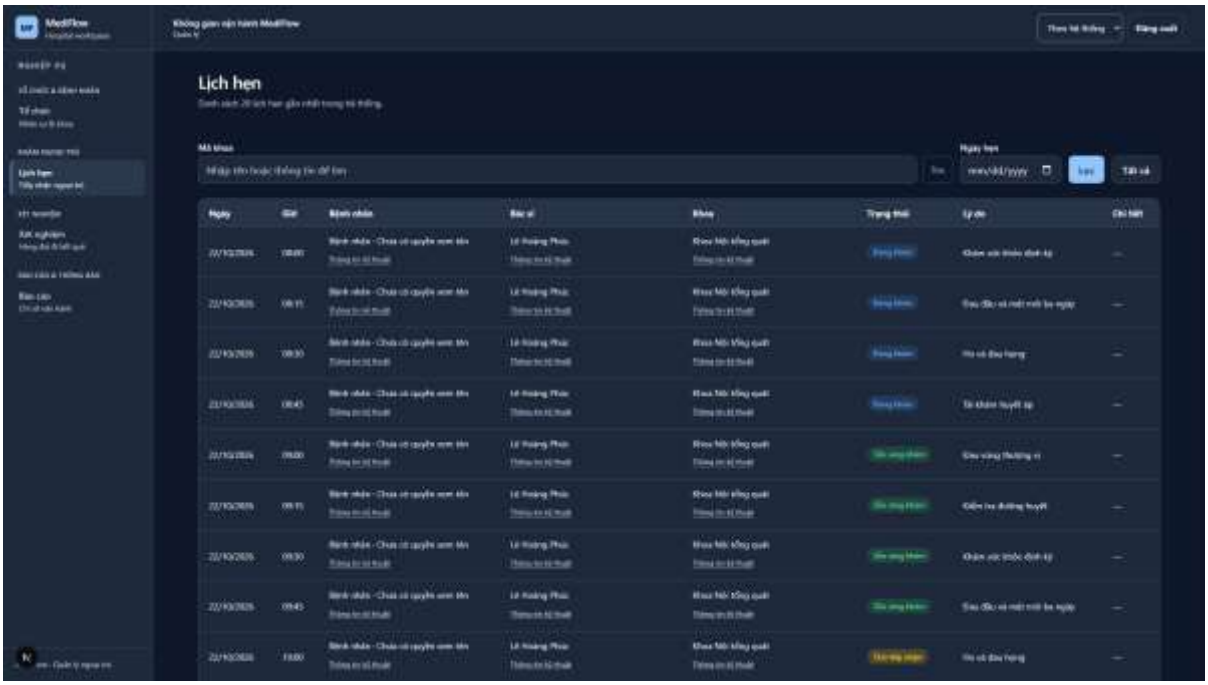
1.3.2. Outpatient report reports missing reconciled coverage (REPORT_NOT_FOUND) for the demo date; no audited figures are fabricated.



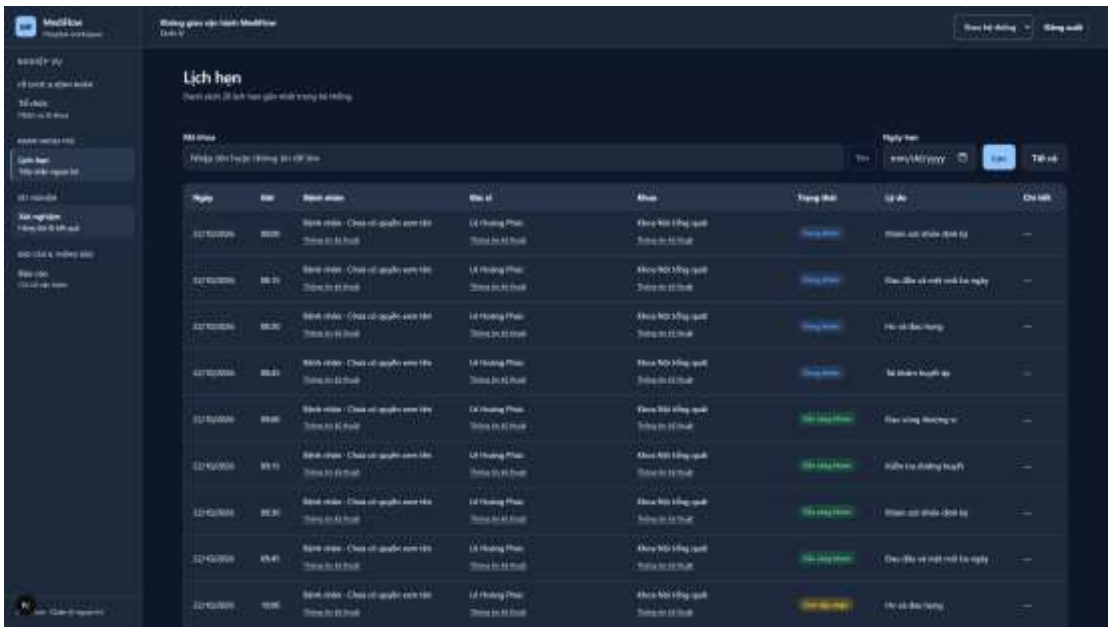
1.3.3. Manager reads departments and staff without administrator-only account controls.



1.3.4. Manager appointment overview respects the role's identity lookup permissions.

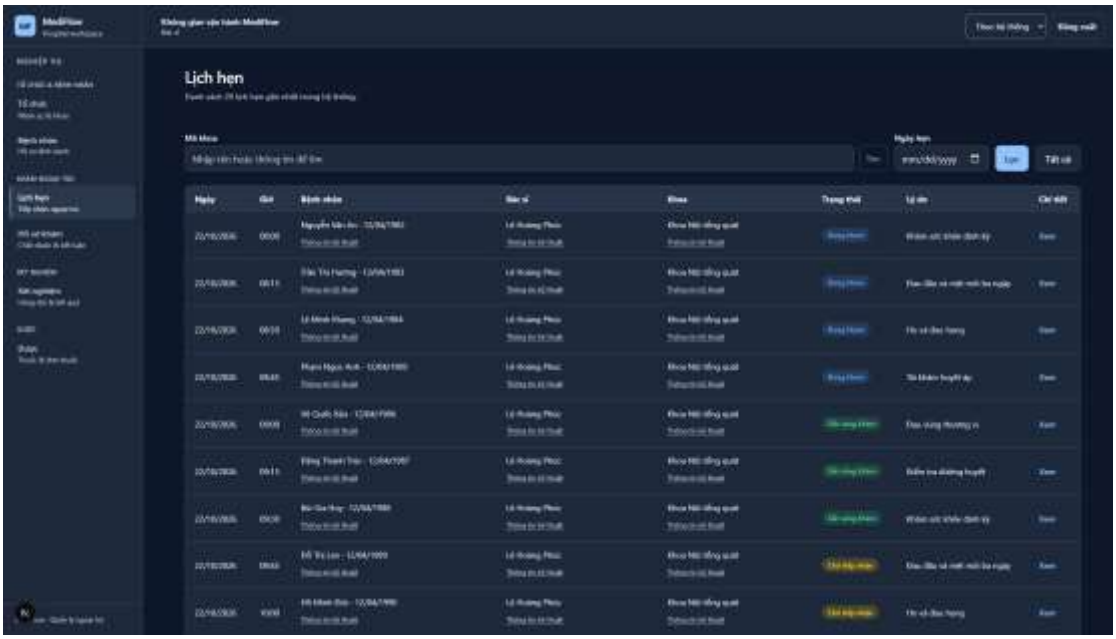


1.3.5. Manager views the laboratory queue and financial states without processing commands.



1.4. Doctor screens

1.4.1. Doctor appointment queue with resolved clinical reference names



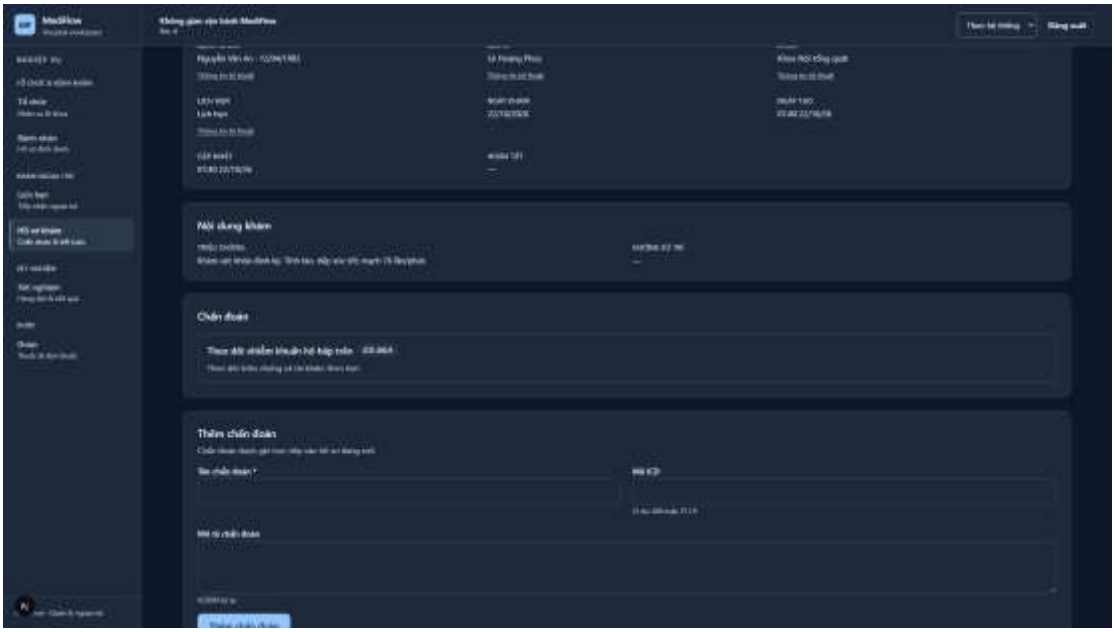
1.4.2. Visit timeline with check-in clearance and examination milestones



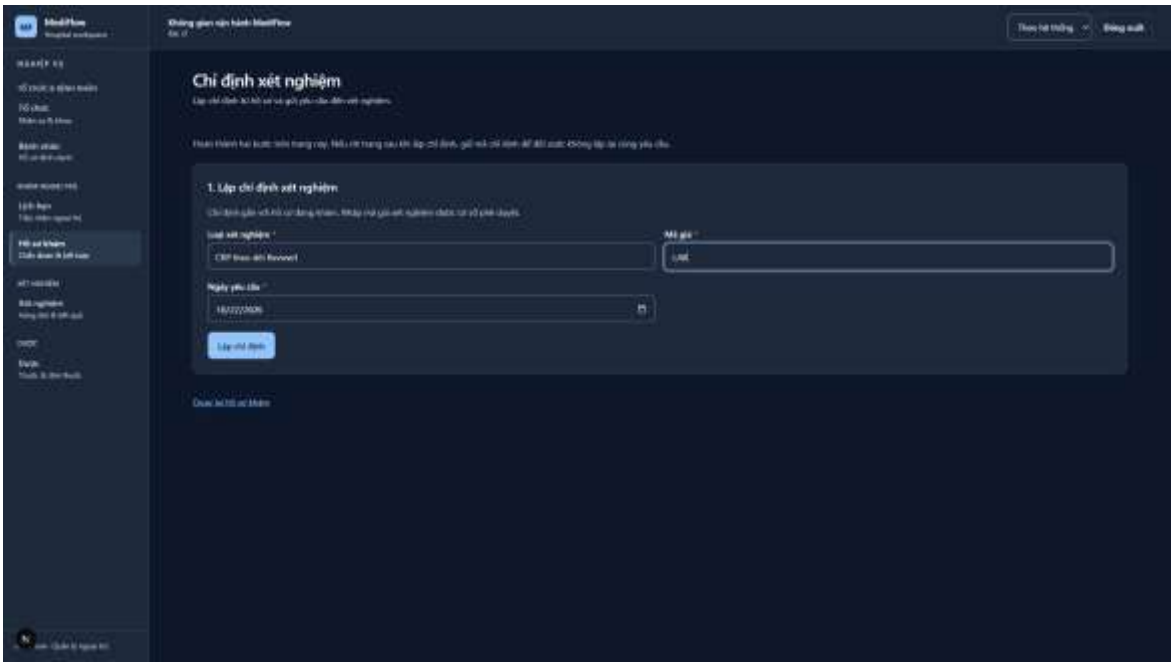
1.4.3. Clinical record with symptoms and resolved patient doctor and department



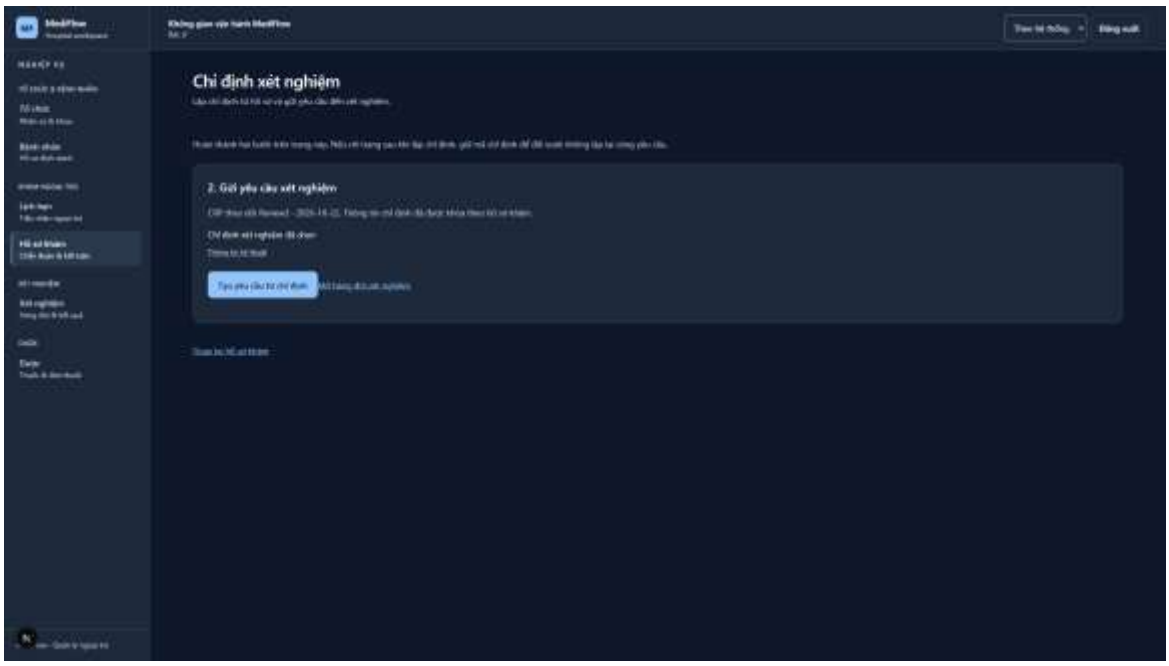
1.4.4. ICD diagnosis and controlled visit completion form



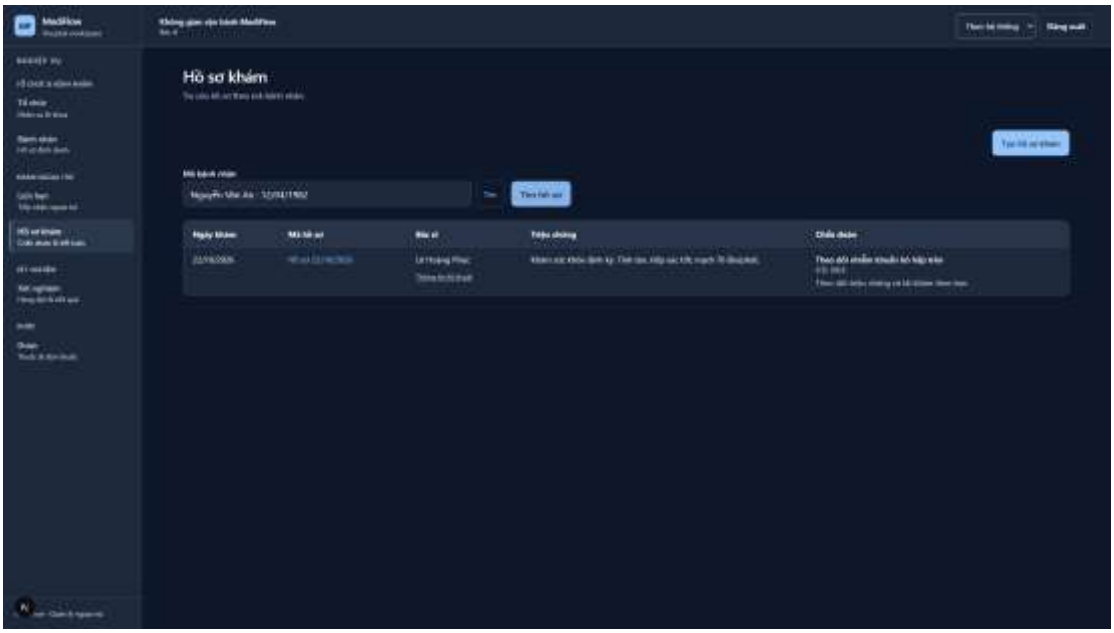
1.4.5. Draft source-linked laboratory order uses the configured LAB catalogue code; this draft is not submitted.



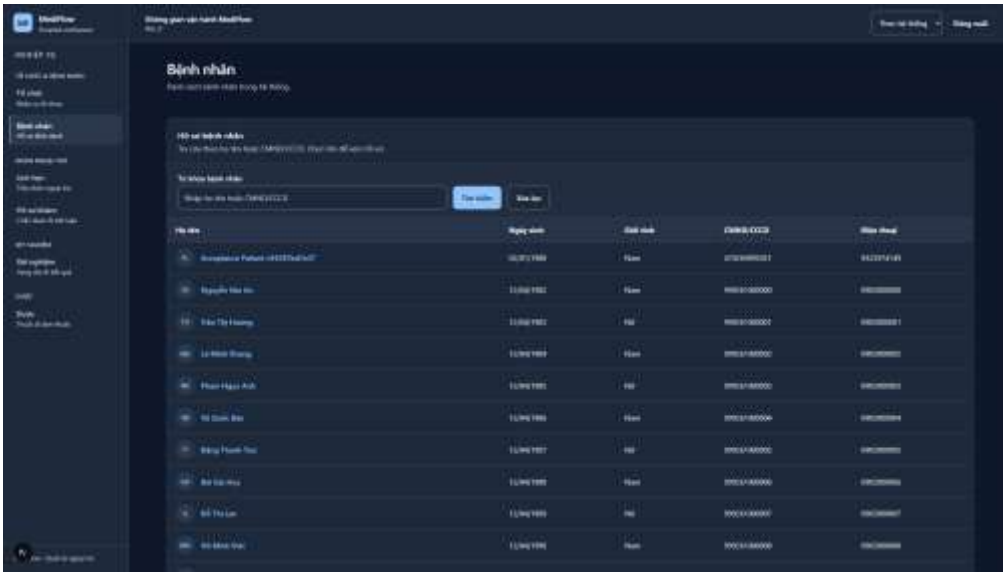
1.4.6. Clinical source order created with configured LAB price code; second-step Lab submission remains explicit.



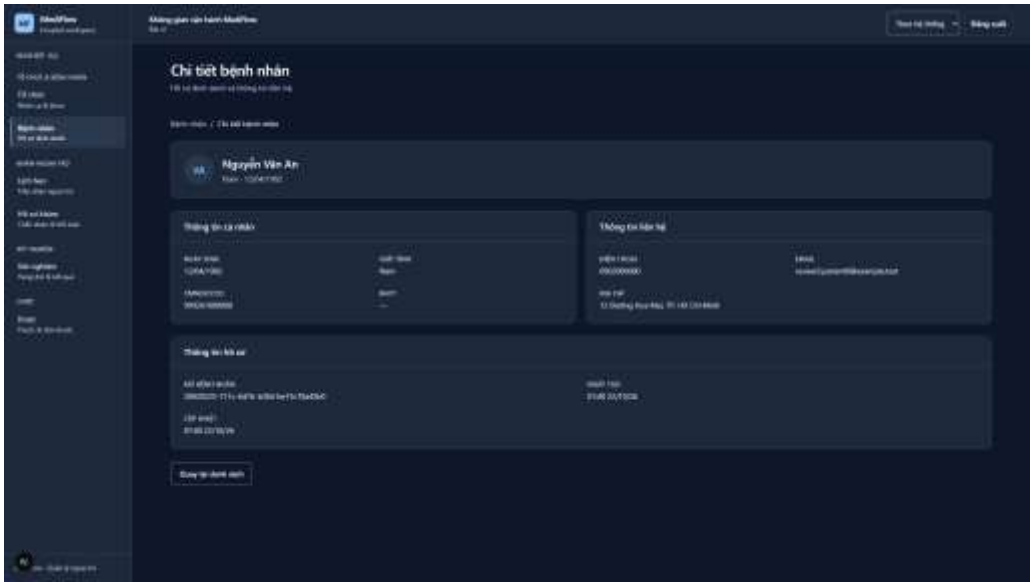
1.4.7. Patient name based clinical record search



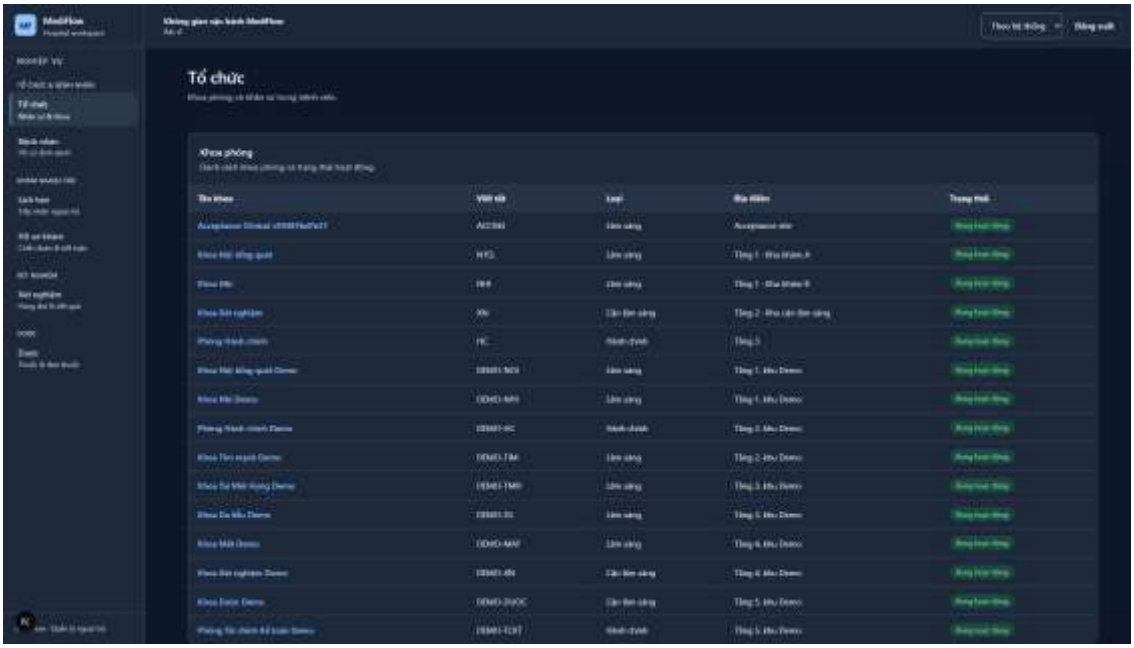
1.4.8. Doctor reads seeded patient identities



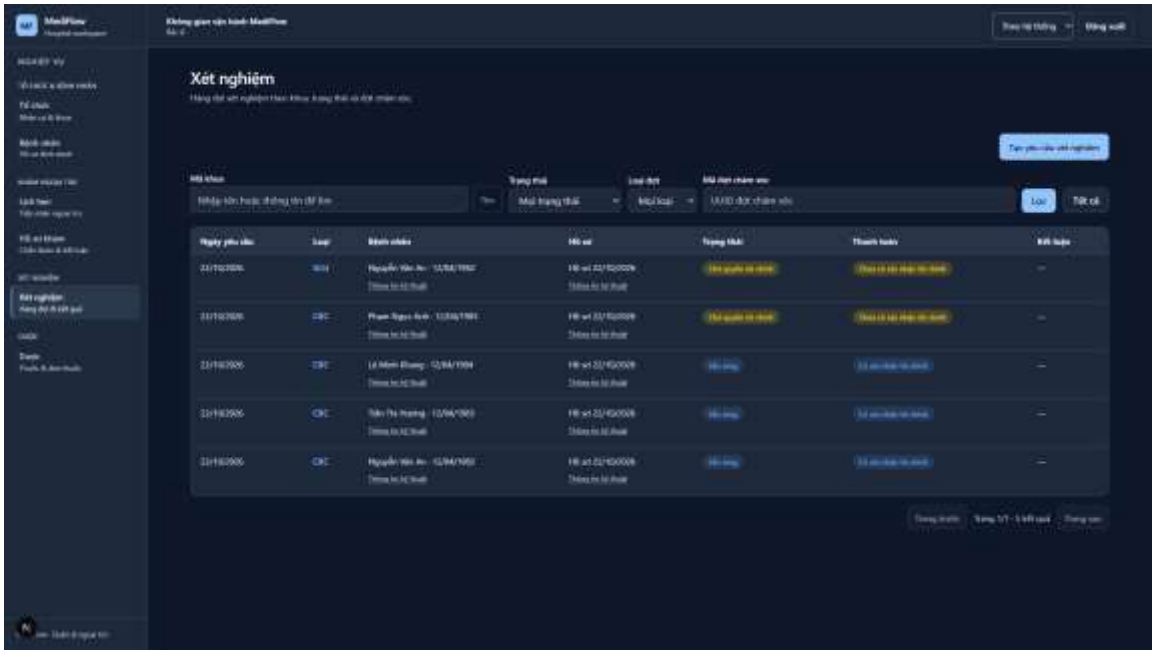
1.4.9. Patient detail with synthetic identity and contact information



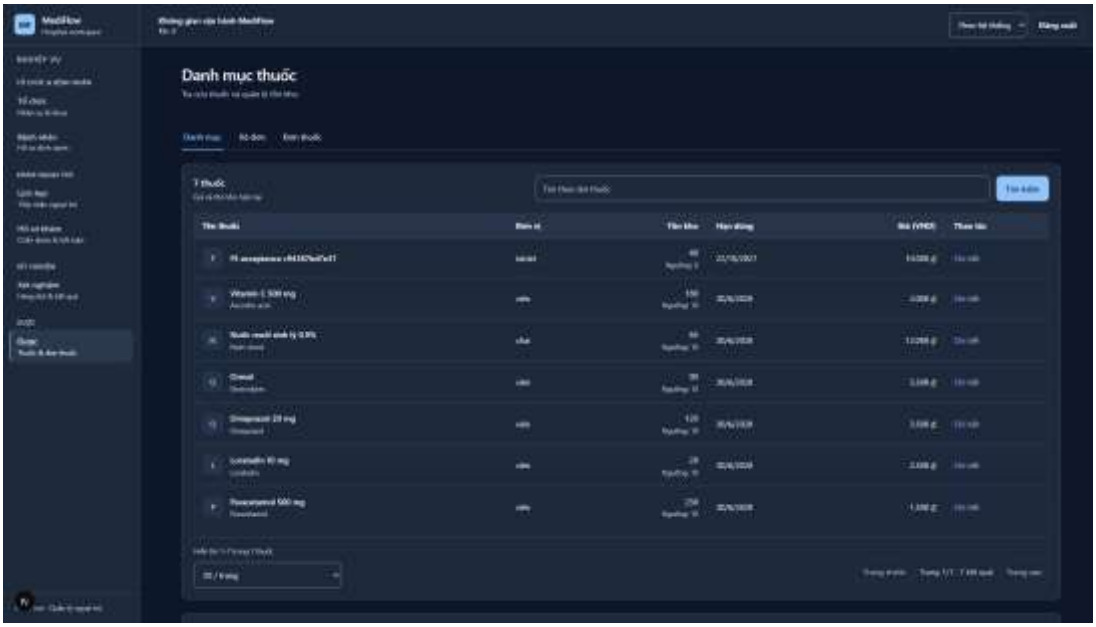
1.4.10. Doctor reads departments and staff references



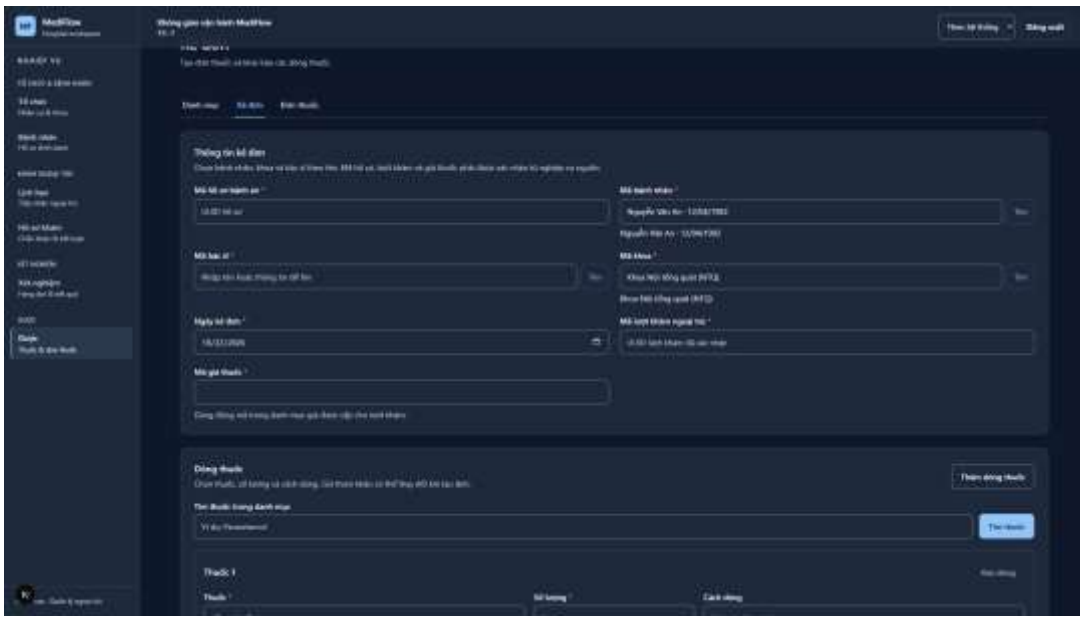
1.4.11. Laboratory queue with clinical record labels and financial readiness



1.4.12. Doctor views the medicine catalogue with stock and prices.

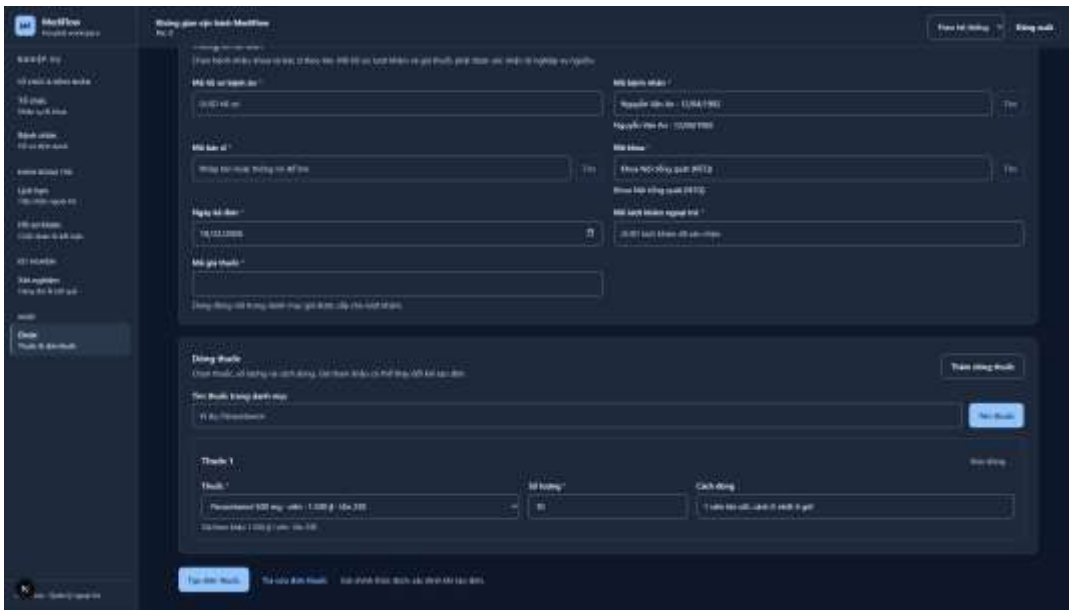


1.4.13. Doctor prepares the prescription context using the record, patient and department selectors.

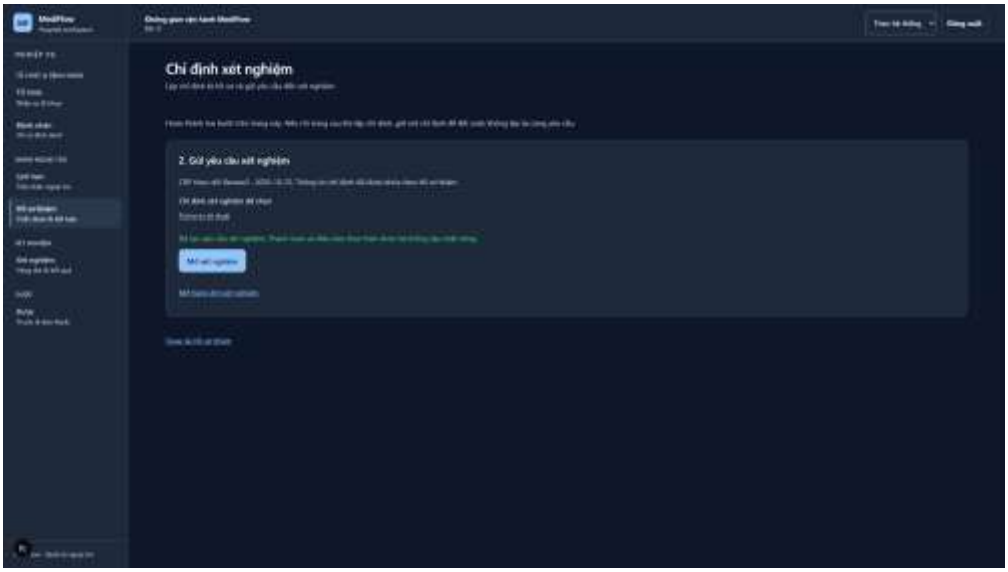


Draft form before submission; the prescriber still needs selection.

1.4.14. Doctor prepares a medicine line with quantity and instructions; draft has not been submitted.

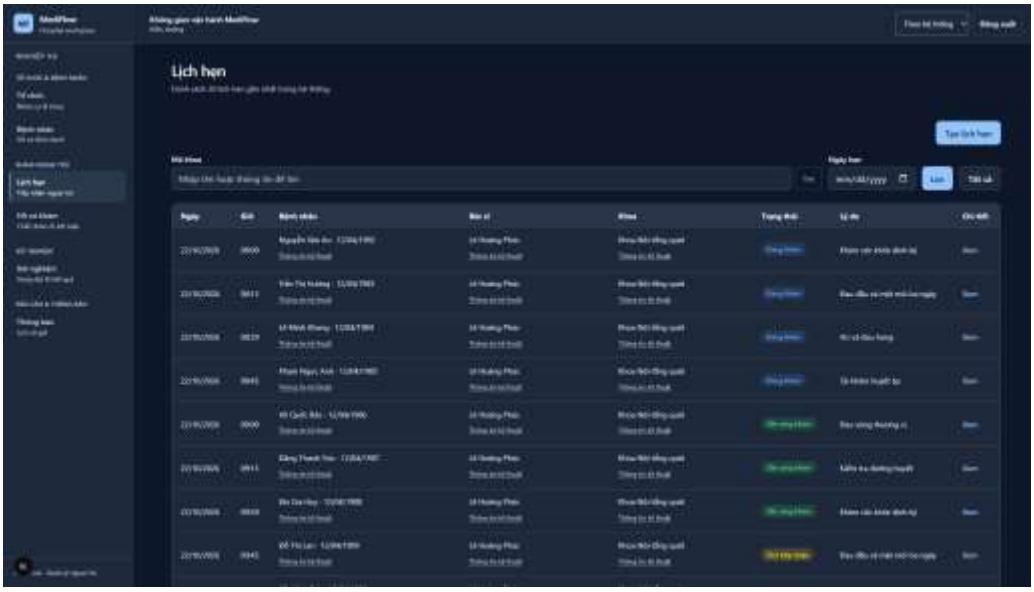


1.4.15. Successful second-step Lab request creation through the actual web form. Financial clearance remains a separate workflow.

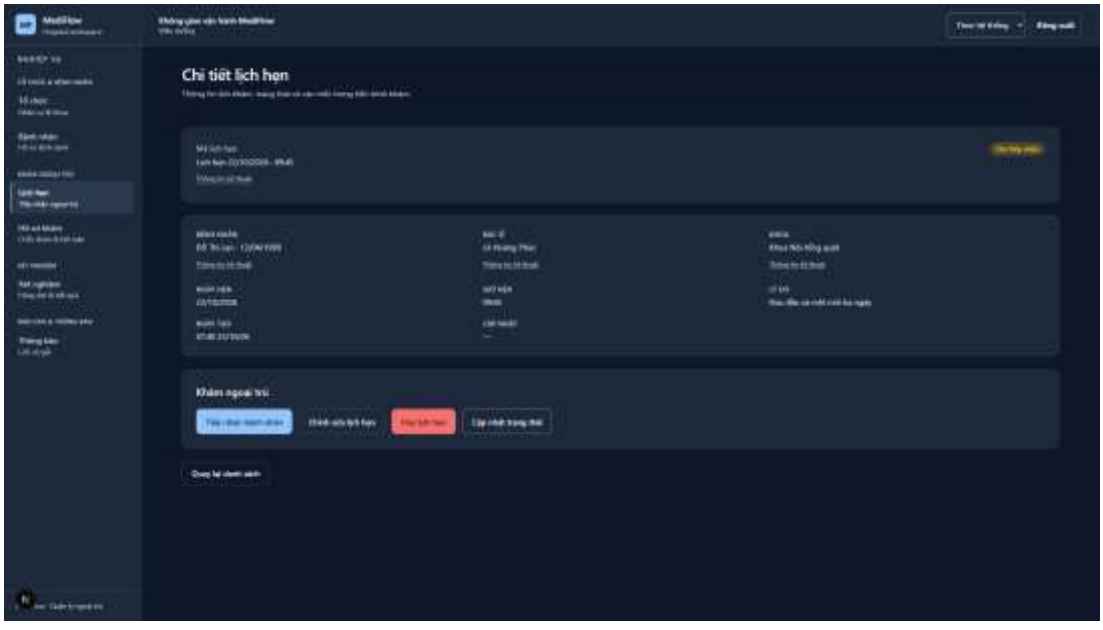


1.5. Nurse screens

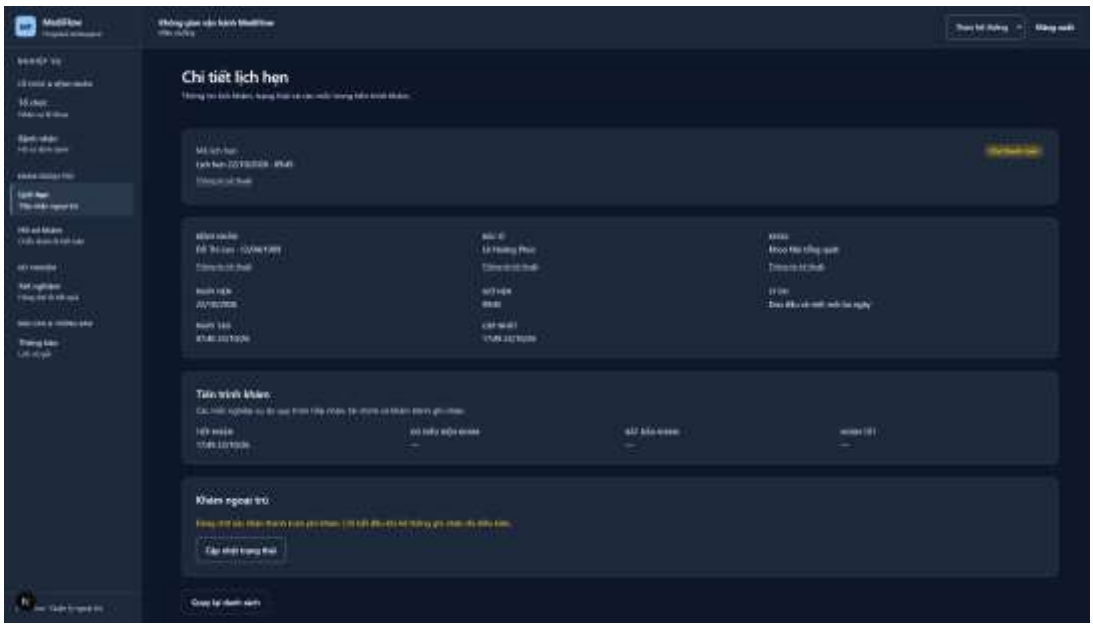
1.5.1. Nurse reception queue displays patient, doctor and department names.



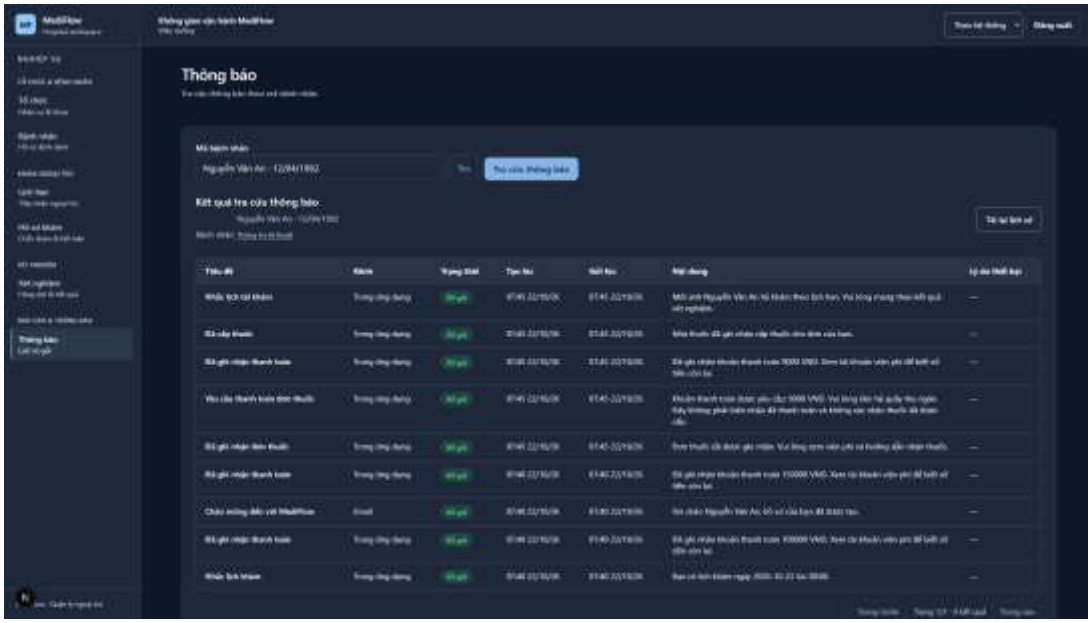
1.5.2. Reception detail before check-in: appointment is pending.



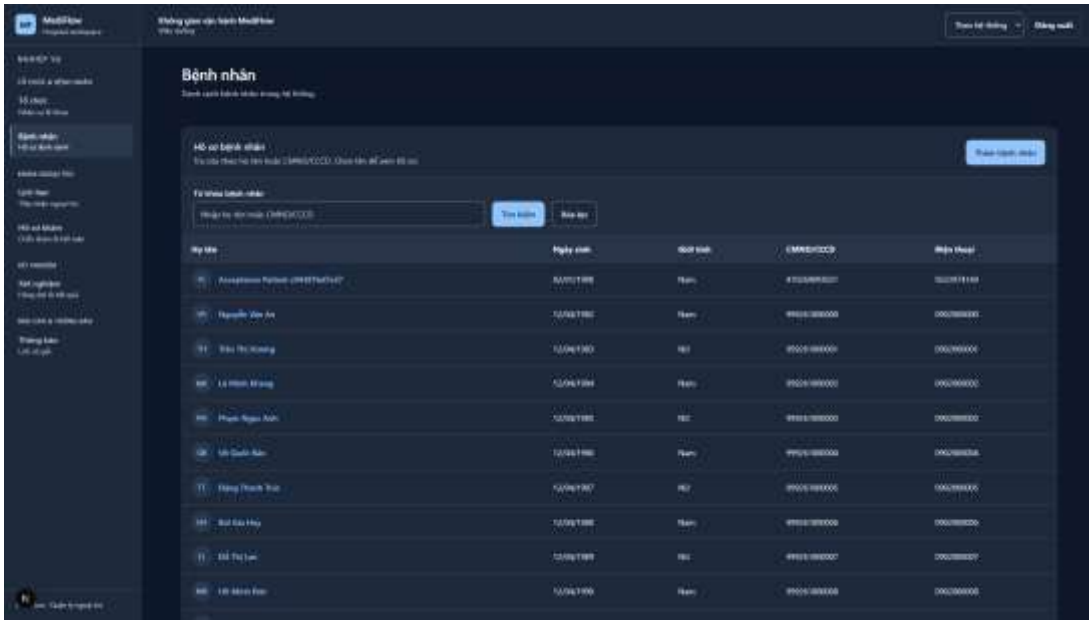
1.5.3. Successful check-in records the reception time and moves the appointment to payment pending; examination remains locked until clearance.



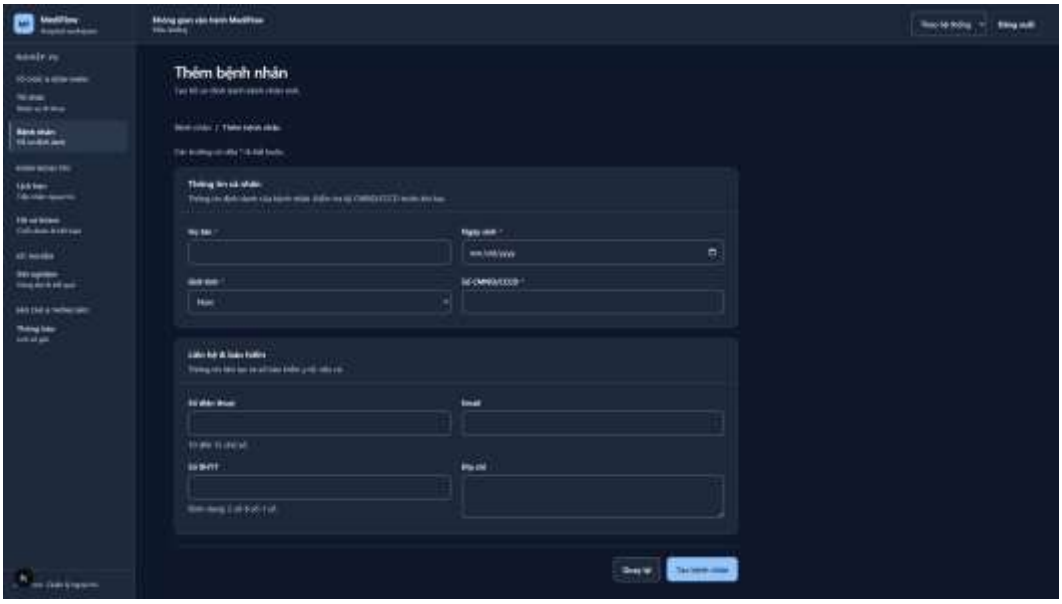
1.5.4. Nurse searches notification history for the selected patient by name.



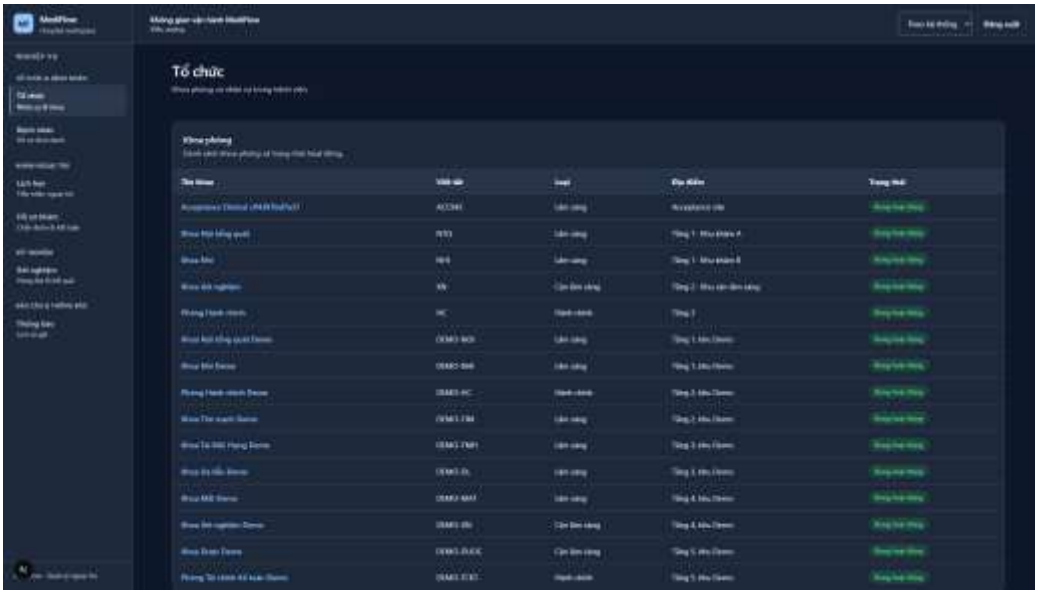
1.5.5. Nurse patient directory and registration entry point.



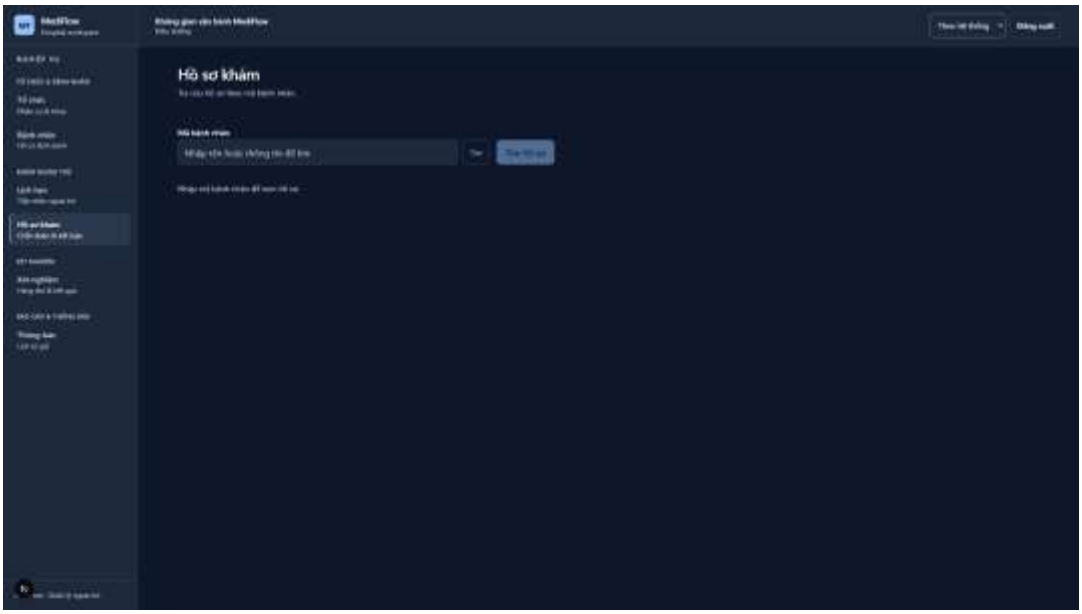
1.5.6. Patient registration form with identity, contact and insurance fields.



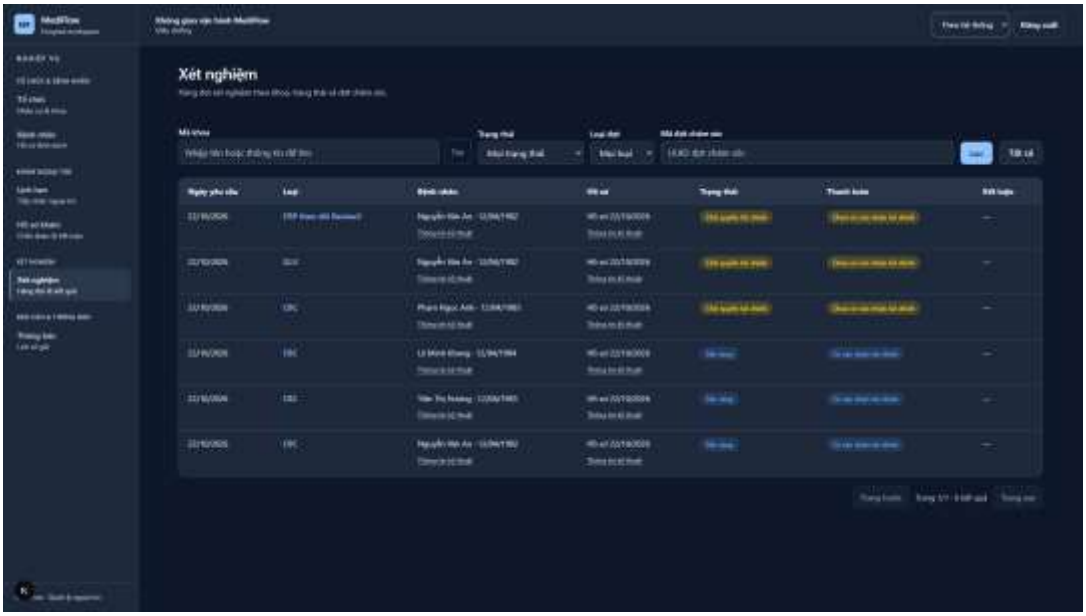
1.5.7. Nurse reads department and staff reference data.



1.5.8. Nurse can search clinical records using a patient name.

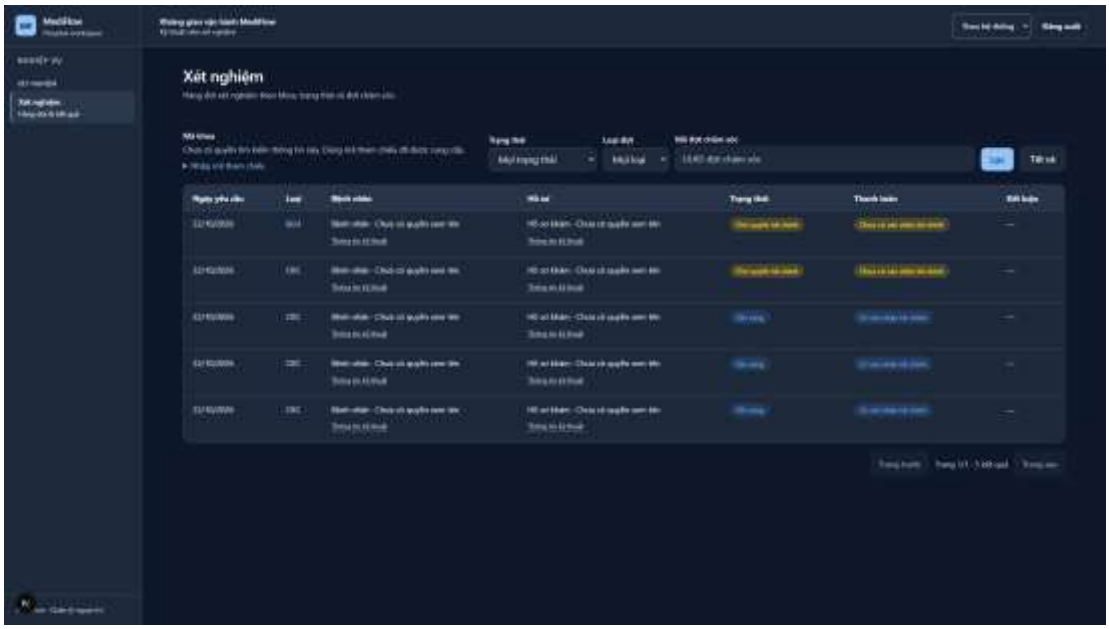


1.5.9. Nurse laboratory queue includes the newly created CRP request.

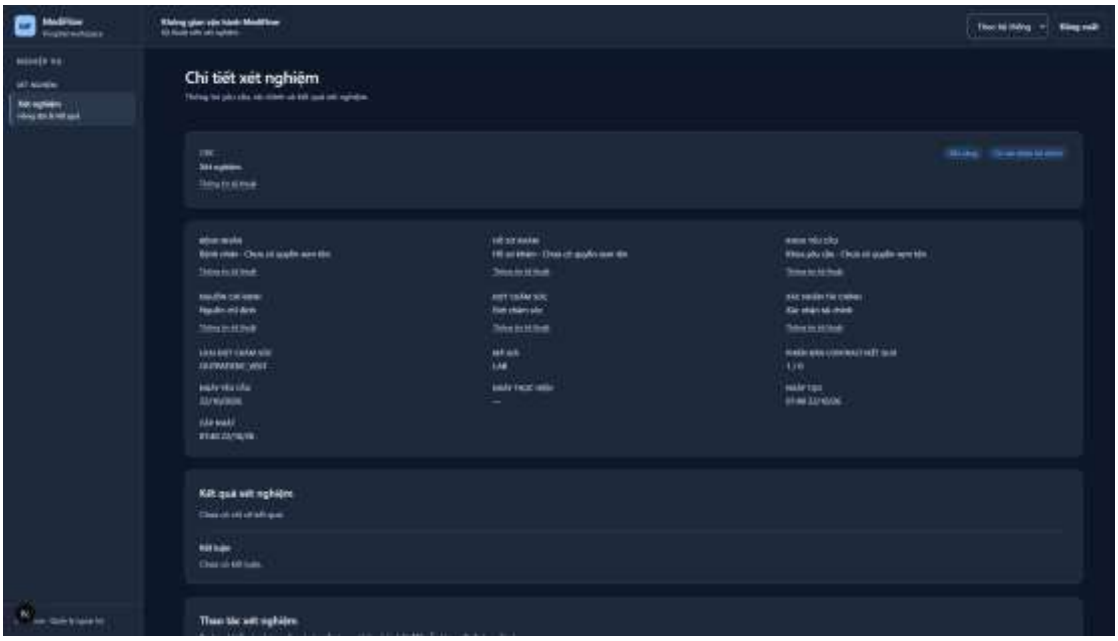


1.6. Laboratory technician screens

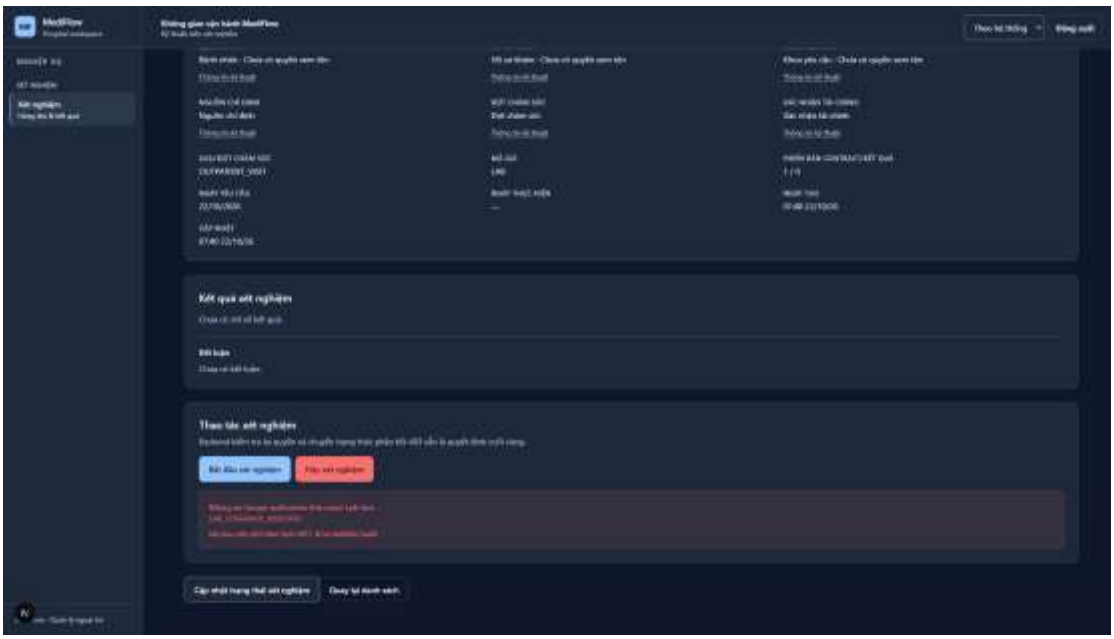
1.6.1. Lab technician queue shows financial status; names are explicitly unavailable under this role's current lookup permissions.



1.6.2. Lab request detail before processing, with financial confirmation and no result yet.

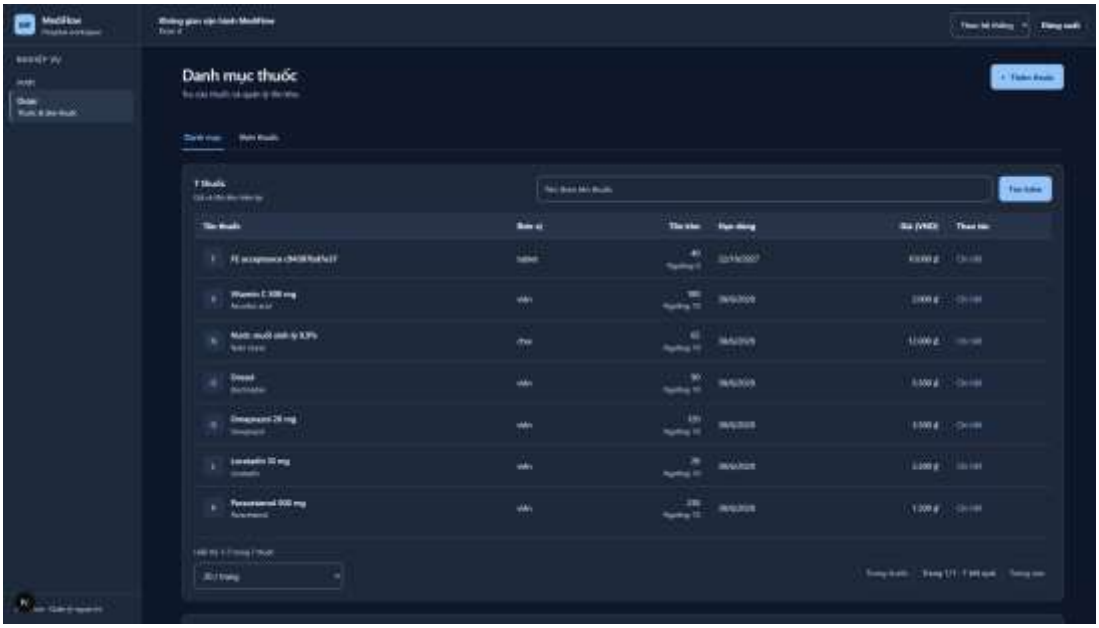


1.6.3. Backend rejects processing because Billing does not authorise this exact Lab request (LAB_CLEARANCE_REQUIRED). Result entry is not claimed as completed.

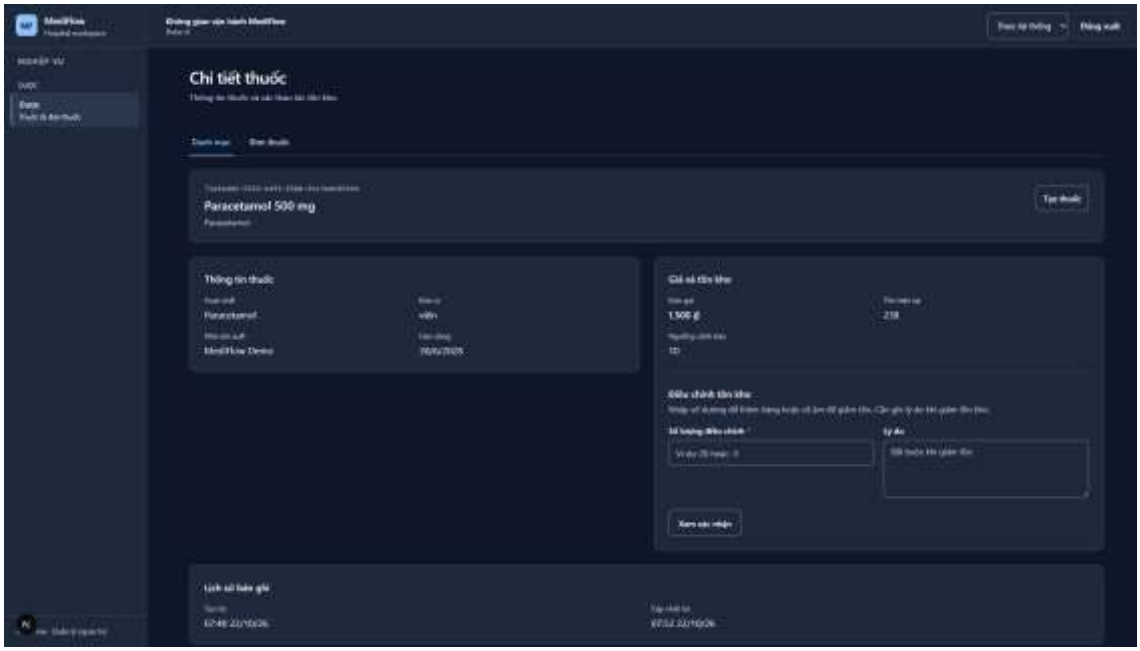


1.7. Pharmacist screens

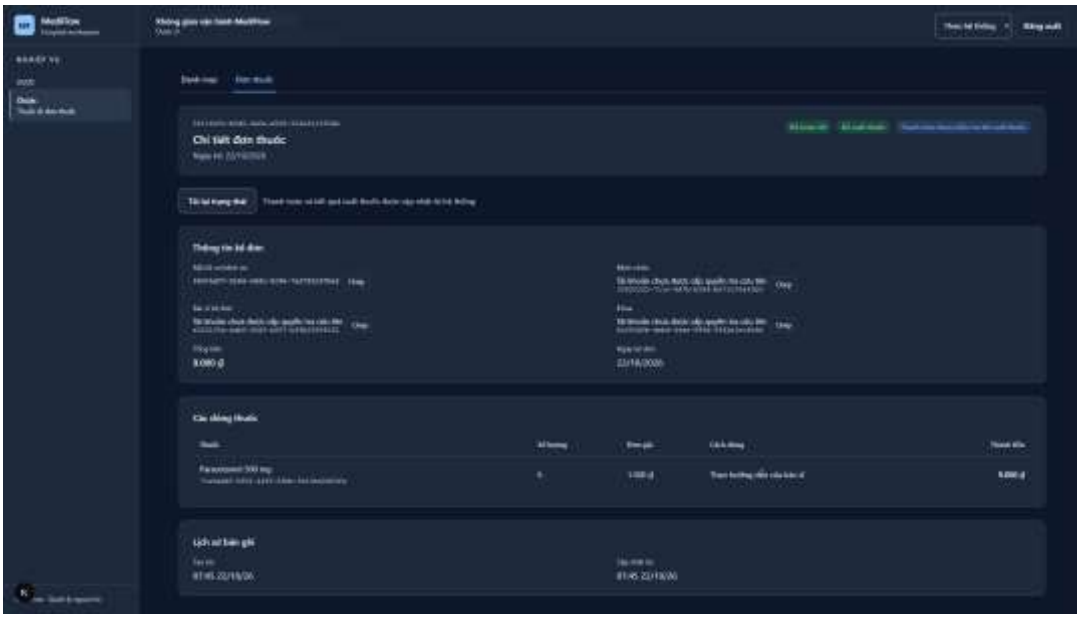
1.7.1. Pharmacist medicine catalogue with current stock and expiry dates.



1.7.2. Medicine detail and stock adjustment controls; no stock change is submitted.

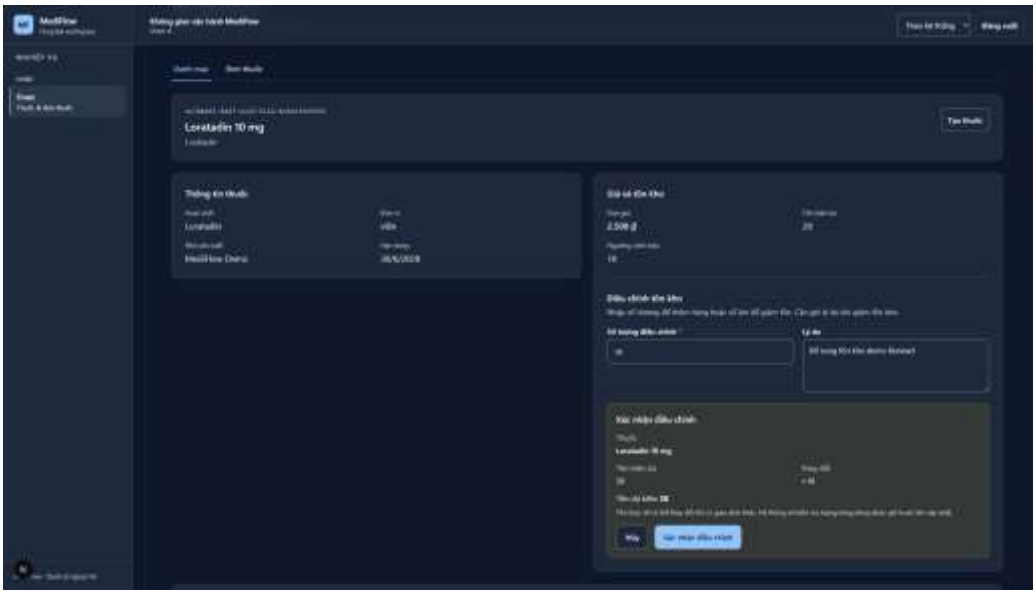


1.7.3. Existing seeded prescription is completed and dispensed. Web prescription detail still exposes technical references when role lookup is denied; tracked for developer handoff.

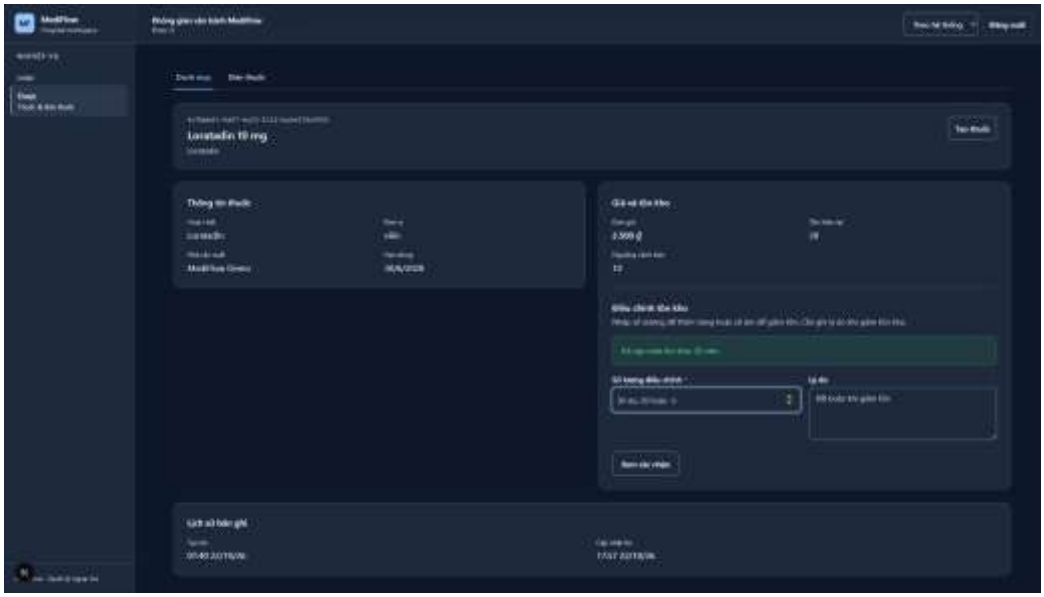


Known existing web identity display limitation; mobile hides these references by default.

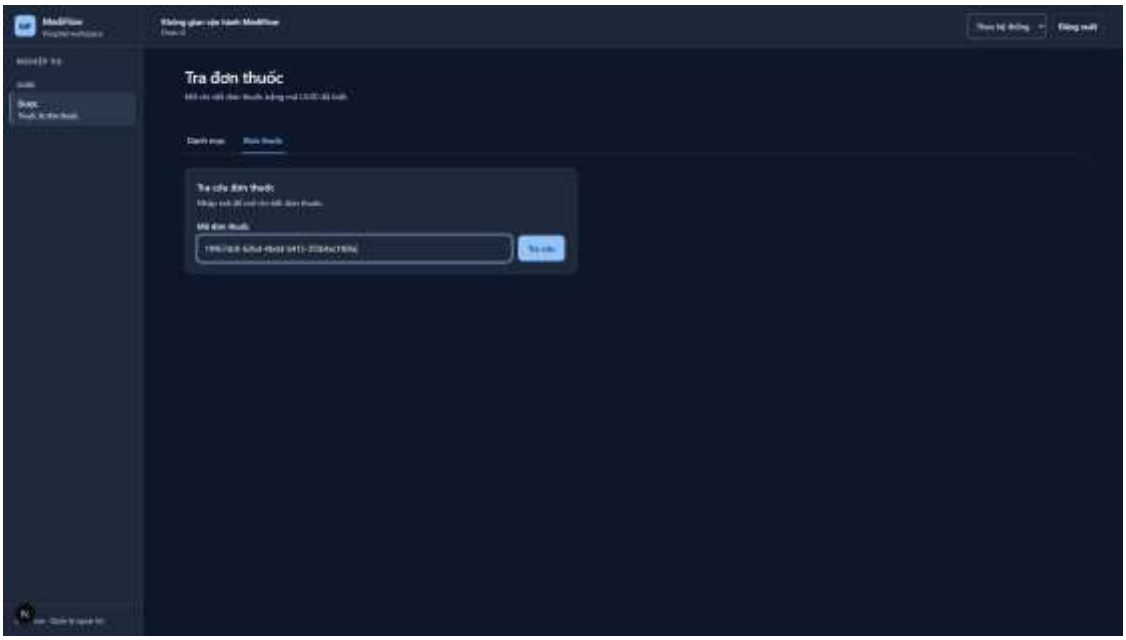
1.7.4. Stock adjustment confirmation adds ten demo units of Loratadin (28 to 38).



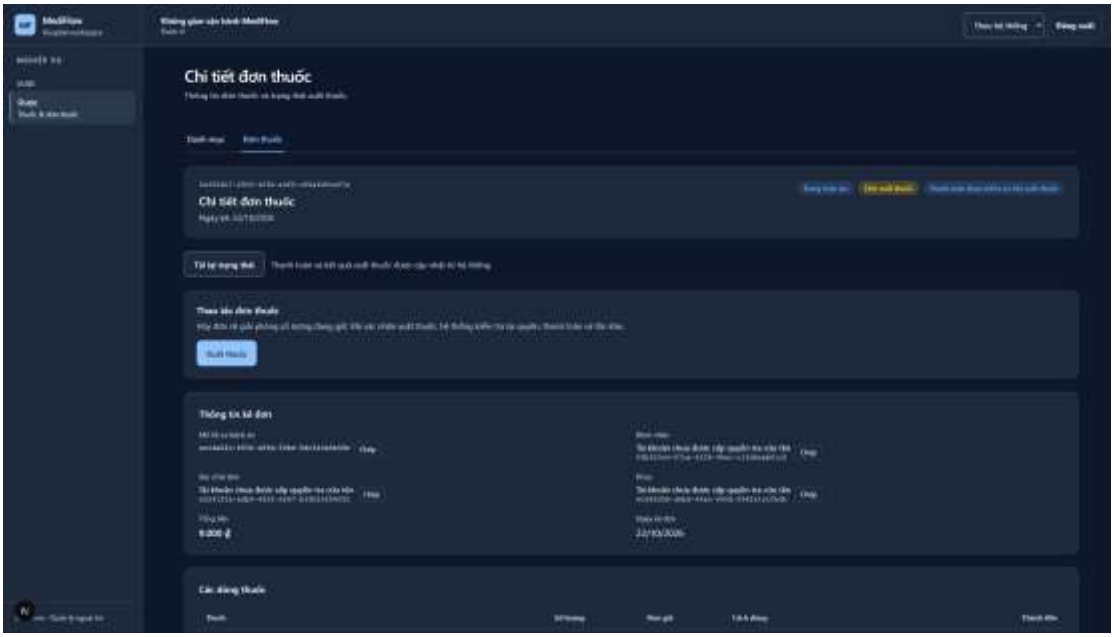
1.7.5. Loratadin stock is updated to 38 after confirmation.



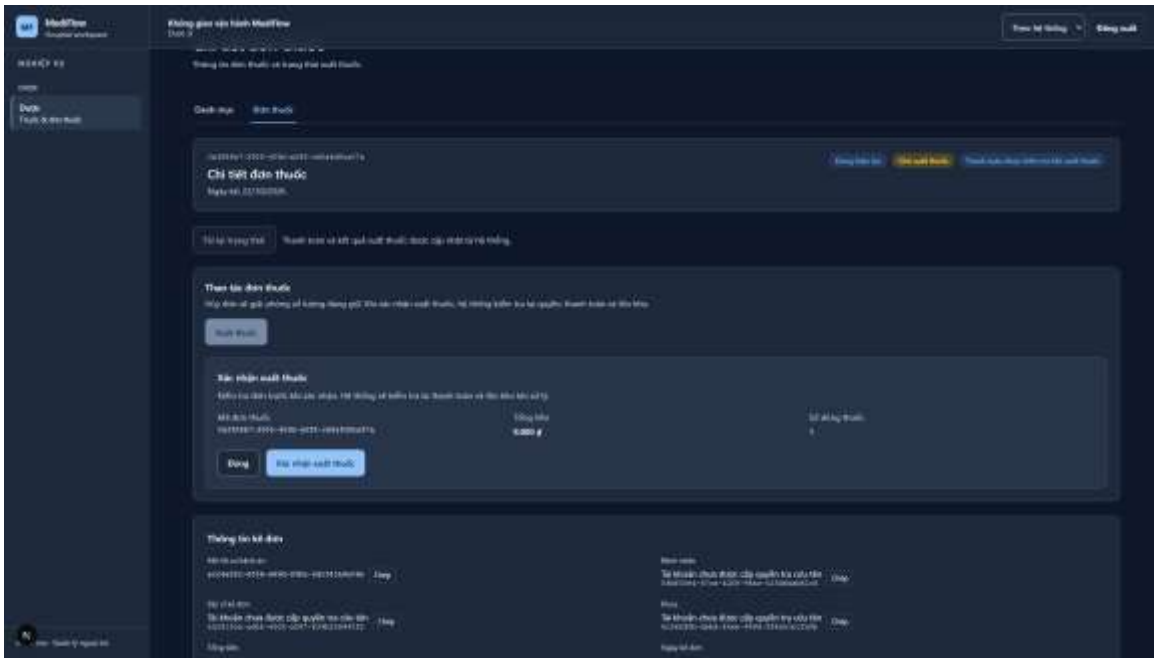
1.7.6. Pharmacist opens an existing prescription using its supplied reference.



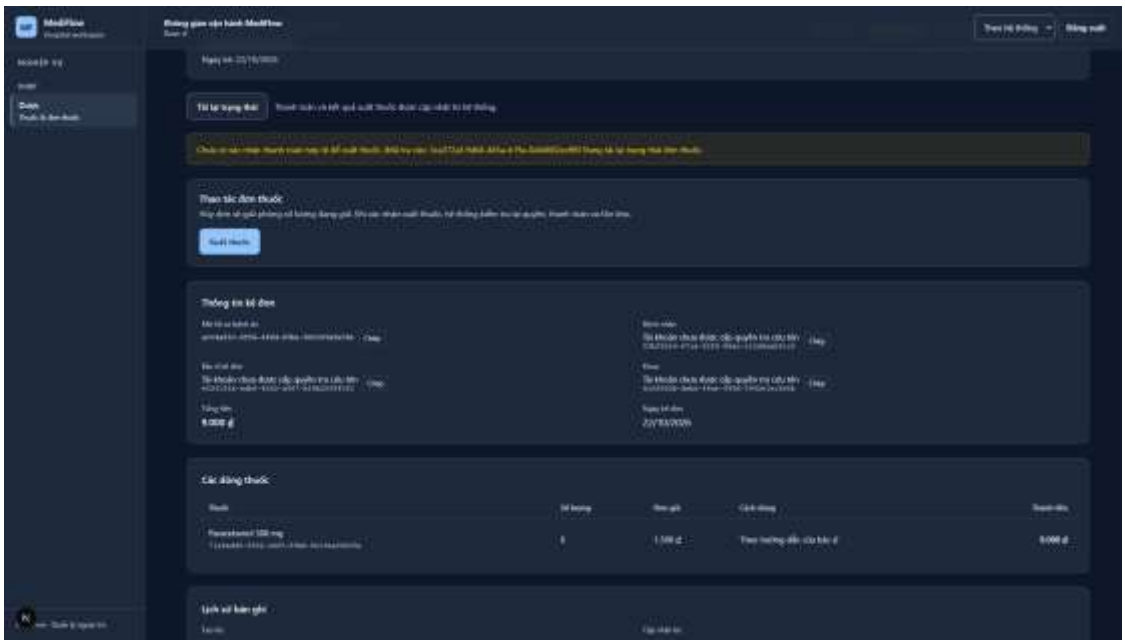
1.7.7. Active prescription awaiting dispense; payment will be revalidated by the backend.



1.7.8. Pharmacist reviews the dispense confirmation before backend revalidation.

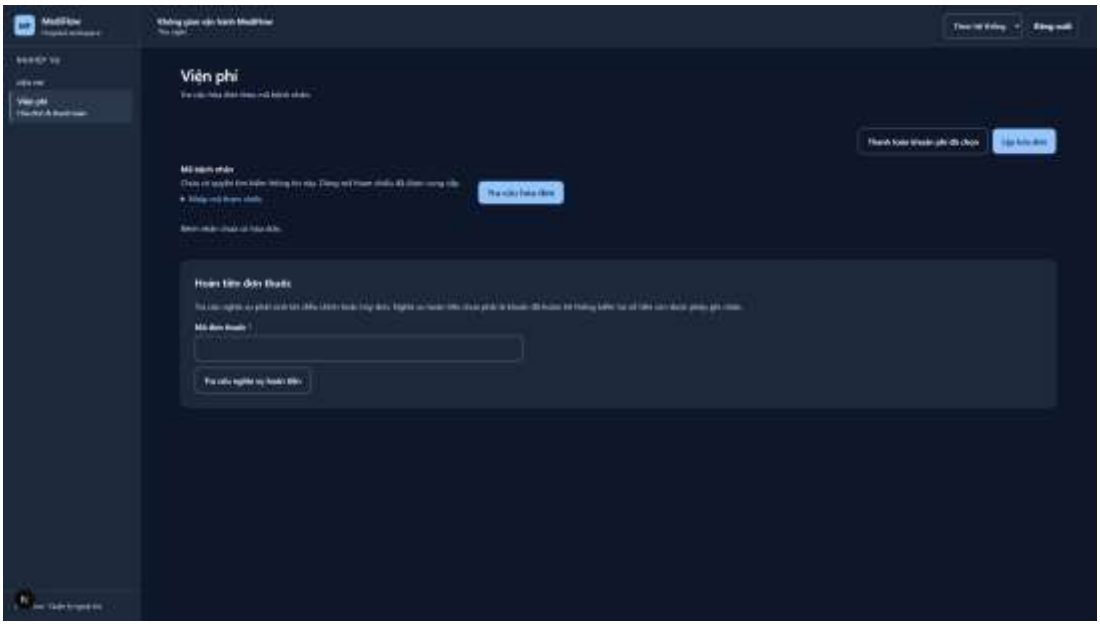


1.7.9. Unpaid demo prescription is correctly blocked from dispensing; an already completed prescription is documented separately.

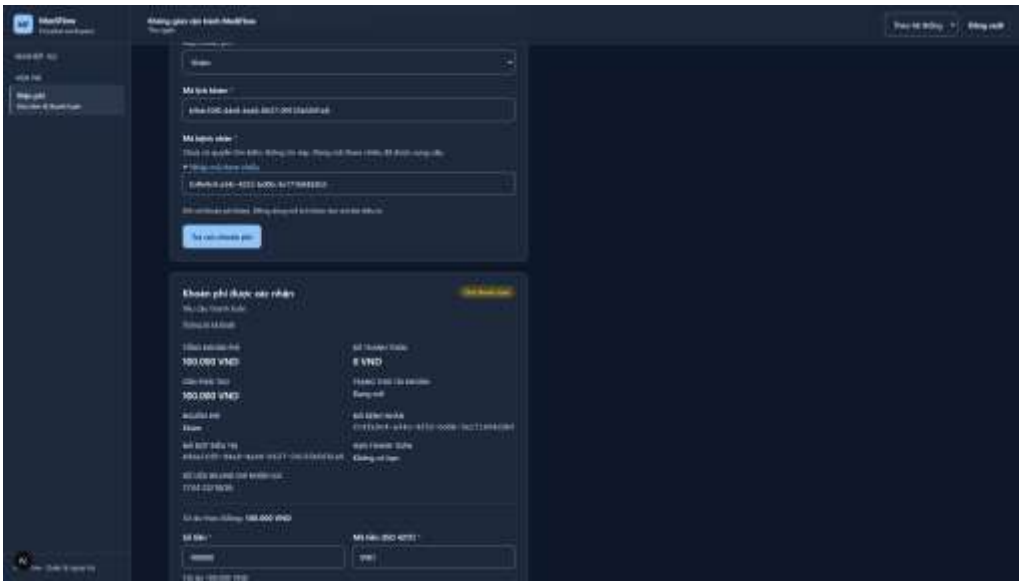


1.8. Cashier screens

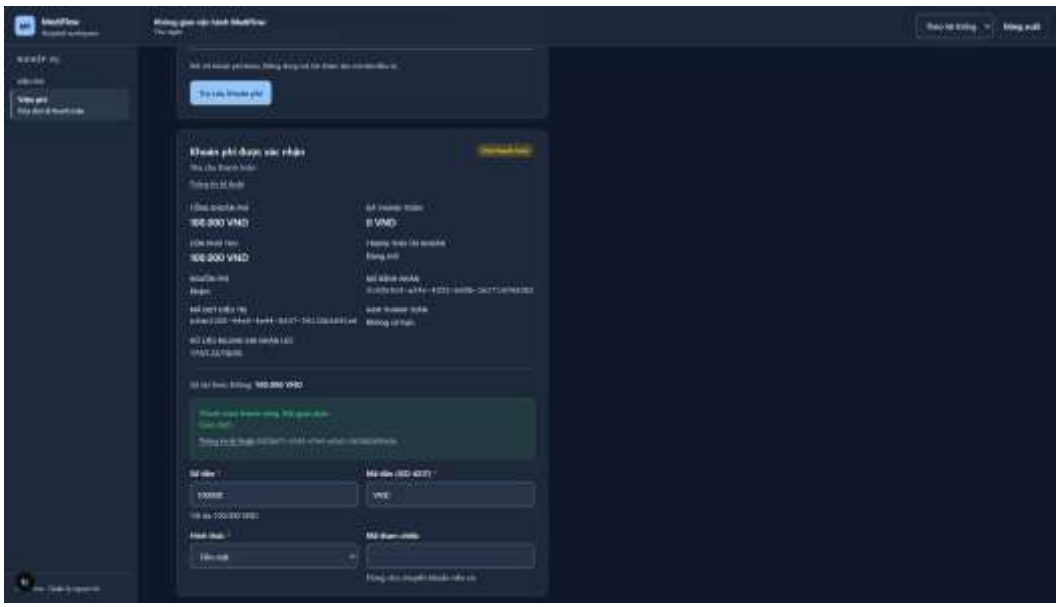
1.8.1. Invoice lookup returns no legacy invoice for the selected patient; outpatient charges use the payment-request flow.



1.8.2. Billing returns a 100,000 VND examination charge for the just checked-in demo visit.

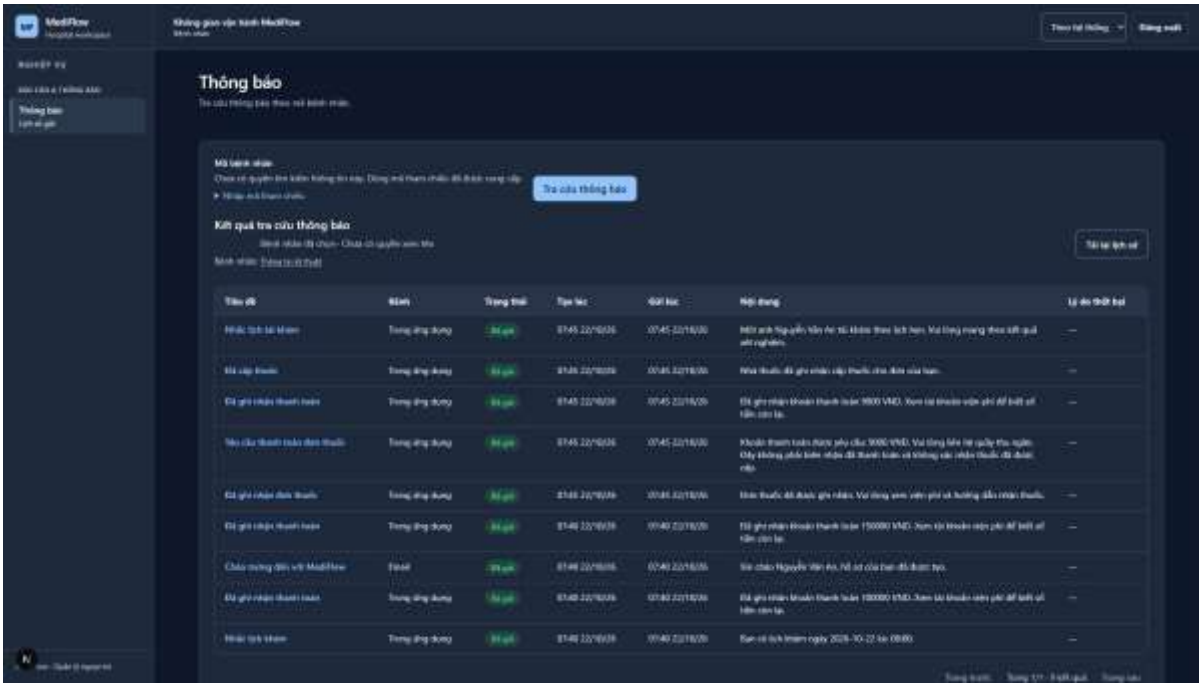


1.8.3. Demo cash payment of 100,000 VND succeeds and returns a transaction receipt. Charge summary shown above is the pre-payment lookup snapshot.

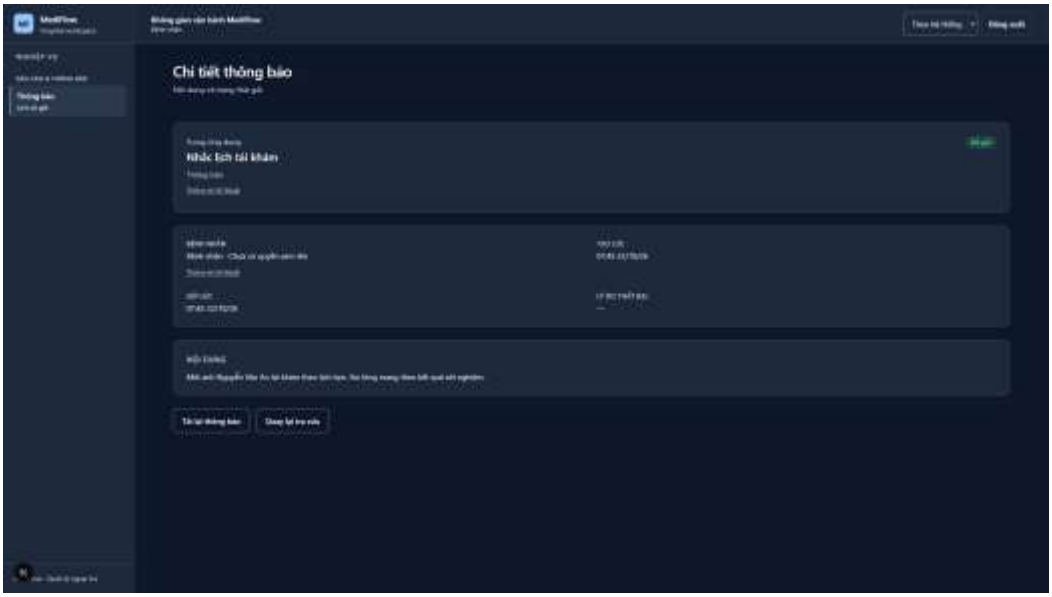


1.9. Patient screens

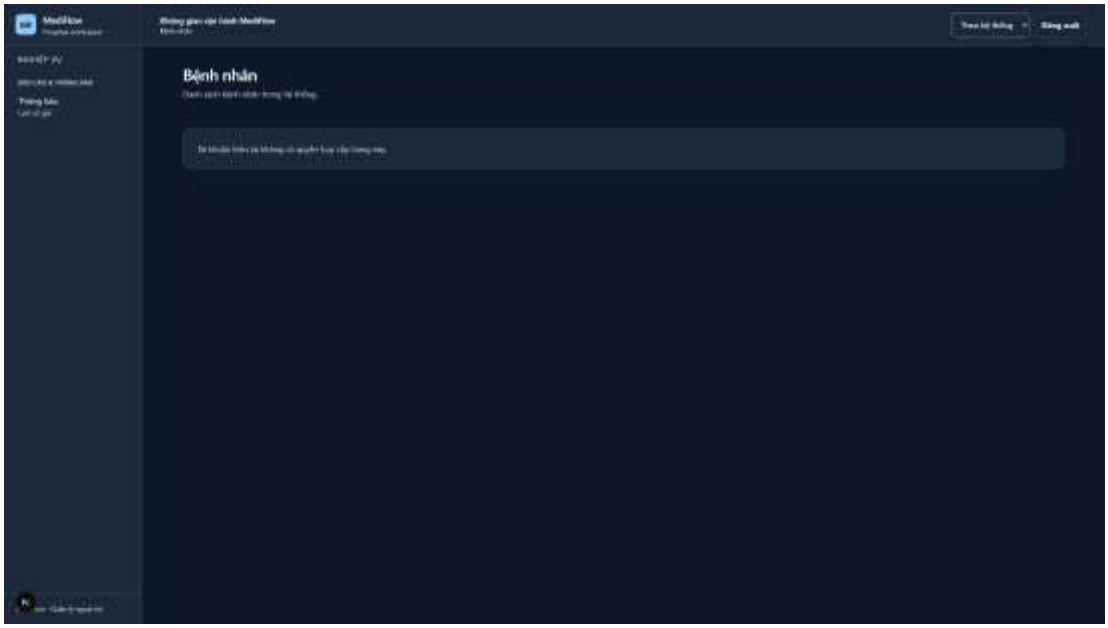
1.9.1. Patient views their own seeded notification history.



1.9.2. Patient reads a follow-up reminder and its sent status.



1.9.3. Patient cannot access the staff patient registry.



2. Mobile GUI

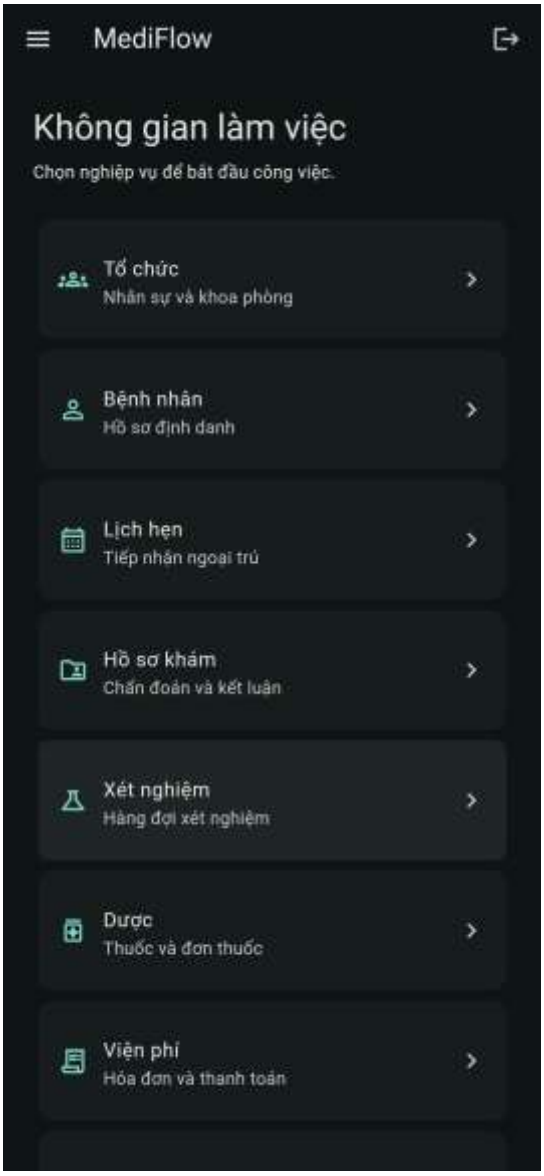
2.1. Shared screens

2.1.1. Mobile login screen at 430 × 932 CSS pixels, rendered at 3x density.

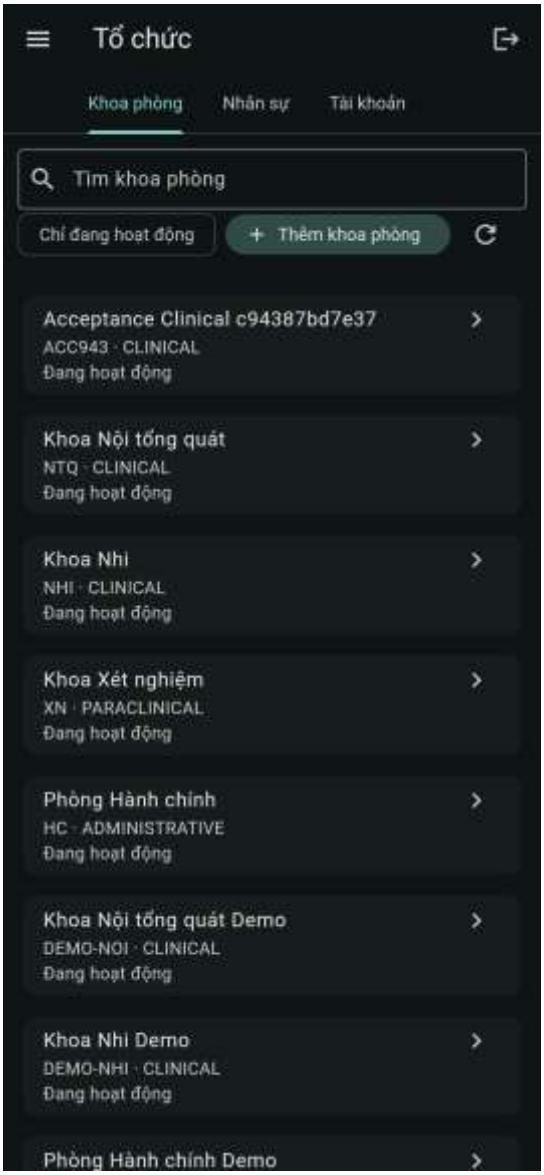


2.2. Administrator

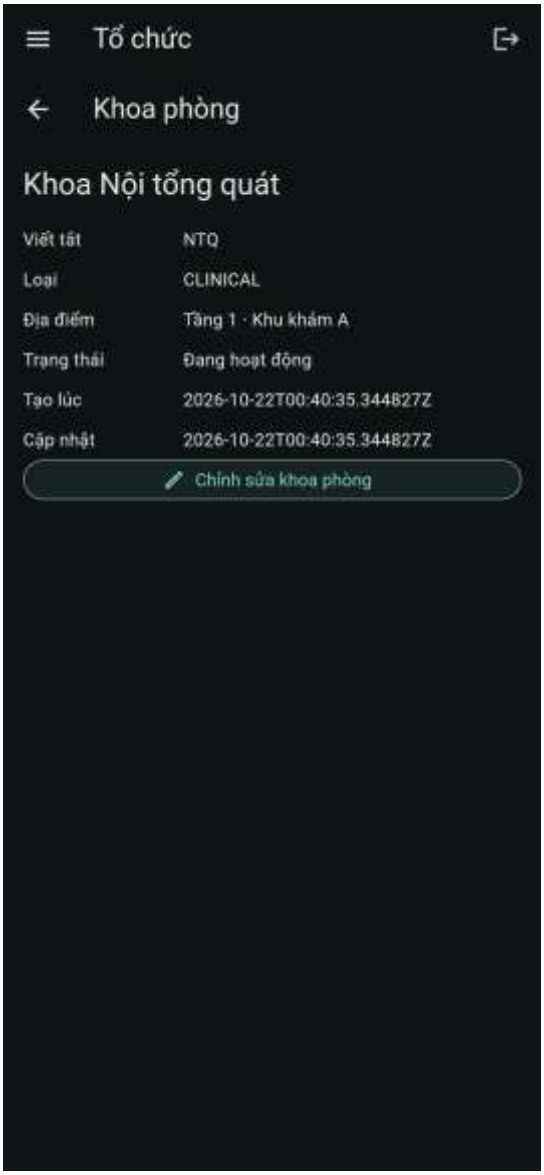
2.2.1. Administrator workspace exposes all active outpatient contexts.



2.2.2. Department directory displays department names without UUIDs.



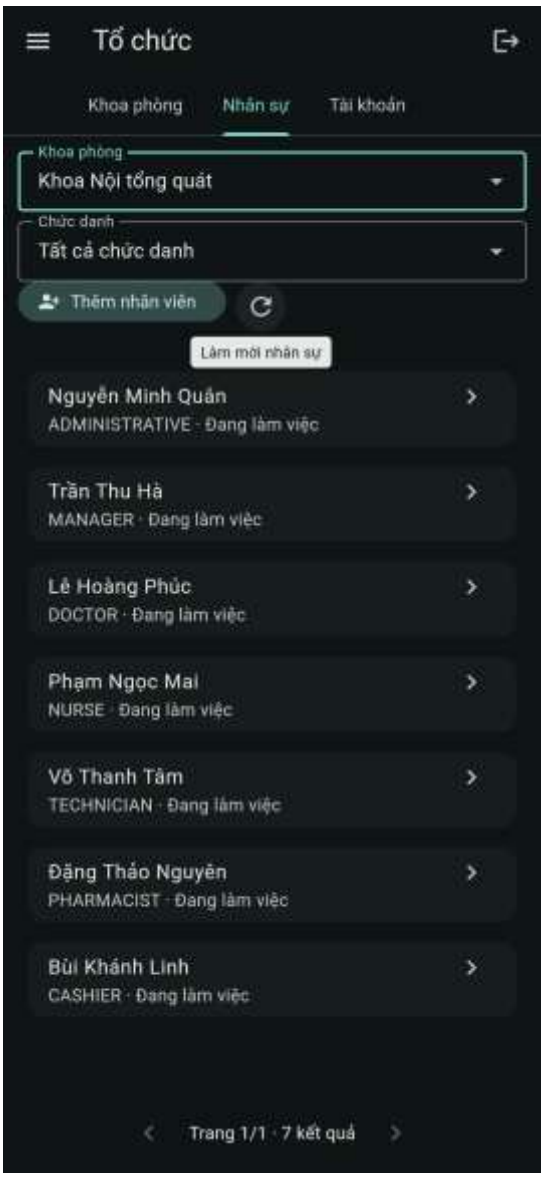
2.2.3. Department detail and edit entry point.



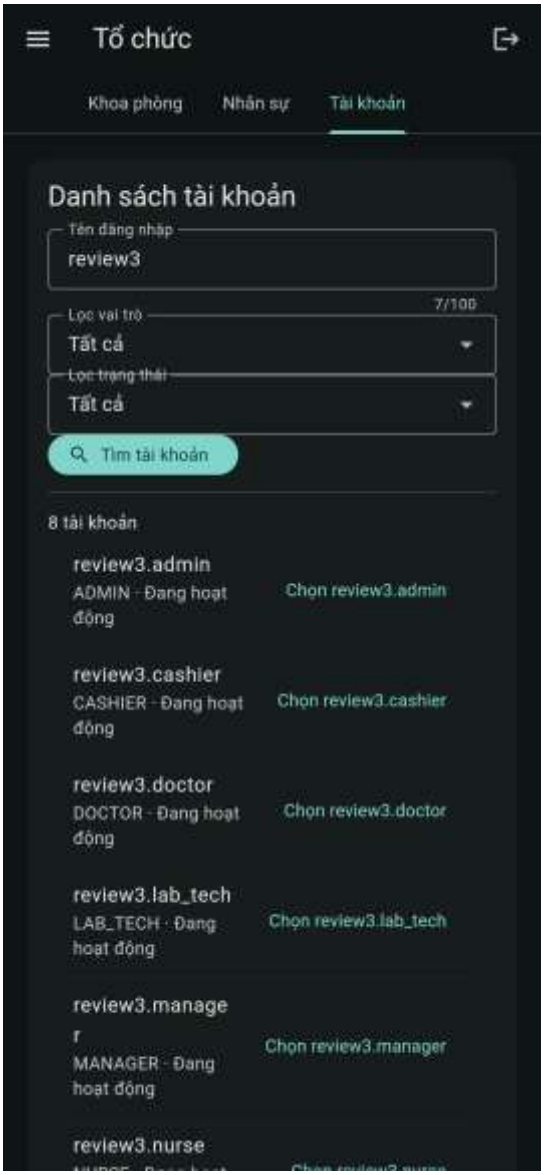
2.2.4. Department head picker lists staff names and job titles from the authorised Gateway endpoint.



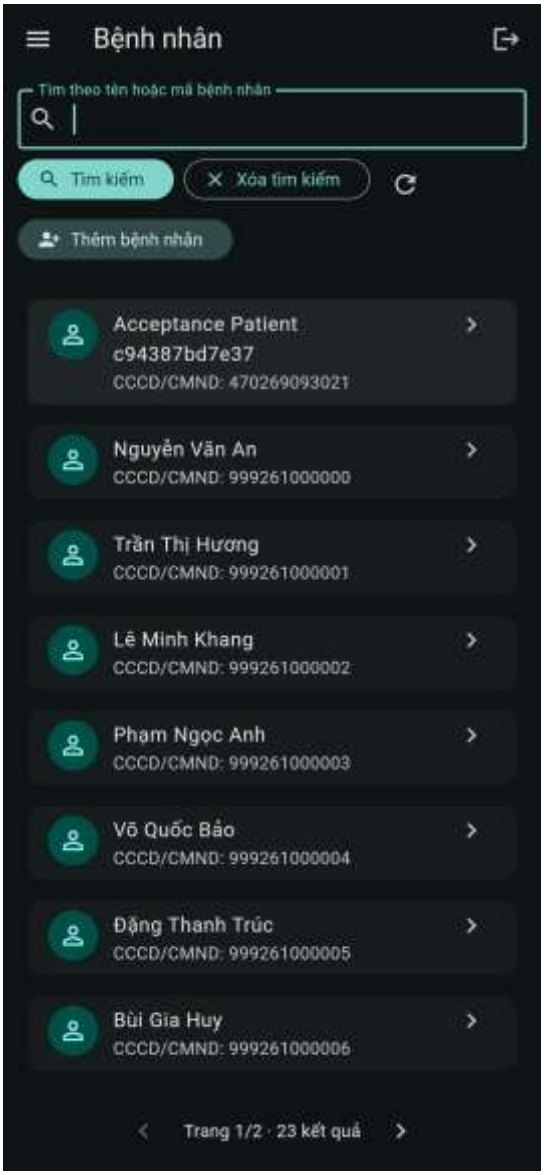
2.2.5. Staff directory filtered to Internal Medicine shows names and job titles without UUIDs.



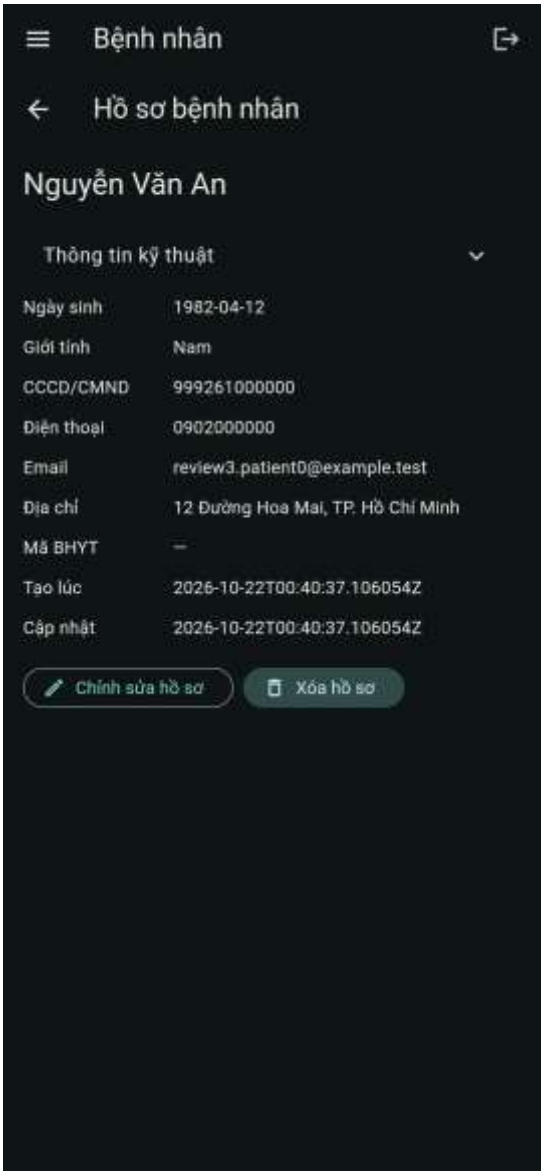
2.2.6. Administrator searches the live account directory with role and status filters.



2.2.7. Patient list shows names and synthetic identity references without UUIDs.



2.2.8. Patient detail presents the name prominently and keeps technical ID collapsed.



2.2.9. Mobile appointment queue resolves patient, doctor and department names through the Gateway.



2.2.10. Appointment form uses name-based patient, department and doctor selection instead of manual UUID fields.



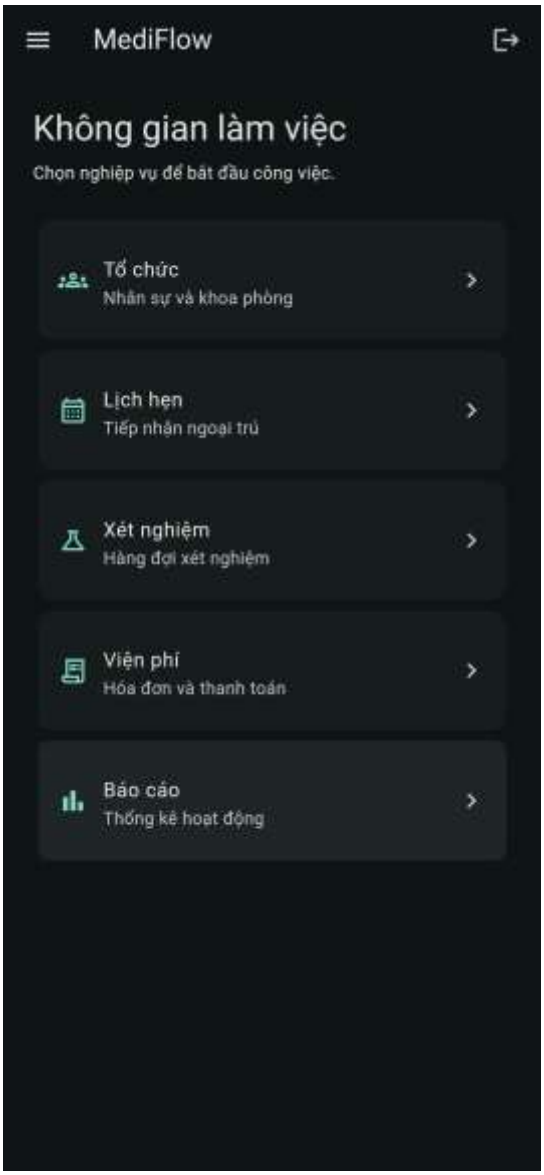
2.2.11. Selecting the doctor account populates status management by account identity.



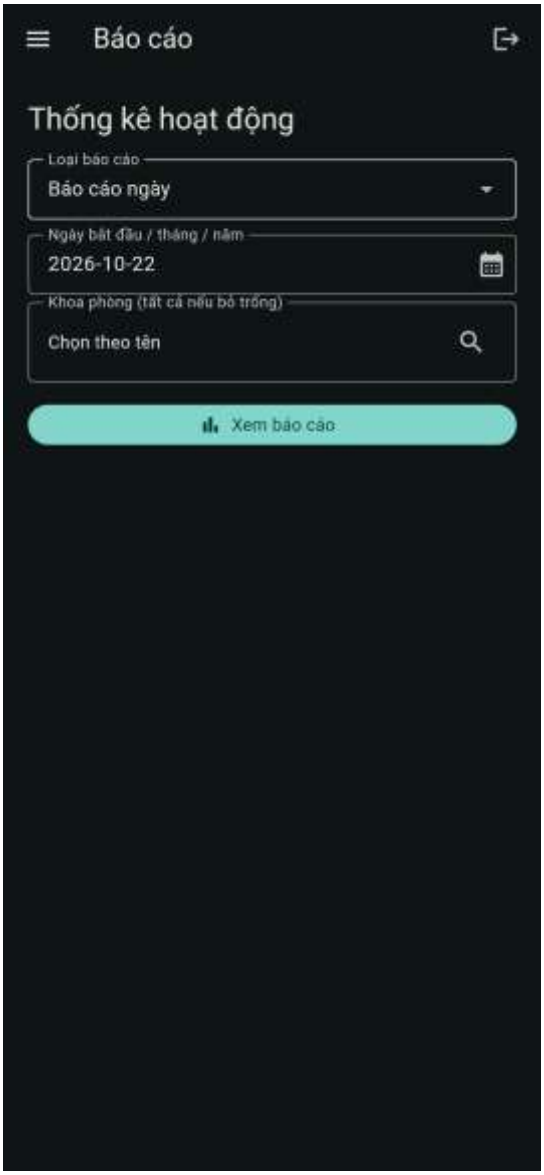
No account status change is submitted.

2.3. Manager screens

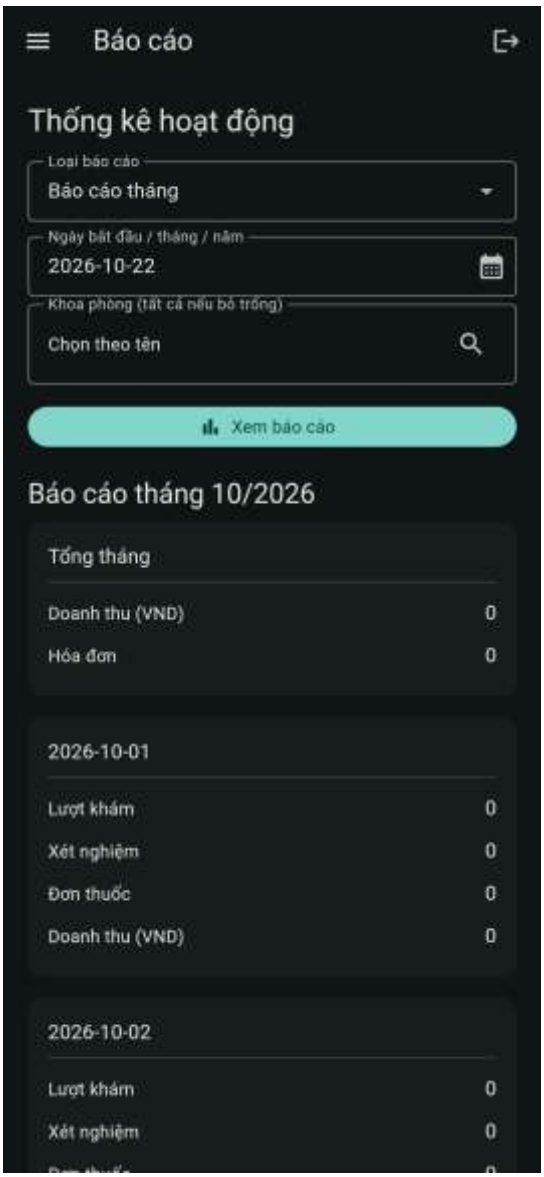
2.3.1. Manager workspace includes organization, appointment, laboratory and reporting views.



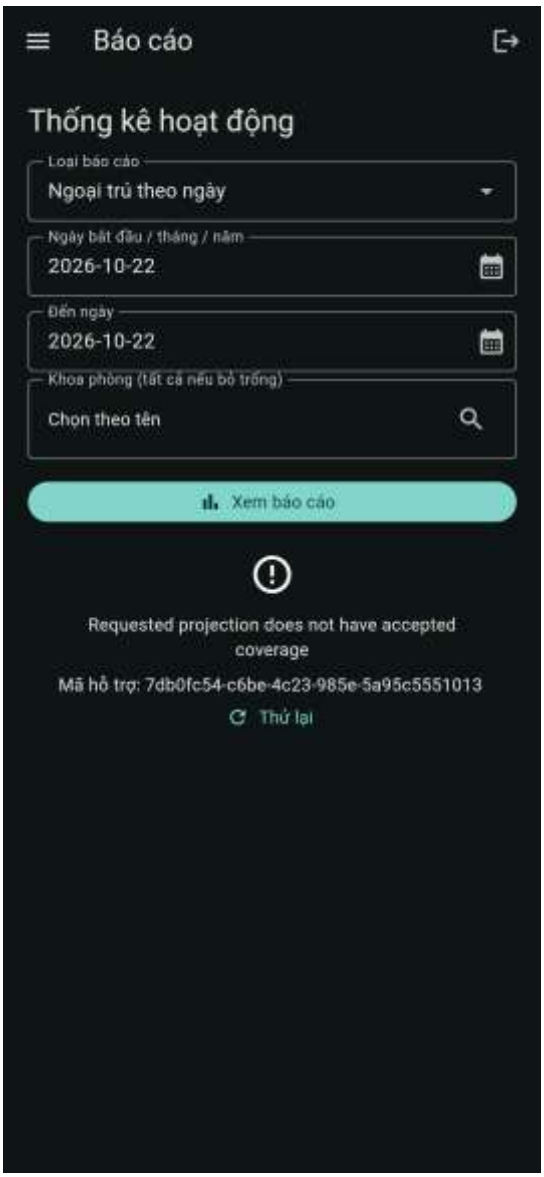
2.3.2. Manager selects the report query and reporting period.



2.3.3. Monthly Report aggregate loads through the real Gateway; zero projection values are shown as returned.



2.3.4. Outpatient report preserves the backend missing-coverage response instead of inventing reconciled figures.

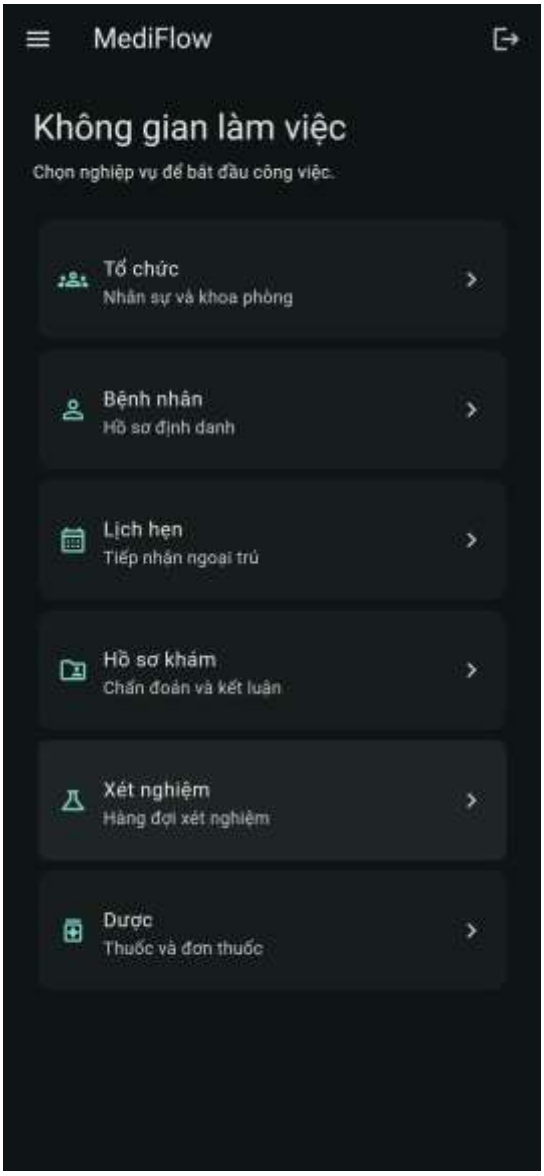


2.3.5. Manager appointment monitoring hides unavailable patient identity and displays authorised doctor/department names.



2.4. Doctor screens

2.4.1. Doctor workspace exposes clinical and pharmacy functions authorised to the role.



2.4.2. Doctor queue shows real patient, doctor and department names.



2.4.3. Mobile visit detail shows the in-progress state with resolved identity labels.



2.4.4. Name-based clinical record search returns the current outpatient record.



2.4.5. Clinical detail shows named patient, doctor, department, appointment, symptoms and ICD diagnosis.



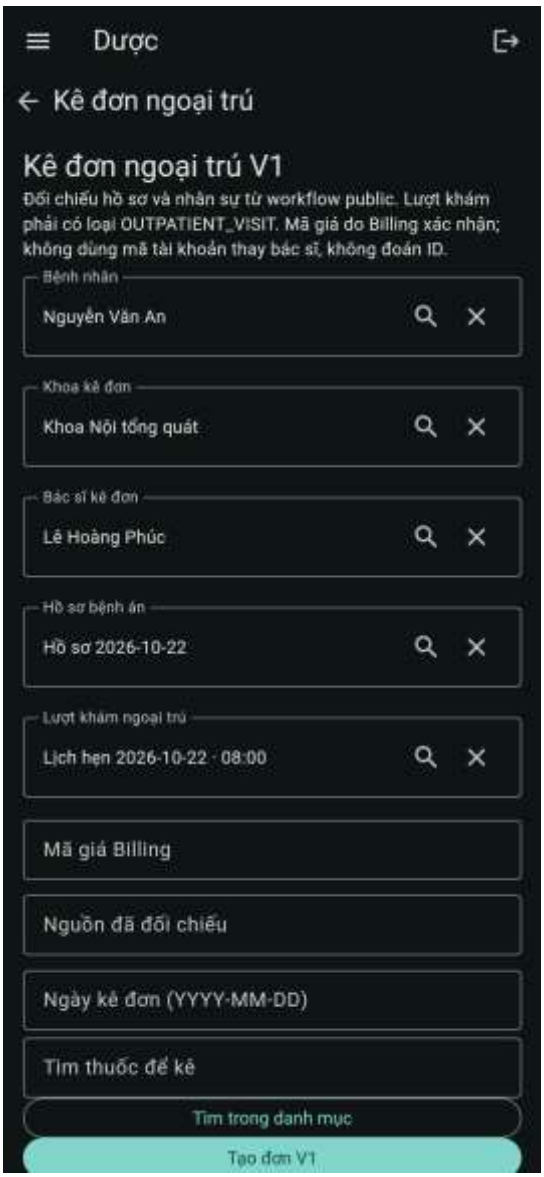
2.4.6. Doctor diagnosis dialog accepts diagnosis, ICD code and description.



2.4.7. Doctor medicine catalogue shows prices, stock and expiry dates.



2.4.8. Prescription draft links named patient, department, doctor, record and visit through scoped selectors; technical UUIDs remain hidden.

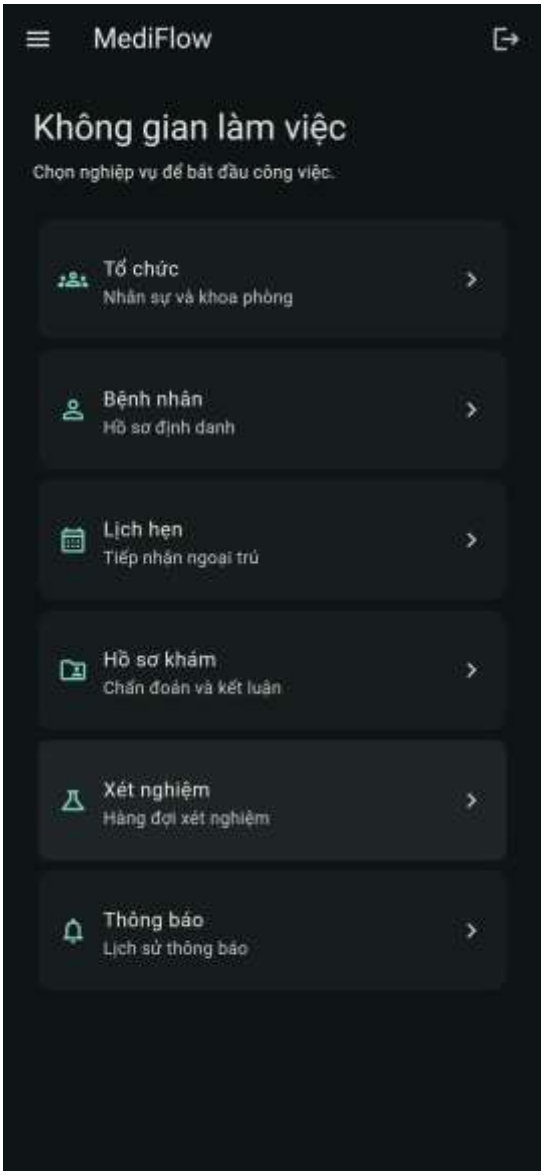


2.4.9. Doctor prepares a Paracetamol line with quantity and instructions; the draft is not submitted without an authorised price context.



2.5. Nurse screens

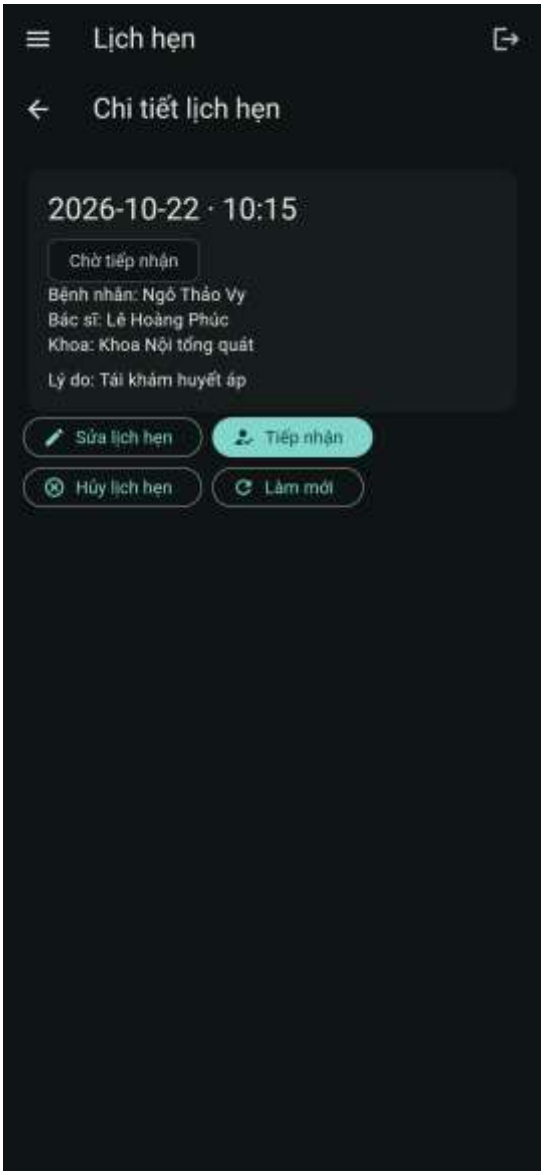
2.5.1. Nurse workspace includes reception, patient registration and notification history.



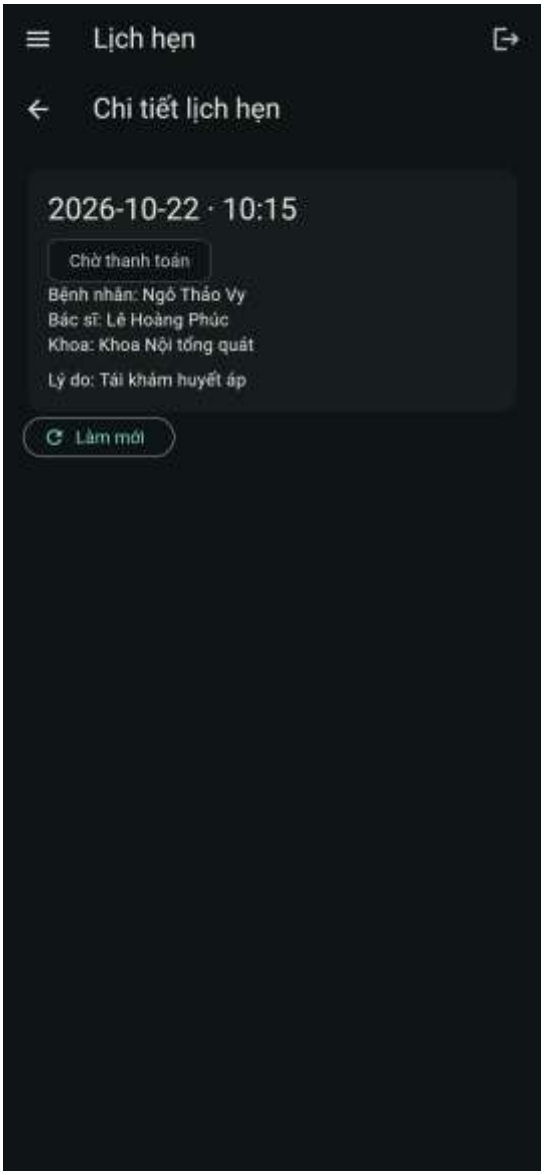
2.5.2. Nurse appointment queue provides reception actions.



2.5.3. Nurse views the named patient appointment before reception.



2.5.4. Reception succeeds and the appointment moves to payment pending.

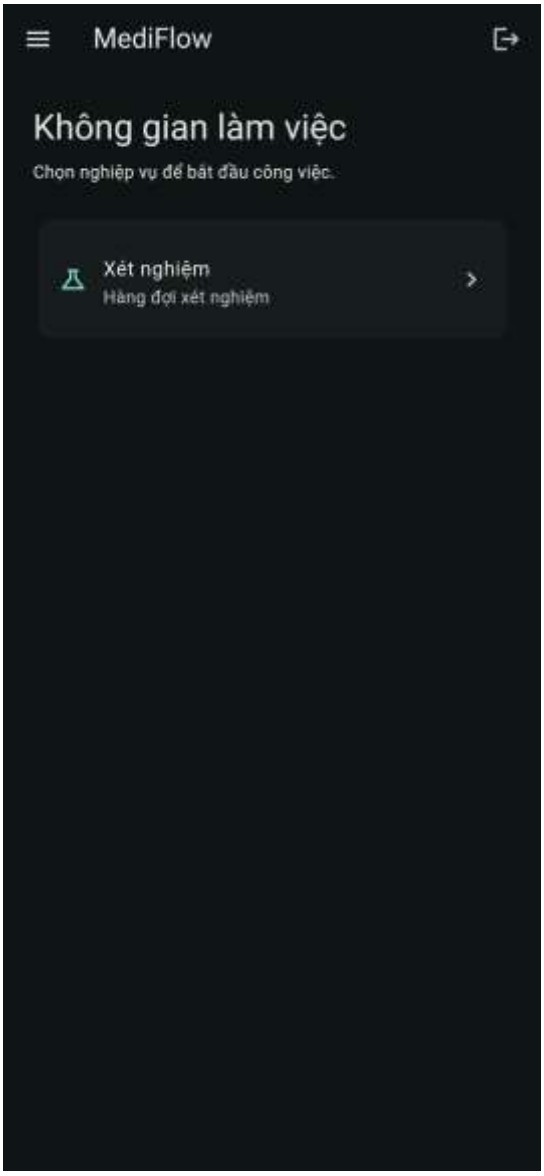


2.5.5. Nurse searches notification history using the patient name.



2.6. Laboratory Technician screens

2.6.1. Laboratory technician workspace shows the permitted Lab workflow.



2.6.2. Lab queue contains real requests and their billing states.



Name lookup is not authorized for this role; the UI states the limitation without exposing UUIDs.

2.6.3. Lab detail shows reconciliation and available workflow actions with technical identifiers hidden.



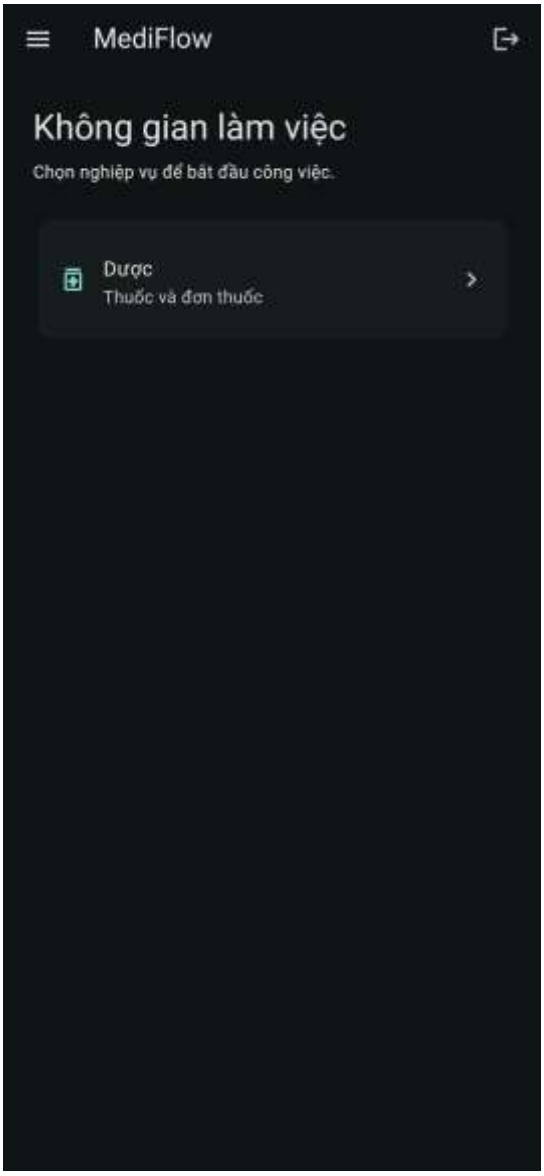
2.6.4. Starting the test is rejected when Billing no longer authorizes the exact request.



Observed backend limitation; this is evidence of the guard, not a successful test start.

2.7. Pharmacist screens

2.7.1. Pharmacist workspace provides drug and prescription access.



2.7.2. Pharmacist searches the live catalog by drug name and sees current stock.



2.7.3. Drug detail shows price, stock, expiry and manufacturer; technical identifiers are collapsed.

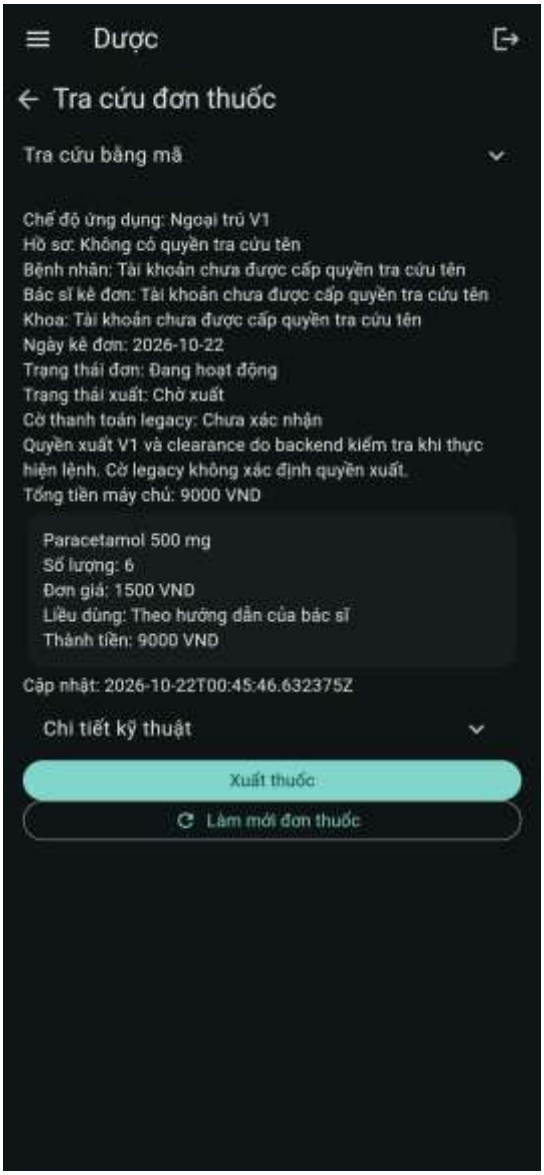


2.7.4. Stock adjustment uses the drug name and current server stock, with technical identifiers collapsed.



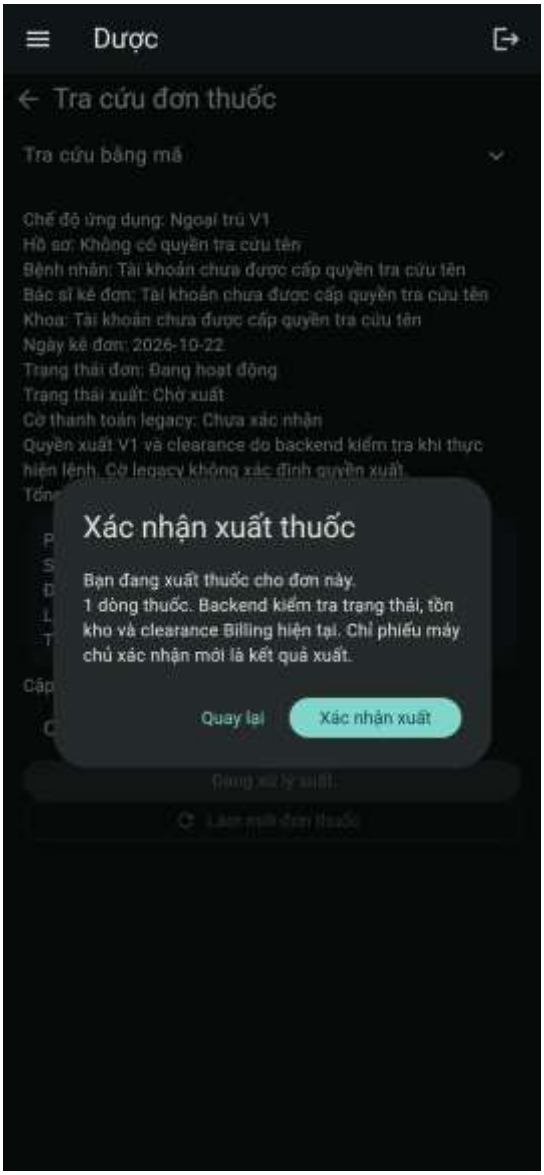
Form shown before submission.

2.7.5. Prescription detail presents the drug, dose, quantity, price and workflow state.

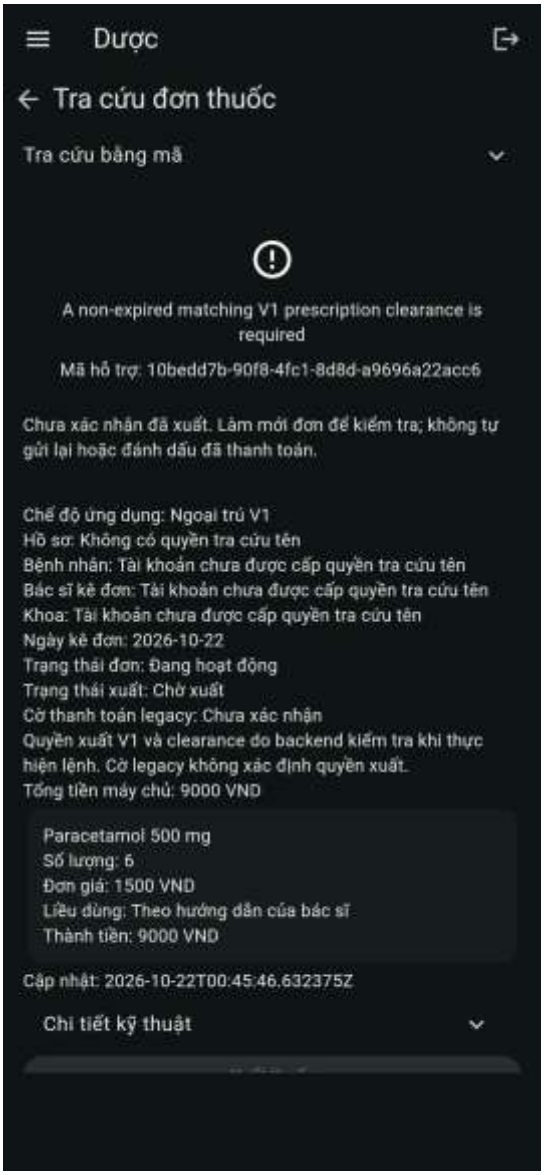


Reference-name lookups are restricted for this role; UUIDs remain in collapsed technical controls.

2.7.6. Dispense confirmation explains server-side stock and Billing clearance checks.



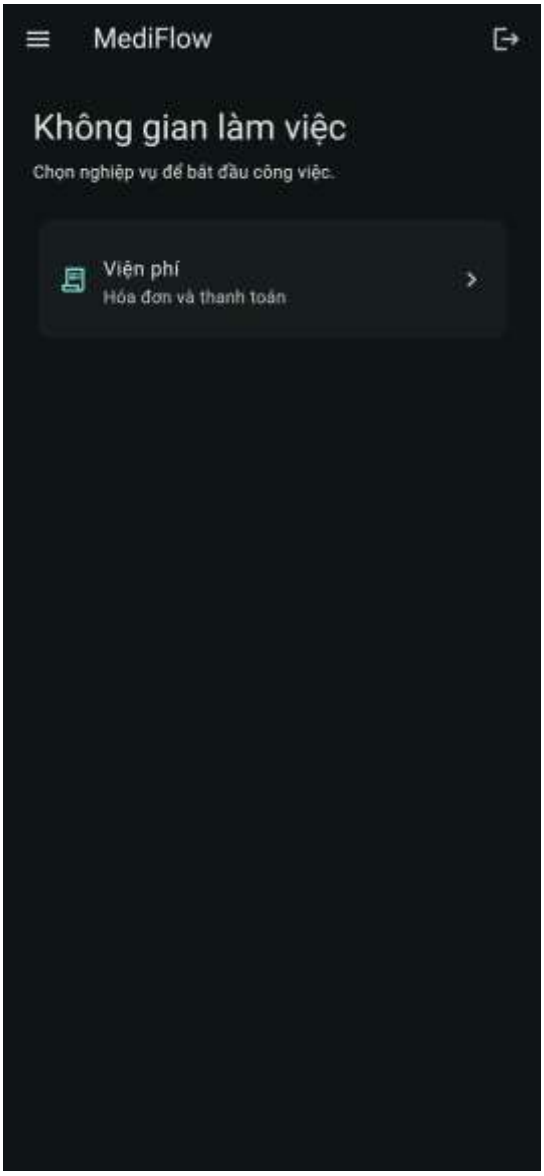
2.7.7. Dispense is rejected for the unpaid prescription without current Billing clearance.



This demonstrates the payment guard; no dispense success is claimed.

2.8. Cashier screens

2.8.1. Cashier workspace provides invoice and payment workflows.



2.8.2. Invoice workspace keeps reception references collapsed in normal use.

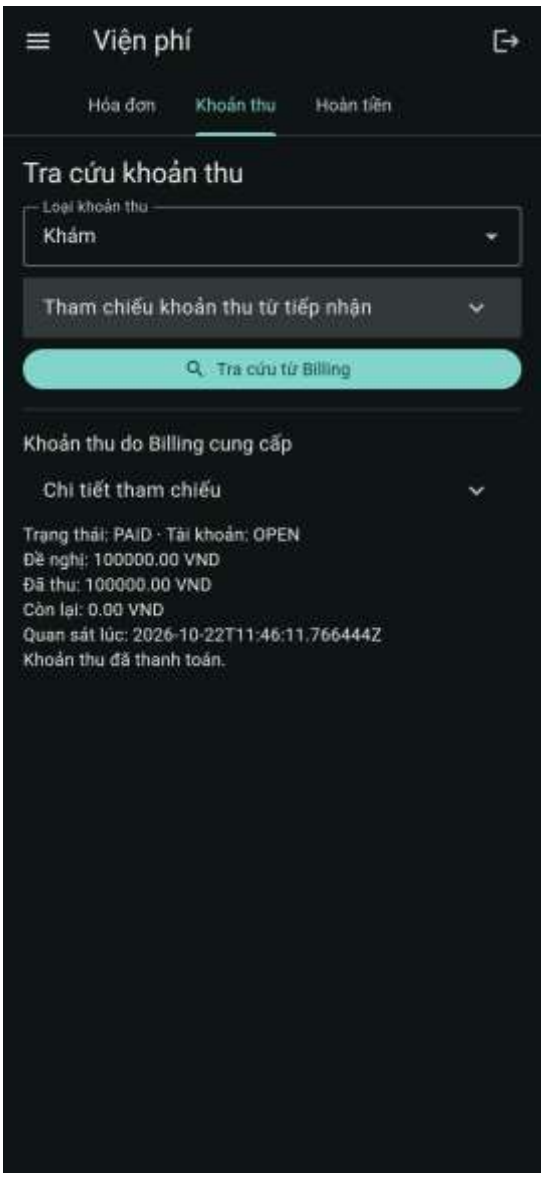


2.8.3. Cashier selects the charge type and looks up the reception reference.

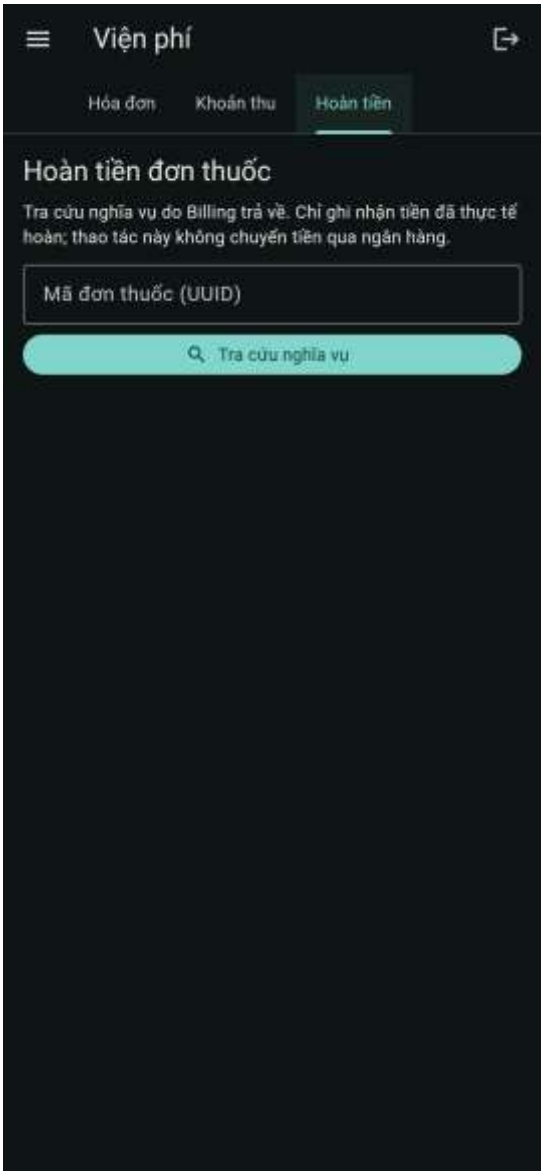


This screenshot shows the lookup workspace, not a pending payment result.

2.8.4. The completed demo payment is confirmed by Billing: PAID, collected 100,000 VND, balance zero.



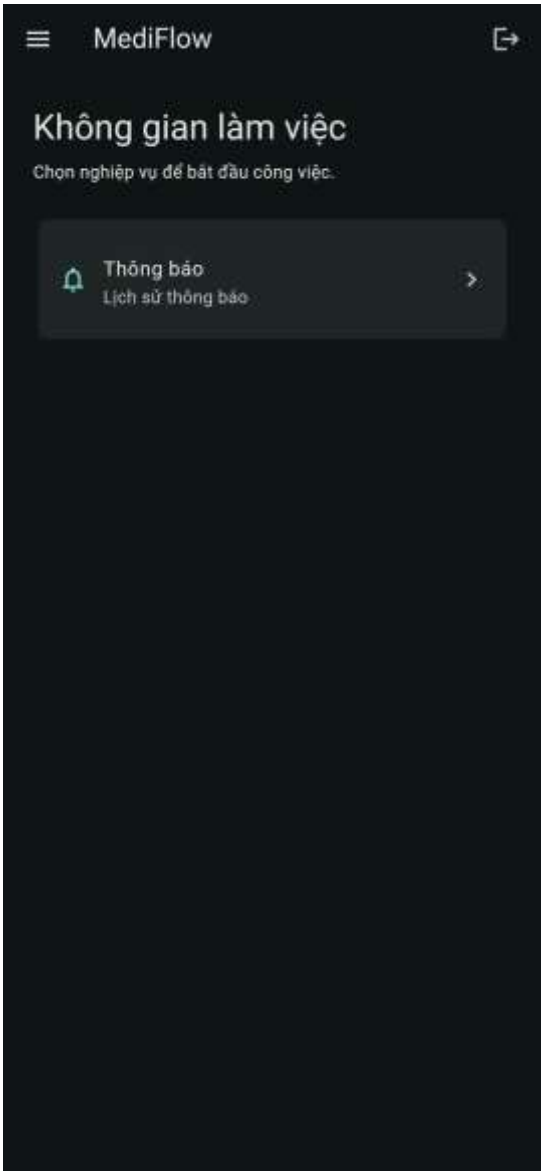
2.8.5. Refund workspace explains reconciliation and provides obligation lookup.



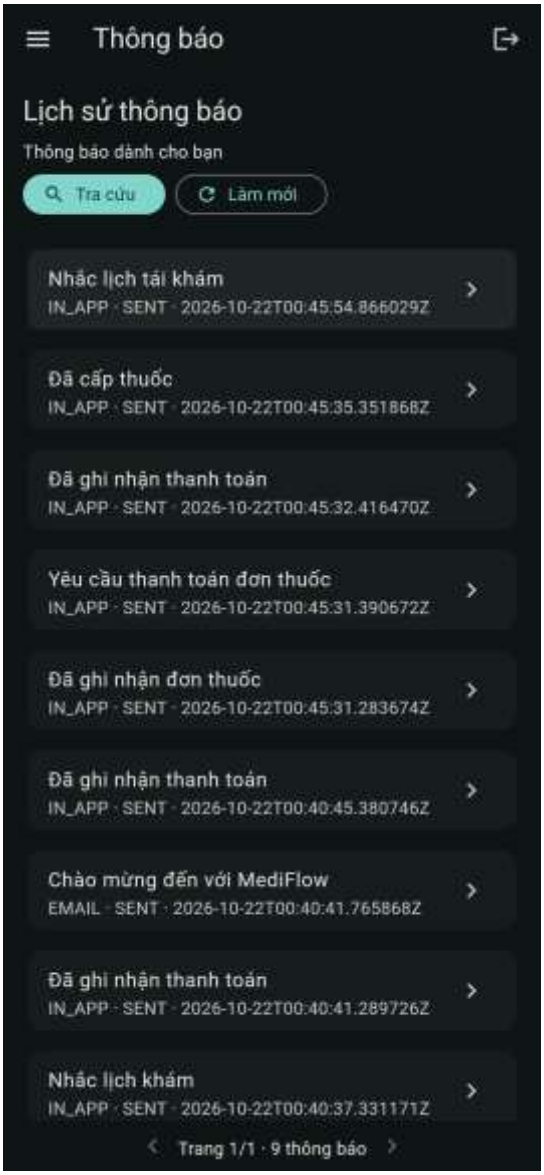
No refund is executed in this screenshot.

2.9. Patient

2.9.1. Patient workspace contains the authorized personal notification workflow.



2.9.2. The patient sees personal notification history resolved from the signed-in identity.

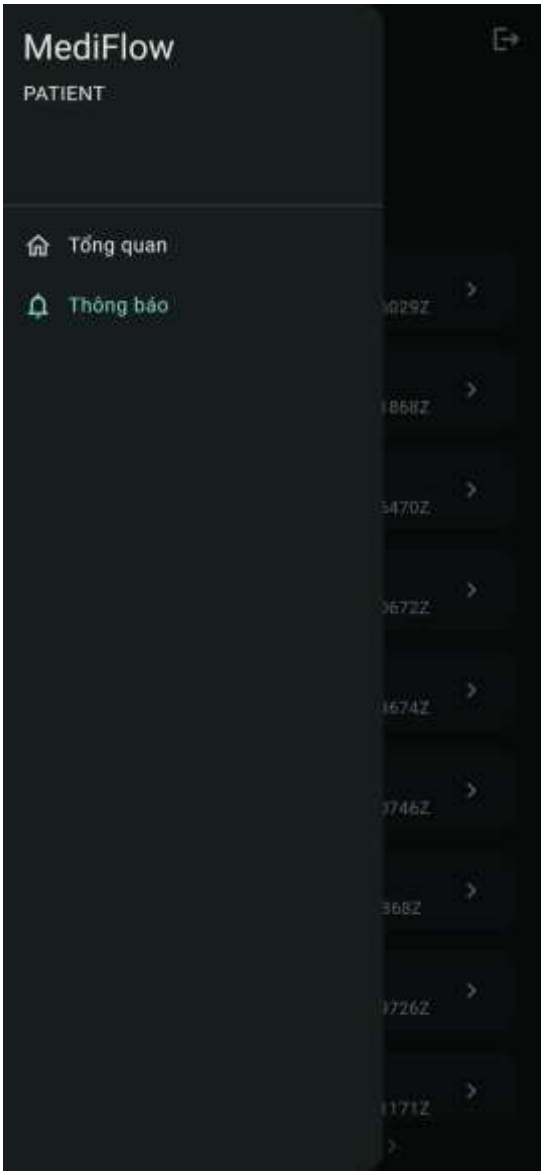


2.9.3. Patient opens an authorized notification detail with delivery state and collapsed technical identifiers.



Recipient lookup is restricted for this role; address and retry data are not exposed by this API.

2.9.4. Patient navigation only exposes authorized destinations.



Testing document

No.	Feature Tested	Remarks
1	Authentication — sign in with a valid username and password.	PASS
2	Authentication — reject invalid credentials and validate required login fields.	PASS
3	Authentication — reject locked or inactive accounts.	PASS
4	Session management — sign out, clear the session and prevent access to protected pages after logout.	PASS
5	Session management — handle expired or invalid tokens and require authentication again.	PASS
6	Authorization — enforce access for Administrator, Manager, Doctor, Nurse, Laboratory Technician, Pharmacist, Cashier and Patient.	PASS
7	Authorization — reject direct access to restricted routes and API operations, including manually entered URLs.	PASS
8	Authorization — prevent a patient from reading another patient's protected information.	PASS
9	Account management — create a staff-linked account and assign a permitted role.	PASS

No.	Feature Tested	Remarks
10	Account management — reject duplicate usernames and invalid identity references.	PASS
11	Account management — update account status and verify the effect of role or account-status changes on authorization.	PASS
12	Department management — list, search and view department details.	PASS
13	Department management — create or update department information with required-field validation.	PASS
14	Department management — validate department type, active status and department-head eligibility.	PASS
15	Staff management — list, search and view staff details.	PASS
16	Staff management — create or update staff information, including identity number, contact information and professional fields.	PASS
17	Staff management — validate duplicate identity information and incompatible job titles or assignments.	PASS
18	Staff management — transfer staff between departments while preserving a valid current assignment.	PASS
19	Patient management — list, search, paginate and open patient records.	PASS
20	Patient management — register a patient using valid demographic and contact information.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

No.	Feature Tested	Remarks
21	Patient management — update patient information and reload the persisted values.	PASS
22	Patient management — validate required fields, dates and identifiers, and handle duplicate identities according to the contract.	PASS
23	Appointment management — list appointments and filter by supported date, status or department criteria.	PASS
24	Appointment management — create an appointment with valid patient, doctor, department, date and time references.	PASS
25	Appointment management — reject invalid references, invalid scheduling data and prohibited scheduling conflicts.	PASS
26	Appointment management — view appointment details and its current care lifecycle.	PASS
27	Appointment management — update or cancel an appointment only when its current state permits the operation.	PASS
28	Reception — check in a patient and display the updated appointment state.	PASS
29	Examination payment gate — prevent examination from starting before the exact examination charge is cleared.	PASS
30	Examination payment gate — allow examination only after current Billing authorization for the correct patient and care episode.	PASS

No.	Feature Tested	Remarks
31	Examination — start an examination and create the associated medical record atomically.	PASS
32	Examination — handle repeated or concurrent start requests without creating duplicate medical records.	PASS
33	Medical records — list or search supported records and open the correct patient's record.	PASS
34	Medical records — record clinical information, vital signs and diagnosis using the supported fields.	PASS
35	Medical records — preserve changes after reload and reject invalid updates or unauthorized modifications.	PASS
36	Medical records — complete an outpatient examination only when its business prerequisites are satisfied.	PASS
37	Laboratory ordering — select a supported laboratory test and create an order from a valid clinical record.	PASS
38	Laboratory ordering — preserve source order, patient, requesting department and care-episode references.	PASS
39	Laboratory queue — list, paginate, filter and open laboratory requests.	PASS
40	Laboratory payment gate — create one charge for a laboratory request despite duplicate event delivery.	PASS

No.	Feature Tested	Remarks
41	Laboratory payment gate — reject execution without valid, current authorization for the exact test.	PASS
42	Laboratory execution — start an eligible paid test and persist the execution state.	PASS
43	Laboratory results — enter indicators, values, units, reference ranges and a conclusion.	PASS
44	Laboratory results — reject invalid result data and prohibited changes to terminal results.	PASS
45	Laboratory results — publish completed results and expose them to the authorized clinical workflow.	PASS
46	Laboratory cancellation — apply permitted cancellation without reopening the source or duplicating financial adjustments.	PASS
47	Drug catalogue — list, search, paginate and view drug details.	PASS
48	Drug catalogue — validate supported drug information and active availability for prescribing.	PASS
49	Inventory — record a supported stock increase and reload the resulting stock balance.	PASS
50	Inventory — reject invalid quantities and prevent negative available stock.	PASS

No.	Feature Tested	Remarks
51	Prescription management — create a prescription linked to the correct patient, clinical record and care episode.	PASS
52	Prescription management — validate drug selection, quantity, dosage, frequency and usage instructions.	PASS
53	Prescription pricing — use server-authoritative prices and retain the accepted price snapshot.	PASS
54	Prescription reservation — handle insufficient stock for a multi-drug prescription without partial reservation effects.	PASS
55	Prescription lookup — find a prescription by its supported identifier and display its lines and lifecycle state.	PASS
56	Prescription payment gate — require current authorization for the exact prescription before dispensing.	PASS
57	Dispensing — confirm an eligible prescription and record the fulfilled state and stock movement.	PASS
58	Dispensing — handle repeated commands or duplicate payment events without double dispensing or double stock deduction.	PASS
59	Prescription lifecycle — handle cancellation, expiry and late payment without reopening a terminal prescription.	PASS
60	Billing discovery — locate examination, laboratory and prescription payment requests using the correct source and care-episode references.	PASS

No.	Feature Tested	Remarks
61	Billing calculation — display requested, paid and remaining amounts with the correct currency and decimal handling.	PASS
62	Payment collection — record a valid supported payment and return the payment receipt.	PASS
63	Payment collection — reject zero, negative, excessive or otherwise invalid payment amounts according to the contract.	PASS
64	Payment idempotency — repeat the same payment intent without creating an additional financial effect.	PASS
65	Billing refresh — reload the authoritative paid status and remaining amount after a successful payment.	PASS
66	Billing separation — distinguish legacy invoices from outpatient payment requests and ledger records.	PASS
67	Billing allocation — prevent over-allocation and allocation to the wrong patient or care account.	PASS
68	Adjustments — apply a permitted cancellation or prescription adjustment with its original source and audit references.	PASS
69	Refunds — calculate refundable amounts, record an authorized refund and prevent over-refunding or duplicate refund effects.	PASS
70	Refund and execution concurrency — serialize examination or laboratory start with refund and cancellation decisions.	PASS

No.	Feature Tested	Remarks
71	Financial audit — preserve original payment links, histories and accepted writes through retries.	PASS
72	Reports — open the supported report workspace and apply supported reporting-period and department filters.	PASS
73	Reports — distinguish cash collection, earned revenue, adjustments and refunds according to the report contract.	PASS
74	Reports — process repeated events without counting the same business contribution twice.	PASS
75	Reports — distinguish an unavailable or uncovered report from a verified zero-value report.	PASS
76	Reports — retain stable acceptance ordering and detect conflicting event content during replay.	PASS
77	Notifications — list and open in-app notifications for the authenticated recipient.	PASS
78	Notifications — display appointment reminders and supported care-related notification details.	PASS
79	Notifications — prevent cross-recipient access and duplicate delivery effects during event replay.	PASS
80	Identity presentation — display the correct patient, doctor and department names instead of using UUID as the only visible label.	PASS

No.	Feature Tested	Remarks
81	Identity presentation — retain the original identifiers in commands and show an honest fallback when a permitted name lookup fails.	PASS
82	Web GUI — display shared landing, login, navigation and access-denied pages.	PASS
83	Web GUI — provide role-appropriate menus, page links and visible actions.	PASS
84	Web GUI — handle loading, empty, validation, not-found and request-error states with supported retry behavior.	PASS
85	Web GUI — maintain readable forms, lists and dialogs at the supported desktop and narrow-screen sizes.	PASS
86	Web GUI — preserve readable text, contrast, keyboard access and labels in supported themes.	PASS
87	Mobile GUI — authenticate, retain the permitted session and sign out through the configured Gateway.	PASS
88	Mobile GUI — display the navigation and workspace appropriate to each of the eight roles.	PASS
89	Mobile GUI — browse supported department, staff and account screens.	PASS
90	Mobile GUI — browse, search and view patients and use supported patient forms.	PASS

No.	Feature Tested	Remarks
91	Mobile GUI — browse appointments, change the supported date filter and open the available appointment workflow.	PASS
92	Mobile GUI — browse laboratory requests and filter by the supported lifecycle status.	PASS
93	Mobile GUI — integrate supported clinical-record, pharmacy and report workspaces with their actual repositories.	PASS
94	Mobile GUI — discover a payment request, submit a valid payment and refresh its authoritative balance.	PASS
95	Mobile GUI — list and open the authenticated patient's notifications.	PASS
96	Mobile GUI — handle unavailable workspaces, connection failures and small-screen scrolling without presenting unfinished features as completed.	PASS
97	Session isolation — discard stale responses and cached identity information after a record change, logout or account switch.	PASS
98	Gateway integration — send web and mobile business requests through the Gateway with the required authentication.	PASS
99	Gateway security — reject unsupported nested routes, wrong-role requests and invalid token types.	PASS
100	Integration — propagate examination and laboratory charge requests and prescription payment events between the intended contexts.	PASS

No.	Feature Tested	Remarks
101	Event reliability — use durable outbox/inbox processing and preserve event identity during retry and redelivery.	PASS
102	Event reliability — handle poison messages, bounded retry, dead-letter routing and unroutable publication.	PASS
103	Recovery — resume pending pharmacy work after service interruption without losing or duplicating its business effect.	PASS
104	Recovery — preserve accepted records and financial effects across database, broker and application restart.	PASS
105	Transaction integrity — roll back failed multi-step operations without leaving partial clinical, stock or financial state.	PASS
106	Core reliability — handle repeated staff-creation requests and event publication failures without duplicate identities.	PASS
107	Deployment — start Core, Clinical, Lab, Pharmacy, Billing and Support with Gateway and Eureka.	PASS
108	Data boundaries — retain separate context-owned stores and use supported API/event contracts for cross-context data.	PASS
109	Backup and rollback — validate restoration or current-store rollback while retaining accepted writes and audit history.	PASS
110	Scope control — keep deferred inpatient and surgery routes, jobs and business actions outside the outpatient MVP acceptance scope.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

Final checklist

No.	Review 01 aspect tested	Suggestion / Remarks
1	Project title and problem statement describe outpatient care management consistently.	PASS
2	Project objectives identify the intended users and the operational problems addressed by MediFlow.	PASS
3	Project scope includes the eight active business contexts and identifies inpatient care and surgery as deferred.	PASS
4	Functional requirements cover the responsibilities and access boundaries of all eight user roles.	PASS
5	Non-functional requirements address security, reliability, usability, maintainability and operational recovery.	PASS
6	Use-case diagrams and descriptions match the active role workflows and business scope.	PASS
7	The architecture distinguishes six business deployments, eight business contexts, infrastructure and shared libraries.	PASS
8	Service boundaries, responsibilities and dependencies are consistent across the architecture and implementation documents.	PASS
9	Sequence diagrams describe appointment, examination, laboratory, prescription, payment and dispensing interactions.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

No.	Review 01 aspect tested	Suggestion / Remarks
10	Class and component diagrams reflect the intended responsibilities and dependency direction.	PASS
11	Data-flow diagrams describe the main clinical, stock, financial, reporting and notification information flows.	PASS
12	Entity relationship diagrams and table descriptions match context-owned persistence boundaries.	PASS
13	Database identifiers, field types, constraints and migrations are documented consistently.	PASS
14	No cross-context database relationship or direct database lookup is introduced to replace a public service contract.	PASS
15	API contracts define supported request fields, response fields, validation rules and stable errors.	PASS
16	Gateway routing, authentication, session handling and role authorization match the intended access model.	PASS
17	Account, department, staff and patient management cover the required outpatient identity information.	PASS
18	Appointment booking, reception, examination eligibility and clinical-record creation form a consistent care lifecycle.	PASS

No.	Review 01 aspect tested	Suggestion / Remarks
19	Laboratory ordering, payment authorization, execution, results and cancellation follow the documented lifecycle.	PASS
20	Prescription pricing, reservation, payment authorization and dispensing preserve stock and financial consistency.	PASS
21	Billing handles charge discovery, payments, balances, adjustments and refunds with the required audit links.	PASS
22	Reports distinguish covered data, unavailable data, cash collection and earned activity according to the contract.	PASS
23	In-app notifications are addressed to the correct recipients and preserve recipient-only access.	PASS
24	Idempotency, concurrency control and transaction boundaries prevent duplicate or partial business effects.	PASS
25	Event publication, delivery, replay and failure handling preserve durable business state.	PASS
26	Web navigation and visible actions are appropriate for Administrator, Manager, Doctor, Nurse, Laboratory Technician, Pharmacist, Cashier and Patient.	PASS
27	Shared web pages, forms, validation feedback and error states use consistent presentation.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

No.	Review 01 aspect tested	Suggestion / Remarks
28	Patient, doctor and department references display readable names where lookup is available and authorized.	PASS
29	Web GUI layouts remain readable at supported viewport sizes and in supported themes.	PASS
30	Mobile architecture, Gateway access, session handling and navigation follow the Flutter project rules.	PASS
31	Mobile role workspaces are evaluated individually, including supported screens and unfinished or unavailable areas.	PASS
32	Mobile forms, lists, filters, details and feedback are usable at a phone-sized viewport.	PASS
33	Native Android/iOS execution evidence is identified separately from Flutter web evidence.	PASS
34	Demonstration data includes linked accounts, staff, departments, patients, appointments, clinical records, tests, prescriptions and charges.	PASS
35	Seed data is confined to the isolated demo environment and does not fabricate medical results or financial authorization.	PASS
36	Backend, web and mobile verification records identify the tested revision, commands, passed cases, failures and skips.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

No.	Review 01 aspect tested	Suggestion / Remarks
37	Integration and recovery evidence uses the required real database and broker rather than relying only on mocked tests.	PASS
38	Screenshots show common pages, role-specific pages and representative interactions for web and mobile.	PASS
39	Screenshot captions, numbering and references match the corresponding role, platform and observed runtime state.	PASS
40	Known issues and unfinished features are recorded in handoffs with owners and observable acceptance criteria.	PASS
41	The Testing document covers valid, invalid, unauthorized, duplicate, concurrent and failure scenarios.	PASS
42	The final checklist aligns the report with the approved MVP instead of claiming completion of deferred features.	PASS
43	Build, configuration and startup instructions describe the environment needed to reproduce the demonstration.	PASS
44	Database persistence, backup and restart instructions allow the demo data to be retained for later review.	PASS
45	The final report uses consistent terminology, readable figures, section titles and page references.	PASS

	Prepared by (Student)	
	Project Group No: 04	
Signature		
Date		

No.	Review 01 aspect tested	Suggestion / Remarks
46	Personal credentials, access tokens and private runtime manifests are excluded from shared report assets and source control.	PASS
47	Requirements, implementation, test evidence and screenshots can be traced to the same supported workflow.	PASS
48	Actual acceptance decisions are reconciled against evidence before replacing this pre-filled template with a signed final result.	PASS

Tasksheet Review 3

Project Ref. No		Project Title	Activity Plan Prepared By	Date of Preparation of Activity Plan			
No.	Task			Actual Start Date	Actuals Days	Teammate Names	Status
1	Screenshots	Mediflow	Quang Huy	20/10/2026	2	Hoang Anh	Completed
2	Testing document			20/10/2026	2	Quang Huy	Completed
3	Final checklist			20/10/2026	2	Dang Vinh	Completed

	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

4	Tasksheet review			20/10/2026	2	The Loc	Completed
Project Detail Tasksheet							
1	Core: accounts, staff, departments/rooms, patients, authentication, and authorization.	MediFlow	Vinh	14/10/2026	15	Hoàng Anh	Completed
2	Gateway: routing, JWT authentication, role-based access control, and lookup APIs.	MediFlow	Vinh	02/10/2026	27	Hoàng Anh	Completed
3	Clinical: appointments, patient reception, medical records, diagnoses, and laboratory orders.	MediFlow	Vinh	10/10/2026	15	Vinh	Completed

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

4	Lab: order reception, test results, laboratory workflow management, and payment eligibility checks.	MediFlow	Vinh	02/10/2026	16	Vinh	Completed
5	Pharmacy: medicine catalogue, inventory, prescriptions, dispensing/cancellation, and dispensing eligibility checks.	MediFlow	Vinh	02/10/2026	20	Huy	Completed
6	Report and Support: reporting, data coverage controls, service hosting consolidation, and separate data stores.	MediFlow	Vinh	02/10/2026	15	Huy	Completed

7	Billing/Notification: foundational contributions to payment processing, payment request discovery, event publication/recovery, and notifications.	MediFlow	Vinh	06/10/2026	10	Lộc	Completed
8	Billing/Notification: backend takeover, security improvements, event replay, preservation of original event bytes, and regression testing.	MediFlow	Vinh	03/10/2026	20	Vinh	Completed
9	Shared Web: login, session management, role-based navigation, responsive layouts, and name-based lookups.	MediFlow	Vinh	08/10/2026	20	Vinh; Huy; Hoàng Anh	Completed

10	Web Organization/Patient: lists, search, details, forms, and account management.	MediFlow	Vinh	02/10/2026	20	Hoàng Anh	Completed
11	Web Clinical/Lab: appointments, patient reception, medical records/diagnoses, laboratory orders, and test results.	MediFlow	Vinh	03/10/2026	15.	Vinh	Completed
12	Web Pharmacy/Report: medicines, inventory, prescriptions/dispensing, filtering, and report queries.	MediFlow	Vinh	14/10/2026	16.	Huy	Completed
13	Web Billing/Notification: payments, revenue reports, refunds/adjustments, event outbox management, and notifications.	MediFlow	Vinh	15/10/2026	7	Lộc	Completed

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: 04	HOÀNG ĐỨC QUANG
Signature		
Date		

14	Shared Mobile and outpatient workspaces, including Organization/Patient and Billing/Notification features added under the handover scope.	MediFlow	Vinh	17/10/2026	10	Vinh	Completed
15	Mobile Pharmacy/Report: medicine catalogue, prescriptions, dispensing actions, and report queries.	MediFlow	Vinh	20/10/2026	n.a.	Huy	Completed

Acknowledgments and Declaration

Project Documentation for Mediflow

Completed on 30/10/2025

The documentation for the **Mediflow** project was completed on November 30, 2026. Our team would like to express our gratitude to our supervisor, **Mr. HOÀNG ĐỨC QUANG**, for his guidance and valuable feedback throughout the project. We also appreciate the constructive input from our fellow students, which contributed to the refinement of our work.

We also thank **Project Aptech** for providing us with the opportunity to work on this real-world project, which helped us apply our academic knowledge in a practical scenario.

This document is the result of our team's efforts, based on the knowledge we have gained during our studies and additional research. We confirm that the content is original and accurate, with no unauthorized copying or plagiarism. Our team is committed to maintaining academic integrity and ensuring the quality of this project.

This is a complete document that reflects the efforts and research of our team as students who are still learning and have not yet entered the professional workforce. We are grateful to our teacher for his support throughout this project. The content is based on the knowledge we have acquired during our studies and reliable sources.